



SRH University Heidelberg

Master's Thesis

Time Series Classification for Paper Web Breaks in the Print Industry

Vaishnavi Sreekumar

Matriculation Number: 100002862

Email: 100002862@stud.srh-university.de

Supervisor 1: Prof. Dr. Achim Gottscheber

Supervisor 2: Dr. Paul Glaysher

April 2026

Abstract

Web breaks are sudden ruptures of the paper web during high-speed offset printing and are the leading cause of unplanned production downtime in commercial printing facilities. Determining whether a break originated from an operational fault or a material defect is critical yet currently performed manually by press operators in a time-consuming process. This thesis investigates automated web break cause classification using 14,073 labelled break events from an active Bertelsmann printing facility in Germany, represented as a 300-frame multivariate time series of camera-extracted defect probability scores combined with operational metadata. Six distinct architectures, the best one of which was additionally evaluated with early metadata fusion, a GPT-4o large language model classifier, and a domain-informed XGBoost pipeline were evaluated under a unified stratified cross-validation protocol; Chronos-2 was evaluated separately under a temporal split to respect its pre-training design. XGBoost with engineered temporal features and operational metadata achieves a mean test AUC of 0.86, outperforming TapNet, the best amongst all deep learning models that were evaluated, with the same metadata as of XGBoost by 0.050 AUC points. An ablation study reveals that metadata contributes more when accessed through explicit feature engineering (+0.070 AUC) than through direct fusion into a deep learning model (+0.044 AUC). The LLM and foundation model approaches both underperform, with the LLM performance attributed to an architectural inability to execute quantitative threshold logic.

Kurzfassung

Papierbahnabrisse sind plötzliche Risse in der Papierbahn während des Hochgeschwindigkeits-Offsetdrucks und stellen die häufigste Ursache für ungeplante Produktionsstillstände in kommerziellen Druckereien dar. Die Feststellung, ob ein Abriss auf einen Betriebsfehler oder einen Materialdefekt zurückzuführen ist, ist von entscheidender Bedeutung, wird jedoch derzeit manuell von Druckmaschinenbedienern in einem zeitaufwendigen Prozess durchgeführt. Diese Arbeit untersucht die automatisierte Klassifikation der Ursachen von Papierbahnabrisen anhand von 14.073 gelabelten Abrissereignissen aus einer aktiven Bertelsmann-Druckerei in Deutschland. Die Daten werden als multivariate Zeitreihen mit 300 Frames dargestellt, bestehend aus von Kameras extrahierten Defektwahrscheinlichkeitswerten, kombiniert mit Betriebsmetadaten. Sechs unterschiedliche Architekturen wurden unter einem einheitlichen, stratifizierten Kreuzvalidierungsprotokoll evaluiert; das beste Modell wurde zusätzlich mit früher Metadatenfusion bewertet, ebenso wie ein GPT-4o Large-Language-Model-Klassifikator und eine domänenspezifisch informierte XGBoost-Pipeline. Chronos-2 wurde separat unter Verwendung eines zeitlichen Splits evaluiert, um seinem Vortrainingsdesign gerecht zu werden. XGBoost mit konstruierten zeitlichen Merkmalen und Betriebsmetadaten erreicht eine durchschnittliche Test-AUC von 0,86 und übertrifft damit TapNet, das beste der evaluierten Deep-Learning-Modelle, bei Verwendung derselben Metadaten um 0,050 AUC-Punkte. Eine Ablationsstudie zeigt, dass Metadaten einen größeren Beitrag leisten, wenn sie über explizites Feature Engineering genutzt werden (+0,070 AUC), als durch direkte Fusion in ein Deep-Learning-Modell (+0,044 AUC). Sowohl der LLM als auch der Foundation-Model-Ansatz schneiden schlechter ab, wobei die Leistung des LLM auf eine architekturbedingte Unfähigkeit zurückgeführt wird, quantitative Schwellenlogik auszuführen.

Contents

Abstract	i
Kurzfassung	ii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Research Questions	2
1.4 Contributions	3
1.5 Thesis Structure	5
2 Background and Related Work	6
2.1 Web Offset Printing	6
2.2 Time Series Classification	7
2.3 Class Imbalance	14
2.4 Related Work on Web Break Analysis	14
2.4.1 Cause Classification versus Fault Diagnosis	15
2.4.2 Deep Learning for Industrial Time Series Classification	16
2.4.3 Research Gap	17
3 Dataset	18
3.1 Data Source and Collection	18
3.2 Camera System and Frame Data	18
3.3 Data Structure	21
3.4 Time Series Representation	22
3.4.1 Justification of the 300-Frame Window	23
3.4.2 Class-Specific Temporal Signatures	24
3.5 Operational Metadata	25

3.6	Data Quality and Class Distribution	27
3.7	Dataset Summary	28
4	Methodology	30
4.1	Problem Formulation	30
4.2	Preprocessing Pipeline	30
4.2.1	Frame-Level Validation and Camera Selection	31
4.2.2	Tear Peak Alignment and Temporal Window Extraction	31
4.2.3	Multi-Camera Sequence Construction	32
4.2.4	Dataset Construction	33
4.2.5	Feature Scaling	33
4.3	Feature Engineering for XGBoost	33
4.4	Evaluation Design	37
4.4.1	Cross-Validation Strategy	37
4.4.2	Evaluation Metrics	37
4.4.3	Threshold Selection	38
4.5	Experimental Pipelines	38
4.5.1	End-to-End Deep Learning Pipeline	38
4.5.2	XGBoost Pipeline	41
4.5.3	LLM Classification Pipeline	41
4.5.4	Chronos Foundation Model	43
4.6	Model Selection Rationale	44
4.7	Handling Class Imbalance	45
5	Results	46
5.1	Experimental Setup	46
5.2	Overview of Results	46
5.3	Convolutional Models	48
5.3.1	CNN	48
5.3.2	InceptionTime	48
5.3.3	ConvTimeNet	49
5.4	Recurrent Models	50
5.4.1	LSTM-FCN	50
5.5	Attention-Based Models	51
5.5.1	TST	51

5.6	Prototype-Based Models	52
5.6.1	TapNet	52
5.6.2	TapNet with Early Metadata Fusion	53
5.7	Foundation Model - Chronos	54
5.8	LLM Classifier	55
5.9	XGBoost - Feature Engineering Pipeline	56
5.9.1	Full Feature Set	56
5.9.2	Ablation Study - Metadata Contribution	57
5.9.3	Feature Importance Analysis	59
5.10	Summary and Comparison	61
6	Discussion	63
6.1	Interpretation of Results: The Performance Hierarchy	63
6.2	Performance of Local Pattern Detection Models	63
6.2.1	CNN	64
6.2.2	ConvTimeNet	64
6.2.3	InceptionTime	65
6.2.4	TST	65
6.3	Why Sequential Memory Helps but is Insufficient: LSTM-FCN	66
6.4	Why TapNet performs better amongst all the Deep Learning Models . .	67
6.4.1	Prototype-Based Classification	67
6.4.2	Weighted Cross-Entropy Loss	67
6.4.3	Why TapNet Still Cannot Close the Gap to XGBoost	68
6.5	Why Domain Knowledge Extraction performed better: XGBoost vs Deep Learning	68
6.5.1	Data Efficiency: The Feature Engineering Advantage	68
6.5.2	The Ablation: Isolating the Metadata Contribution	69
6.5.3	Why Metadata is So Informative	69
6.6	Why Chronos underperformed: Forecasting Pre-Training on a Classification Task	69
6.7	Why the LLM Underperformed: An Architectural Mismatch	70
6.8	The Role of Class Imbalance	71
6.9	Industrial Relevance and Deployment Pathway	72
6.9.1	Operational Significance of the XGBoost Result	72
6.9.2	Proposed Deployment Architecture	73

6.9.3	Prerequisites Before Deployment	74
6.10	Limitations and Threats to Validity	75
6.10.1	Limitations	75
6.10.2	Threats to Validity	75
7	Conclusion	77
7.1	Summary of the Thesis	77
7.2	What Did Not Work Well	78
7.3	Answers to Research Questions	79
7.4	Future Work	80

List of Figures

2.1	FCN architecture as implemented via <code>sktime FCNClassifier</code> with default parameters. Adapted from [WYO17].	7
2.2	InceptionTime inception module with bottleneck, parallel multi-scale convolutions, and residual connections. Adapted from [F ⁺ 20].	8
2.3	ConvTimeNet architecture with deformable patch embedding and hierarchical depthwise convolutional encoder. Adapted from [C ⁺ 24].	9
2.4	LSTM-FCN architecture with parallel LSTM and fully convolutional branches concatenated before classification. Adapted from [KMDH19]. .	10
2.5	Time Series Transformer with multi-head self-attention, positional encoding, and stacked encoder layers. Adapted from [Z ⁺ 21].	11
2.6	TapNet architecture with random dimension permutation, parallel LSTM and CNN encoders, and attentional prototype classification. Adapted from [Z ⁺ 20].	12
3.1	Multi-angle camera views of the paper web passing through the press during operation.	19
3.2	Camera frame data availability across all 14,083 events.	19
3.3	Map of an event JSON file	22
3.4	Web break event analysis	23
3.5	Window size comparison across 150, 300 and 500 frames	24
3.6	Per-timestep absolute class separation across window sizes	24
3.7	Average defect score evolution by cause class	25
3.8	Metadata field coverage across all 14,083 events. Bars show the percentage of events in which each field is populated.	26
3.9	Class distribution by press machine for the ten most frequent presses . .	26
3.10	Distribution of key numeric metadata fields by cause class. Visual overlap between classes suggests no single field provides clean separation. . . .	27
3.11	Dataset composition: (left) funnel showing how the final 14,083 events were derived from 15,843 raw records; (right) class distribution showing the 84/16 split for the full dataset and the 80/20 split for the tear subset	28

5.1	Mean test ROC-AUC across all models with standard deviation error bars.	47
5.2	Per-fold test AUC results for all deep learning models. Diamond markers show mean AUC across three folds.	54
5.3	LLM classification results: metric comparison and confusion matrices for both configurations.	55
5.4	Top 30 features by normalised mean gain importance from the XGBoost full model, averaged across three cross-validation folds. Green bars indicate time-series-derived features, orange bars indicate categorical metadata features, and blue bars indicate numeric metadata features. . .	59
5.5	The first decision tree (tree 0) from the XGBoost ensemble trained on fold 1. Split thresholds are in standardised units following StandardScaler preprocessing. The root split on <code>cam1_max_tear</code> is consistent with the feature importance analysis. Camera time series features dominate the upper levels of the tree while metadata fields (<code>pap_len</code> , <code>speed</code> , <code>grammage_weight</code> , <code>paper grade</code>) appear at deeper levels. Green leaves indicate a positive score contributing towards paper-caused classification and red leaves a negative score towards machine-caused classification. The final prediction is the sigmoid-transformed sum of scores across all trees in the ensemble.	61
6.1	Top 30 features by normalised mean gain importance from the XGBoost full model, averaged across three cross-validation folds. Green bars indicate time-series-derived features, orange bars indicate categorical metadata features, and blue bars indicate numeric metadata features. . .	73

List of Tables

3.1	Top-level metadata field in each event JSON file	21
3.2	Dataset summary	29
5.1	Test ROC-AUC results for all 3-fold CV models ordered by mean AUC. XGBoost TS-only and TapNet + meta are ablation/extension experiments. Chronos (temporal split) and LLM (100-event sample) use different protocols and are reported in Sections 5.7 and 5.8.	47
5.2	CNN per-fold classification results at F1-optimised threshold.	48
5.3	InceptionTime per-fold classification results at F1-optimised threshold.	49
5.4	LSTM-FCN per-fold classification results at F1-optimised threshold.	50
5.5	TST per-fold classification results at F1-optimised threshold.	51
5.6	TapNet per-fold classification results at F1-optimised threshold.	52
5.7	TapNet + metadata (early fusion, input shape (300, 18)) per-fold classification results at F1-optimised threshold.	53
5.8	Chronos results under temporal split evaluation	54
5.9	LLM classifier results on 100 held-out events. Confusion matrices: v3 - machine [59 correct, 30 wrong], paper [5 correct, 6 missed]; v5 - machine [69 correct, 20 wrong], paper [2 correct, 9 missed].	55
5.10	XGBoost full (TS + metadata) per-fold classification results at F1-optimised threshold.	56
5.11	XGBoost TS-only (no metadata) per-fold classification results at F1-optimised threshold.	57
5.12	Metadata contribution comparison across both model families. The same eight metadata fields are used in both TapNet early fusion and XGBoost full.	57
5.13	Summary of all models and configurations. *Chronos uses temporal split evaluation (not comparable to 3-fold CV). †LLM evaluated on 100-event sample without probability outputs. ConvTimeNet threshold-based metrics unavailable due to implementation constraints.	62

6.1	Metadata contribution across both model families. The same eight meta-data fields are used in TapNet early fusion and XGBoost full. Δ AUC is relative to the corresponding no-metadata baseline.	69
6.2	chronos results under temporal split evaluation (30/6/6 months, single-camera (300,5) input).	70
6.3	LLM classifier results on 100 held-out events. v5 provided the model with exact XGBoost-derived feature thresholds and domain reasoning rules in addition to more labelled examples.	71

Chapter 1

Introduction

1.1 Background

In the high-speed web offset printing industry, the press downtime represents a direct and measurable economic loss, making operational efficiency a major concern for printing facilities worldwide. A key performance metric used to define a good printing press is its runnability. A press with high runnability operates continuously with minimal interruptions. Among the various disruptions that affect press productivity, paper web breaks represent the most significant source of unplanned stoppages [Web13]. Each break requires the press to be stopped, the broken web removed, and the machine re-threaded before production can resume. This entire process takes around 20 to 40 minutes of unplanned downtime, disrupting production schedules and wasting materials already committed to the run [Web13].

The economic impact of web breaks accumulates significantly at scale. According to industry benchmarks, commercial presses experience typical break rates of 3-5% per 100 rolls, with each incident costing between €335 and €743 depending on press size and duration of stoppage [Web13]. These figures underscore that the productivity loss from even a modest break rate compounds rapidly across a full production year.

Web breaks typically result from the simultaneous occurrence of multiple factors, originating from either the printing machine or the paper itself. Press-side factors contributing to web breaks include web tension irregularities, thermal miscalibration in the drying system, mechanical wear, and insufficient blanket washing frequency, which allows ink and paper debris to accumulate on blanket surfaces, increasing tack and break risk [Web13]. Paper-related causes typically arise from inherent material defects such as tensile weakness, non-uniform moisture distribution, or splice failures introduced during paper manufacturing or handling. Distinguishing between these two causes is critical for the print industry yet this in practice remains a largely manual, time-consuming process.

Existing research on web breaks has largely focused on predicting their occurrence from process sensor data, treating the problem as one of anomaly detection or failure forecasting. While break prediction is an important line of work, it faces fundamental challenges: web breaks are inherently stochastic events, determined by the combined effect of tension variations and paper strength distributions rather than deterministic

cause-effect relationships [Ues04], making reliable anticipation difficult in practice. Predictive models are further constrained by severe class imbalance out of which breaks represent a small fraction of all recorded press events [AVH⁺09] and by the fact that up to 20–50% of breaks occur without any clearly attributable cause being recorded at the time [Web13], depriving prediction models of clean supervisory signal. Web breaks cannot be eliminated in practice; they remain an inherent risk of high-speed continuous printing. What is less studied, and arguably more actionable, is the classification of root cause once a break has occurred. Knowing whether a break originated from a machine fault or a material defect directly determines the corrective action: a machine-caused break triggers internal maintenance and press adjustment, while a paper-caused break warrants a supplier complaint and roll-level quality review [Web13]. Critically, this attribution decision must be made post-event, after the break has already stopped production, meaning prediction systems that flag elevated risk offer no help at the moment when the operational question actually arises. Cause classification, by contrast, operates on the recorded break event itself and produces the actionable label that prediction cannot. Despite its clear operational value, this cause classification problem has received less attention in the literature and remains an open research question.

1.2 Problem Statement

This thesis addresses the following classification problem: given a recorded web break event captured by a Custom Vision based monitoring system installed on an active web offset printing press, the objective is to determine whether the break was caused by a operational fault in the printing machine or a defect in the paper material. Each event is represented as a structured record containing event-level metadata, including press speed, time elapsed since the last blanket wash, paper grammage, web width, and paper type, alongside a multivariate time series of frame-level defect scores extracted from camera footage captured in the moments leading up to the break. Each frame is assigned a score across multiple defect categories, with the highest-scoring category serving as the frame-level label.

The task is formulated as a binary classification problem. All breaks attributable to paper-related defects are labelled as paper material-caused, while the remaining breaks are labelled as operational or machine-caused. This distinction carries direct operational significance, that is, a material-caused classification enables the printing facility to raise formal complaints with paper suppliers and pursue cost recovery, while a pattern of machine-caused breaks signals the need for targeted press maintenance. Currently, this attribution is performed manually by press operators, making it time-consuming, expertise-dependent, and not very consistent across shifts. The thesis aims to automate this process, producing a reliable and uniform classification that saves time and supports systematic quality improvement across the facility.

1.3 Research Questions

This thesis investigates web break cause classification through four research questions, each probing a different assumption about how discriminative information can be

extracted from the available data.

The first question establishes whether the task is solvable at all using standard deep learning applied directly to the raw temporal signal:

- **RQ1:** Can end-to-end deep learning models applied to preprocessed time series of frame-level defect scores reliably classify web break causes in an industrial web offset printing environment?

The second question asks whether domain-specific training can be avoided entirely by leveraging large pre-trained models:

- **RQ2:** Can general-purpose pre-trained models, specifically large language models and time series foundation models effectively classify web break causes without domain-specific training and does the performance stem from information loss during input encoding or from an architectural inability to execute quantitative numerical reasoning?

The third question challenges the end-to-end learning assumption directly, asking whether encoding expert knowledge of the break process explicitly is more effective than learning it from data:

- **RQ3:** Does domain-informed feature engineering that is encoding expert knowledge of the defect build-up pattern preceding a break outperform generic end-to-end deep learning approaches on this classification task?

The fourth question cuts across all modelling approaches to ask what role operational context plays and whether its utility depends on how it is consumed:

- **RQ4:** Does the inclusion of operational metadata improve classification performance, and does its utility vary across different modelling approaches?

Together, these questions move from asking whether the task is feasible (RQ1), to whether pre-trained representations transfer (RQ2), to whether domain knowledge outperforms learned representations (RQ3), to how structured operational context interacts with each modelling paradigm (RQ4). Answers to each research questions are discussed in Section 7.3

1.4 Contributions

The four research questions are addressed through the following contributions:

1. A real industrial dataset at operational scale

All experiments are grounded in 14,073 labelled web break events collected from an active Bertelsmann web offset printing facility over three and a half years of

continuous production. Unlike curated benchmark datasets, this data reflects the natural class imbalance, annotation noise, and metadata inconsistency of a genuine production environment characteristics that are critical for evaluating whether a classification system will generalise to real deployment conditions and that are underrepresented in published industrial fault classification research. This dataset forms the empirical foundation for all four research questions.

2. A systematic comparison of nine classification approaches on the same task and data

RQ1, RQ2, and RQ3 are addressed through a unified evaluation of seven deep learning architectures spanning convolutional, recurrent, attention-based, and prototype-based families; a large language model classifier operating on text-encoded defect sequences; a time series foundation model; and a domain-informed XGBoost pipeline, all evaluated on the same 14,073 events under the same cross-validation protocol. This breadth of comparison provides a principled basis for determining not just which approach performs best, but why each modelling paradigm succeeds or fails on this class of problem.

3. Evidence that domain knowledge outperforms generic end-to-end learning

Addressing RQ3, the results show that domain-informed feature engineering outperforms deep learning under both fair comparisons. Without metadata, XGBoost achieves a mean test AUC of 0.79, outperforming the best deep learning model (TapNet, 0.77) by 0.024 AUC points. With equivalent metadata access, XGBoost full (0.86) outperforms TapNet with early fusion (0.81) by 0.05 AUC points. Neither pre-trained foundation models nor large language models match even the CNN baseline. For the LLM specifically, the failure is shown to be architectural rather than a consequence of information loss during text encoding: providing the model with exact domain-engineered feature thresholds derived from the XGBoost pipeline does not improve performance, isolating the bottleneck as the model's inability to execute quantitative threshold logic rather than insufficient information access. Together these findings demonstrate that explicit domain knowledge, encoded through feature engineering, is more effective than any generic end-to-end learning approach evaluated on this task irrespective of whether metadata is available to the deep learning model.

4. A nuanced finding on the role of metadata integration mechanism

Addressing RQ4, an ablation study reveals that the utility of operational metadata depends critically on how it is consumed rather than simply whether it is available. Despite both TapNet and XGBoost receiving the same eight metadata fields, the benefit is 1.6 times larger for XGBoost (+0.07 AUC) than for TapNet (+0.04 AUC). This asymmetry demonstrates that gradient boosted trees exploit structured operational context through direct categorical splits requiring minimal additional learning, while early fusion into a deep learning backbone requires the model to discover metadata relevance through gradient descent over 300 timesteps a substantially less efficient mechanism. The finding has a direct practical implication that metadata is highly informative for this task, but its value is only fully realised when the integration mechanism matches the structure of the information.

1.5 Thesis Structure

The remainder of this thesis is organised as follows, with each chapter building directly on the foundation established by the previous one.

Chapter 2 establishes the theoretical and domain context needed to motivate and interpret the experimental choices made later. It covers the mechanics of web offset printing and the web break problem, reviews the five distinct deep learning model families evaluated in this work alongside XGBoost, large language models, and time series foundation models, examines the challenge of class imbalance in industrial machine learning, and surveys related work on fault detection and web break analysis. Together these establish what is already known and where the gaps lie.

Chapter 3 introduces the data on which all experiments are grounded. It describes the Bertelsmann printing facility, the vision-based camera monitoring system, the frame-level defect scoring pipeline that produces the raw input signal, the operational metadata available for each event, and the dataset statistics including the 84/16 class distribution. Understanding the structure and characteristics of this data is prerequisite to every methodological decision that follows.

Chapter 4 describes how the architectures reviewed in Chapter 2 are applied to the dataset described in Chapter 3. It defines the classification problem precisely, details the preprocessing pipeline that converts raw event JSON files into model-ready representations, specifies the three experimental pipelines, defines the evaluation protocol and metrics, and describes the strategies used to handle class imbalance across model families.

Chapter 5 presents the experimental results produced by applying the pipelines of Chapter 4 to the data of Chapter 3. Results are reported for each modelling approach right from convolutional models to XGBoost and its ablation with a comparative summary that ranks all approaches by mean test AUC and operational performance metrics.

Chapter 6 interprets the results of Chapter 5 through the lens of the architectural properties reviewed in Chapter 2 and the data characteristics established in Chapter 3. It explains why each model family succeeds or fails on this specific task, quantifies the contribution of domain knowledge and operational metadata, and discusses the practical pathway to deploying the best-performing system in a live printing environment.

Chapter 7 concludes the thesis with a summary of contributions, direct answers to the four research questions, and directions for future work.

Chapter 2

Background and Related Work

2.1 Web Offset Printing

Web offset printing is one of the most widely used high-volume printing technologies, used to produce newspapers, magazines, catalogues and commercial publications at scale. Unlike sheet-fed printing where individual sheets are fed one at a time, offset printing uses a continuous roll of paper called the web, that is fed through the press at high speed. A single paper roll can weigh up to one tonne and is unwound continuously as the press runs, with speeds reaching up to 40,000 impressions per hour [Kip01].

The paper web travels through several key sections of the press. It starts at the infeed section where the roll is unwound, and web tension is established [Web13]. The web then passes through a series of printing units, each applying one colour by transferring ink from a plate to a rubber blanket and then onto the paper [AVH⁺09]. On heat set presses, the web then passes through a hot air dryer, followed by chill rolls that cool the paper, before reaching the folder where it is cut and folded into the final product [AVH⁺09, Web13].

A critical parameter throughout this process is web tension, the pulling force applied to the paper as it moves through the press. The infeed runs slightly slower than the printing units, while components after the printing units run progressively faster, creating a controlled tension gradient that keeps the paper taut [HU02]. High tension overstresses the paper and may cause it to snap. Low tension causes the web to wander, crease, or wrap around a roller. Both conditions can result in a web break, a sudden rupture of the paper web that halts production entirely [Web13].

Web breaks typically occur when a tension peak coincides with a localised weakness in the web [Ues04]. The causes fall into two broad categories. Press-side causes include web tension irregularities, thermal miscalibration in the drying system, mechanical wear, and insufficient blanket washing frequency [Web13]. The causes on the Paper-side include insufficient tensile strength, non-uniform moisture distribution, caliper variations, and splice failures introduced during manufacturing or handling [Ues04]. In practice these two categories are not always easy to distinguish, as a paper with marginal tensile strength may run without issue under normal conditions but fail when a press-side tension peak occurs. This interaction between paper properties and press conditions is what makes cause attribution difficult [AVH⁺09].

Field data from pressrooms consistently show that press-related breaks and breaks of unknown cause constitute the majority of all web breaks, with paper defects contributing relatively little to the total break rate [Ues04]. A survey of 50 international printers found that 20-50% of break causes go unrecorded due to time pressure on press crews and the difficulty of post-hoc diagnosis [Web13]. This gap motivates the need for automated cause classification, which is examined in the context of related work in Section 2.4.

2.2 Time Series Classification

Time series classification is the task of assigning a class label to an ordered sequence of observations recorded over time. Unlike tabular data where each sample is an independent set of features, time series data carries temporal structure, the order and dynamics of the values matter as much as the values themselves. Several families of deep learning models have been developed to address this challenge, each taking a different architectural approach to capturing temporal patterns. The model families reviewed in this section were selected based on their established performance on the UCR and UEA benchmark archives, the standard evaluation suites for time series classification and their availability within the sktime and other toolkits [RFL⁺21]. The deep learning model families reviewed in this section were implemented using the sktime and tsai libraries, which provide consistent and reproducible implementations enabling fair comparison across all approaches using the same evaluation pipeline. In this thesis, each web break event is represented as a multivariate time series of frame-level defect scores, making time series classification the natural framework for the cause classification task. This section reviews the five model families evaluated in the thesis, providing the architectural background needed to interpret the experimental results in Chapter 5.

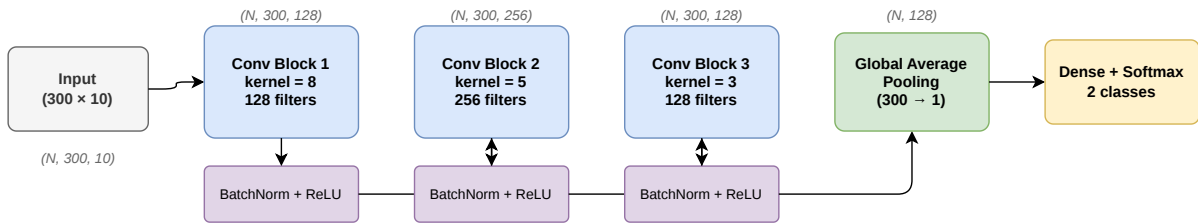


Figure 2.1: FCN architecture as implemented via sktime FCNClassifier with default parameters. Adapted from [WYO17].

Convolutional neural networks approach time series classification by applying learnable filters that slide along the temporal dimension, detecting local patterns at fixed scales regardless of where they appear in the sequence. [WYO17] proposed a Fully Convolutional Network (FCN) as a strong baseline for time series classification, consisting of three convolutional blocks with 128, 256 and 128 filters and kernel sizes of 8, 5 and 3 respectively, each followed by a batch normalisation layer and ReLU activation. Batch normalisation is applied after each convolutional layer to stabilise training and improve generalisation by keeping activations in a stable range throughout the network. Rather than using fully connected layers, which introduce many parameters and risk overfitting, the extracted feature maps are passed through a Global Average Pooling (GAP)

layer, which averages each filter’s response across the entire time dimension to produce a compact fixed-length representation of the input sequence. This representation is then passed to a softmax layer for classification. This architecture was applied in this thesis to the raw 300×10 multivariate time series of frame-level defect scores using the sktime FCNClassifier with default parameters, preserving the original three-block design with kernel sizes 8, 5 and 3 unchanged. Evaluated across 44 UCR benchmark datasets, spanning diverse domains including sensor readings, motion capture, and industrial monitoring, FCN achieved competitive performance against state-of-the-art methods including ensemble approaches, establishing convolutional networks as a strong baseline for time series classification [WYO17].

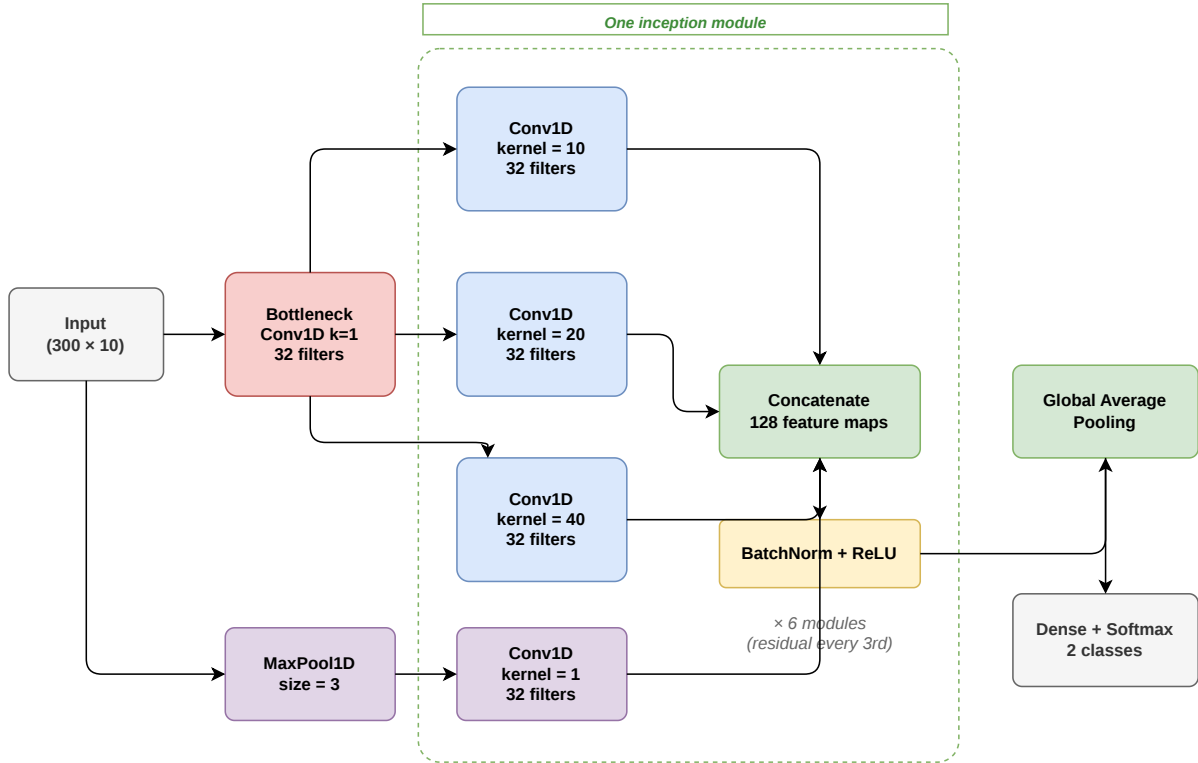


Figure 2.2: InceptionTime inception module with bottleneck, parallel multi-scale convolutions, and residual connections. Adapted from [F⁺20].

InceptionTime, proposed by [F⁺20], addresses the fixed kernel size limitation of basic convolutional networks by applying filters of multiple lengths simultaneously to the same input. Each Inception module first applies a bottleneck convolution with kernel size 1 and 32 filters to reduce dimensionality, before passing the result to three parallel convolutional branches with kernel sizes of 10, 20 and 40 respectively, each producing 32 feature maps. A fourth parallel branch applies MaxPooling of size 3 followed by a Conv1D projection, also producing 32 feature maps. The outputs of all four branches are concatenated to produce 128 feature maps per module, followed by batch normalisation and ReLU activation. This parallel multi-scale design allows the network to capture temporal patterns at short, medium and long scales within a single layer, rather than being restricted to one fixed receptive field as in FCN. The full architecture stacks six such Inception modules sequentially, with residual connections added every three modules and shortcut connections that add the module’s input directly to its output, allowing gradients to flow through the network without vanishing during

training. After all modules, global average pooling is applied to produce a compact event-level representation, followed by a softmax layer for classification. In this thesis, a single Inception network was applied to the raw 300×10 multivariate time series of frame-level defect scores using the sktime InceptionTimeClassifier with default parameters, rather than the full ensemble of five networks described in the original paper. Evaluated on 85 datasets from the UCR benchmark archive, InceptionTime achieved state-of-the-art performance matching HIVE-COTE, a strong ensemble of 37 classifiers, establishing deep convolutional networks as a competitive approach for time series classification [F⁺20].

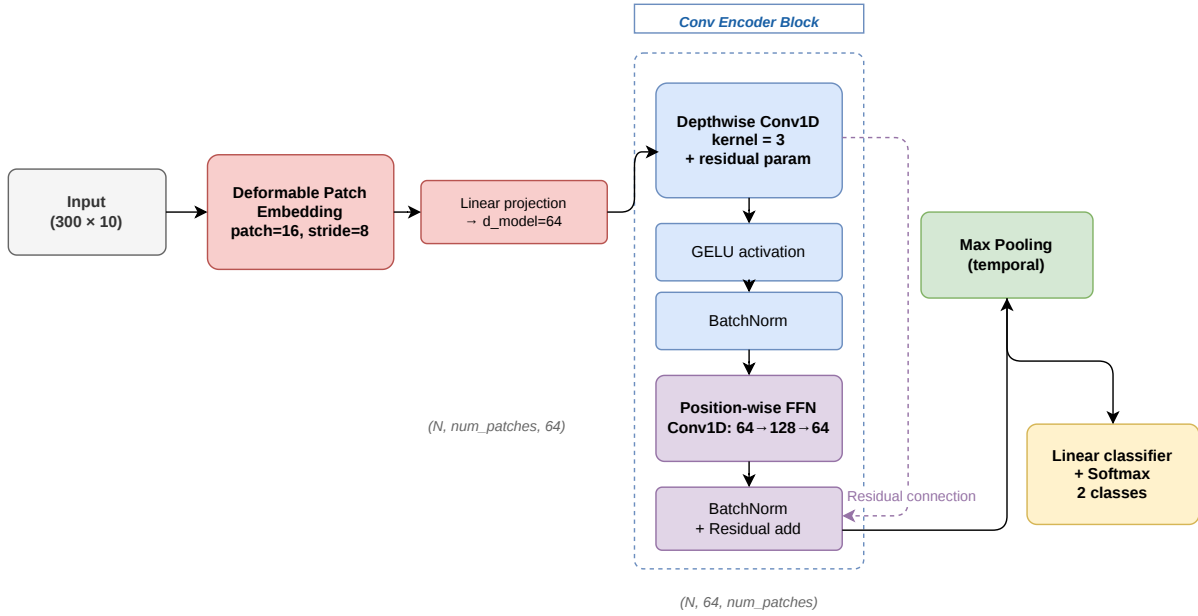


Figure 2.3: ConvTimeNet architecture with deformable patch embedding and hierarchical depthwise convolutional encoder. Adapted from [C⁺24].

ConvTimeNet, proposed by [C⁺24], extends standard convolutional approaches by replacing fixed receptive field processing with adaptive patch-based representations. Rather than sliding fixed kernels across the entire sequence, the model first applies a deformable patch embedding layer, which divides the time series into small overlapping segments and applies a learned projection to embed each patch, allowing the network to focus on the most informative temporal regions. These embedded patches are then processed through a hierarchy of depthwise convolutional blocks, where each level captures patterns at progressively larger temporal scales, moving from fine local structures to broader global dependencies. Each block incorporates batch normalisation, learnable residual connections to facilitate gradient flow, and GELU activation, which provides a smoother alternative to ReLU by weighting negative values rather than discarding them entirely, leading to improved stability in deeper architectures. This sequential multi-scale representation contrasts with InceptionTime’s parallel design, as ConvTimeNet builds scale-awareness gradually across layers. This architecture was applied in this thesis to the raw 300×10 multivariate time series of frame-level defect scores, with implementation details described in Chapter 4. Evaluated on 10 datasets from the UEA multivariate archive, ConvTimeNet outperformed state-of-the-art approaches including TST and TimesNet in over 80% of cases, demonstrating strong performance in capturing complex temporal dependencies across multiple scales [C⁺24].

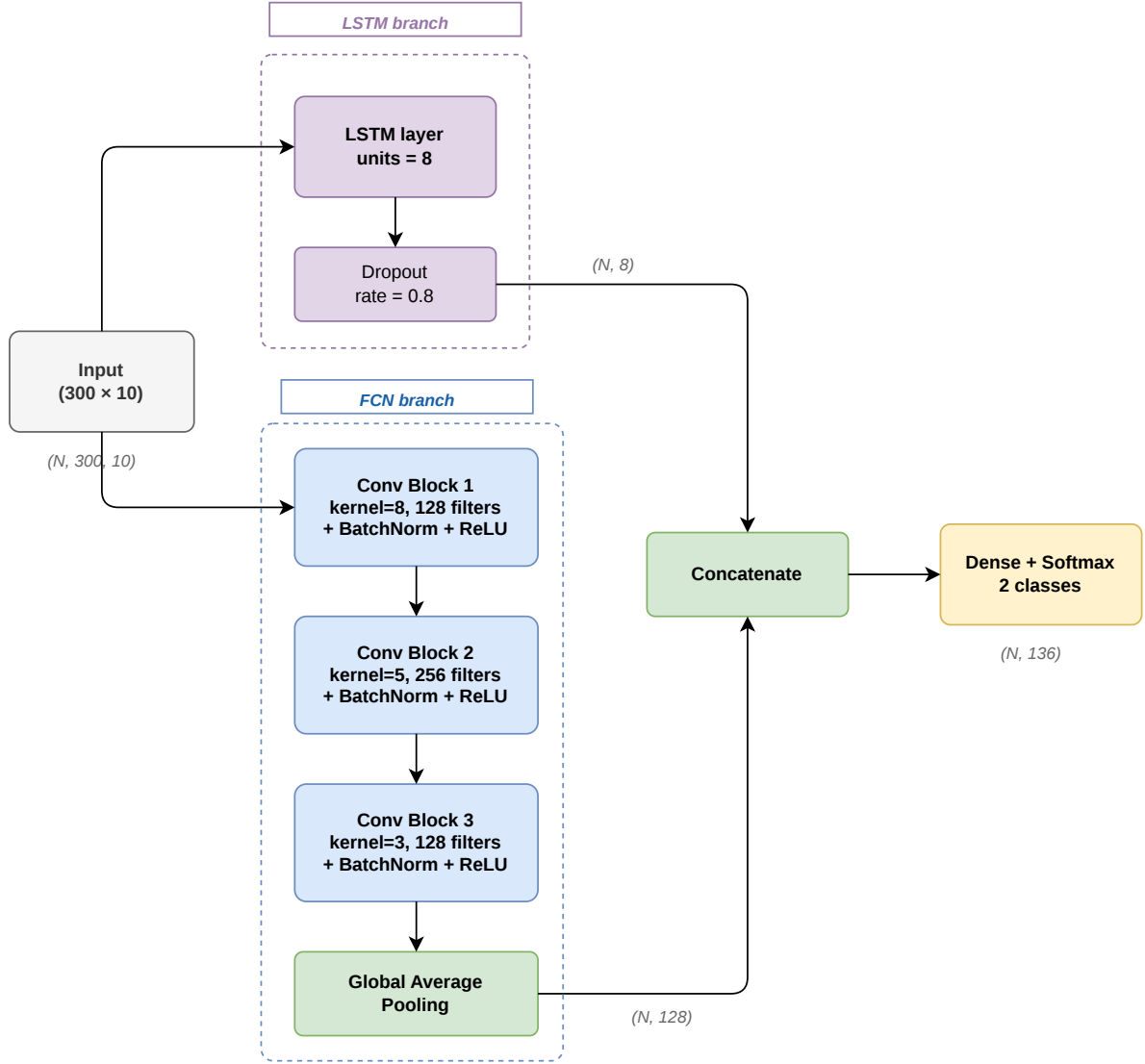


Figure 2.4: LSTM-FCN architecture with parallel LSTM and fully convolutional branches concatenated before classification. Adapted from [KMDH19].

LSTM-FCN, proposed by [KMDH19], addresses the limitations of purely convolutional approaches by combining two parallel processing branches that operate on the same input simultaneously. One branch is a fully convolutional network consisting of three convolutional blocks with 128, 256 and 128 filters and kernel sizes of 8, 5 and 3 respectively [WYO17], each followed by batch normalisation and ReLU activation, followed by Global Average Pooling. A squeeze and excitation block is incorporated after the convolutional layers in the original architecture, functioning as a channel-wise attention mechanism that learns to amplify the most discriminative feature maps while suppressing less informative ones. The other branch processes the input through a dimension shuffle operation before passing it through an LSTM layer with dropout regularisation, allowing the LSTM to model temporal dependencies across each feature channel independently. The outputs of both branches are concatenated and passed to a softmax layer for final classification. This architecture was applied in this thesis to the raw 300×10 multivariate time series of frame-level defect scores, with implementation details described in Chapter 4. Evaluated across 85 UCR benchmark datasets, LSTM-FCN demonstrated state-of-the-art performance, establishing hybrid CNN-LSTM

architectures as a strong approach for time series classification [KMDH19].

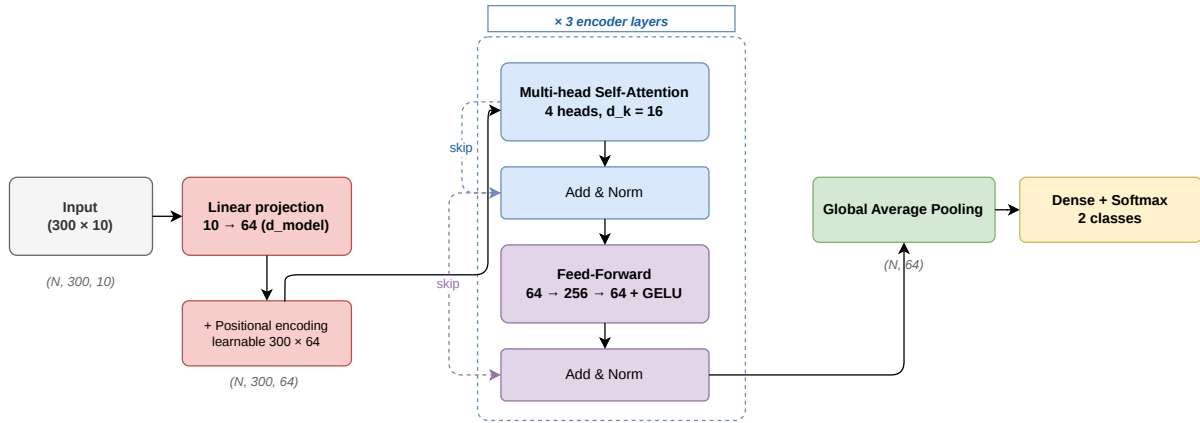


Figure 2.5: Time Series Transformer with multi-head self-attention, positional encoding, and stacked encoder layers. Adapted from [Z⁺21].

The Time Series Transformer (TST), proposed by [Z⁺21], adapts the transformer architecture for multivariate time series classification by applying self-attention directly across the temporal dimension. Unlike convolutional models that detect local patterns within fixed receptive fields, TST can attend to any pair of time steps simultaneously, allowing it to capture long-range dependencies across the full sequence without the vanishing gradient problem that affects recurrent models. The core mechanism is multi-head self-attention, where multiple attention operations run in parallel, each learning different relationships between time steps, with their outputs concatenated to form a richer representation of the sequence. Since the attention mechanism has no inherent notion of order, positional encoding is added to the input embeddings to preserve the temporal position of each time step, a critical requirement for time series where the ordering of observations carries meaning. TST follows an encoder-only architecture, like BERT in natural language processing, making it well suited for classification and regression tasks rather than sequence generation. A key architectural adaptation for numerical time series data is the use of batch normalisation instead of the layer normalisation typically used in NLP transformers, which was found to improve performance on continuous-valued inputs. Variable masking is also incorporated during training as a form of regularisation, encouraging the model to learn robust representations by randomly masking portions of the input. The `sktime` and `tsai` implementations of TST follow this architecture. This architecture was applied in this thesis to the raw 300×10 multivariate time series of frame-level defect scores, with implementation details described in Chapter 4. Evaluated on 6 multivariate regression datasets and 11 classification datasets, TST demonstrated competitive performance against state-of-the-art approaches, establishing transformer-based architectures as a viable framework for multivariate time series analysis [Z⁺21].

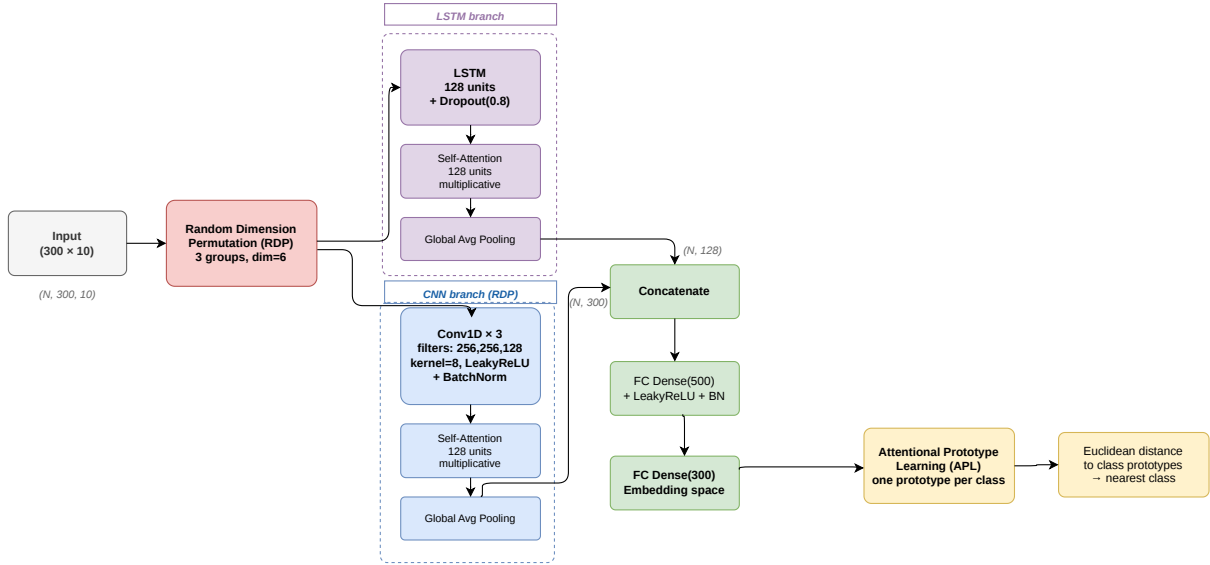


Figure 2.6: TapNet architecture with random dimension permutation, parallel LSTM and CNN encoders, and attentional prototype classification. Adapted from [Z⁺20].

TapNet, proposed by [Z⁺20], takes a fundamentally different approach to multivariate time series classification by replacing learned decision boundaries with prototype-based classification, learning representative embeddings for each class rather than drawing separating boundaries between them. The architecture consists of three sequential modules. The first is Random Dimension Permutation (RDP), which randomly groups the input dimensions into subsets before passing them to the encoding layers, encouraging the network to learn interactions between feature dimensions that would otherwise be missed if dimensions were always processed in the same fixed order. The second module is the multivariate time series encoder, which processes the raw input through two parallel branches. An LSTM layer captures global temporal dependencies, with a self-attention mechanism applied to the LSTM outputs before global average pooling produces a fixed-length feature vector. Three 1D convolutional layers with self-attention process the RDP output, producing a complementary representation of the local temporal structure. The outputs of both branches are concatenated and passed through fully connected layers to produce a compact low-dimensional embedding for each event. The third module is Attentional Prototype Learning (APL), which constructs a prototype vector for each class as a weighted sum of training sample embeddings, with an attention mechanism assigning higher weights to the most representative samples. At inference time, a new event is classified by computing the Euclidean distance between its embedding and each class prototype, assigning it to the nearest class. This architecture was applied in this thesis to the raw 300×10 multivariate time series of frame-level defect scores, with implementation details described in Chapter 4. Evaluated across 30 multivariate datasets from the UEA archive, TapNet achieved the lowest average rank among all compared methods, demonstrating strong performance on multivariate classification tasks with limited labelled data [Z⁺20][RFL⁺21].

Chronos, developed by Ansari et al. [AST⁺24] at Amazon, is a pre-trained foundation model for time series forecasting, built on the T5 encoder-decoder transformer architecture [AST⁺24]. Unlike the task-specific models reviewed above, Chronos is trained on a corpus of 28 real-world datasets augmented via TSMixup and one million synthetic series generated through KernelSynth [AST⁺24], learning general-purpose temporal

representations without being designed for any specific domain or task. The model operates by tokenising continuous time series values into discrete bins converting numerical observations into a sequence of tokens analogous to how language models tokenise words and processing these tokens through the T5 encoder-decoder to produce forecasts [AST⁺24]. In this thesis, Chronos is evaluated in two configurations as described in Chapter 4, both using the group attention mechanism, passing all five defect score channels from each event jointly to the encoder via shared `group_ids`, causing the encoder to mix cross-channel information within each event’s representation. The frozen encoder embeddings are extracted and passed to a lightweight MLP classification head trained on the target task, evaluating whether representations learned from large-scale forecasting pre-training transfer usefully to the web break cause classification problem a setting where the pre-training objective differs fundamentally from the target task.

Large language models (LLMs) have recently been explored as a general-purpose approach to time series analysis, motivated by their strong reasoning capabilities and broad pre-training on diverse text corpora. [G⁺23] demonstrated that LLMs such as GPT-3 and LLaMA-2 can perform zero-shot time series forecasting when numerical values are serialised as text, suggesting that the patterns learned during language pre-training may transfer to temporal reasoning tasks. Building on this, [JWM⁺24] proposed reprogramming time series data into text prototype representations compatible with LLM input formats, keeping the LLM backbone frozen while training a lightweight reprogramming layer, achieving competitive forecasting performance with minimal task-specific adaptation. The general approach involves encoding time series observations as sequences of symbolic text tokens representing discrete states and prompting the LLM to reason about the encoded sequence and produce a classification or prediction. While this approach is appealing because it requires no architectural modification and leverages the LLM’s existing reasoning capabilities, a fundamental concern when applied to continuous-valued industrial signals is that the discretisation process may lose fine-grained continuous dynamics that carry discriminative information. Whether this information loss is the primary limitation of LLM-based classification, or whether the bottleneck lies in the model’s ability to execute quantitative numerical reasoning on encoded representations, is an open empirical question that this thesis investigates in Chapter 4.

Gradient boosted tree ensembles, and XGBoost in particular, represent a fundamentally different paradigm for time series classification. Rather than learning representations end-to-end from raw sequences, this approach first transforms the time series into a fixed-length feature vector through manual or automated feature engineering, then applies a gradient boosted decision tree classifier to the resulting tabular representation. [CG16] introduced XGBoost as a scalable and regularised implementation of gradient boosting that achieves state-of-the-art performance across a wide range of supervised learning tasks, including classification problems from industrial and sensor domains. For time series data, the critical step is feature extraction where statistical summaries such as mean, variance, slope, and entropy are computed over temporal windows, capturing the global trajectory shape of each sequence in a compact numerical form. [CBNKL18] formalised this approach with the `tsfresh` library, which automates the extraction of 794 candidate features from time series using 63 established characterisation methods drawn from signal processing, statistics, and nonlinear dynamics, followed by hypothesis-test-based feature selection. When domain knowledge is available, manual

feature engineering can encode physically meaningful quantities more directly than automated extraction, such as the slope of a degradation signal, the timing of a peak, or the cross-sensor difference in a multivariate recording. This domain-informed approach is particularly advantageous when labelled data is limited, as the learning task reduces to estimating decision boundaries in a pre-computed feature space rather than discovering relevant representations from scratch. In this thesis, a domain-informed feature engineering pipeline combined with XGBoost is evaluated as an alternative to end-to-end deep learning, directly testing whether expert knowledge of the web break process can be encoded more efficiently than it can be learned [CG16].

2.3 Class Imbalance

Class imbalance is a pervasive challenge in industrial machine learning, where fault or failure events are inherently rare relative to normal operating conditions, resulting in datasets where between-class imbalance ratios of 100:1 or greater are commonly encountered [HG09]. This natural imbalance reflects the operational reality of the web break classification problem, where machine-caused breaks dominate the recorded break rate and paper-caused breaks represent a smaller but operationally significant minority.

Three complementary strategies are used in this thesis to address class imbalance. Class weighting assigns higher loss penalties to minority class misclassifications during training, implemented through custom weighted cross-entropy loss in deep learning models and through the `scale_pos_weight` parameter in XGBoost, which adjusts the gradient contribution of positive class samples by the ratio of majority to minority class counts. Threshold tuning adjusts the decision boundary away from the default 0.5 on the validation set, allowing explicit control over the precision-recall trade-off without modifying the model or training data. Synthetic oversampling via SMOTE [C⁺02] is not applied, as interpolating between raw time series sequences risks generating synthetic events with trajectories that have no physically realisable counterpart [B⁺16].

Selecting the right evaluation metric is equally important. Accuracy is misleading under imbalance as a classifier predicting the majority class for every instance achieves 84% accuracy on this dataset while detecting no paper-caused breaks. ROC-AUC is adopted as the primary metric because it evaluates discriminative ability across all classification thresholds simultaneously, is independent of class distribution, and provides a fixed random baseline of 0.5 regardless of imbalance ratio [Faw06]. These properties make it well suited to systematic multi-model comparison [RFL⁺21, HG09]. False Positive Rate (FPR) is monitored as a secondary operational metric, as misclassifying a machine-caused break as paper-caused carries direct consequences by incorrectly directing a supplier complaint when the fault lies within the press.

2.4 Related Work on Web Break Analysis

Research on web breaks in pressrooms has a long history, with most studies focusing on predicting break occurrence or identifying influential process parameters rather than

diagnosing the cause of a break after it has occurred. [AVH⁺09] applied data mining techniques to a web offset pressroom dataset, combining genetic algorithm-based feature selection with SVM classification to identify press and paper parameters associated with break occurrence, framing the problem as break versus no-break classification rather than post-hoc cause attribution.

[B⁺02] developed a predictive model for estimating time-to-breakage in the wet end of paper making machines, focusing on anticipating when a break is likely to occur. [Ues04] reviewed pressroom literature to investigate runnability problems and identify process conditions associated with increased break risk. More recently, [D⁺23] proposed a machine learning approach for predicting web breaks in tissue paper mills, achieving 86% accuracy and demonstrating reductions in production losses of up to 6,000 tonnes per year, but again framing the task as prediction of break occurrence rather than classification of causal origin.

Taken together, existing literature primarily addresses three related problems: prediction of imminent web breaks, estimation of time-to-breakage, and identification of process parameters associated with increased break likelihood. The problem of post-event cause attribution, determining why a break occurred after it has been recorded, and whether its origin is machine-related or material-related, has received limited attention in published work. This distinction matters operationally. While prediction systems alert operators to increased risk, they do not resolve the attribution problem that arises after a break: who is responsible, the press or the paper supplier? Attribution directly influences supplier complaints, maintenance scheduling, and process improvement decisions. As noted in Section 1.1, industry reports indicate that 20–50% of web breaks are recorded without a clearly identified cause [Web13], reflecting not a failure of prediction but a failure of post-hoc diagnosis.

Prior published research has not, to our knowledge, systematically addressed automated cause attribution for web breaks using data-driven classification approaches. Furthermore, all prior web break studies reviewed here rely on process and machine variables, web tension measurements, paper parameters, press speed, as input. This thesis uses multivariate time series of frame-level defect scores derived from a vision-based camera monitoring system, capturing the visual evolution of the web in the 300 frames preceding each break. To the best of our knowledge, vision-derived temporal representations of this kind have not been explored in prior web break analysis, introducing a novel input modality that encodes both machine behaviour and material behaviour simultaneously rather than relying solely on mechanical process sensors.

2.4.1 Cause Classification versus Fault Diagnosis

A natural point of comparison is fault diagnosis, a well-established area in industrial machine learning. Fault diagnosis typically analyses sensor data from a machine to identify its internal health state, for example, distinguishing bearing faults from gear faults based on vibration or acoustic signals [LYJ⁺20]. The system under analysis is the machine itself, and the classification targets correspond to different mechanical fault conditions within that machine. Web break cause classification differs from standard fault diagnosis in three important respects.

1. **Classification target.** The target is not a mechanical fault state but a causal origin, the question is not what is wrong with the machine, but whether the break originated in the machine or in the paper material.
2. **Signal nature.** The signal reflects the interaction between machine behaviour and material properties rather than the isolated condition of a single mechanical component, making the underlying patterns more ambiguous and less directly observable.
3. **Input representation.** The input is not a conventional mechanical sensor stream but a vision-derived multivariate time series capturing the visual dynamics of the paper web prior to failure, a representation that encodes the combined effect of press conditions and paper defects rather than a single system's mechanical state.

These structural differences place web break cause classification closer to causal attribution in a coupled system than to classical single-machine fault diagnosis.

2.4.2 Deep Learning for Industrial Time Series Classification

Beyond the web break domain, the broader literature on time series classification provides important methodological context. Benchmark archives such as UCR and UEA have enabled systematic comparison of TSC algorithms across diverse datasets [BLB⁺17]; [RFL⁺21], and recent work has begun to extend this evaluation to manufacturing settings. [F⁺24] conducted one of the most comprehensive evaluations of TSC algorithms in a manufacturing context, comparing 36 algorithms across 22 manufacturing datasets and identifying ResNet, InceptionTime, DrCIF and ARSENAL among the top-performing approaches. However, this study uses curated public benchmark datasets rather than real industrial deployments, and addresses product quality classification rather than post-event cause attribution.

[V⁺24] compared six algorithms including LSTM, ConvLSTM and Transformer on three real industrial machine datasets for failure prediction, finding that deep learning outperformed traditional ML only when fault patterns preceding failure were sufficiently diverse and complex. While this is the closest existing work in spirit to this thesis, it is limited to six models, addresses failure prediction rather than cause classification, and does not include prototype-based models, foundation models or LLM-based approaches. A consistent pattern emerges model evaluations on industrial data are narrow in scope and focused predominantly on anomaly detection or failure prediction. The theoretical behaviour of each TSC model family under conditions characteristic of industrial cause classification, long multivariate sequences, class imbalance, and subtle global temporal patterns, has rarely been empirically validated at scale on real deployment data. Convolutional architectures learn local temporal patterns within fixed receptive fields, which may limit their capacity to capture discriminative signals that span the full sequence length, a consideration motivating the multi-scale design of InceptionTime [F⁺20].

Recurrent models such as LSTM-FCN are designed to capture sequential dependencies, although performance under class imbalance is generally sensitive to training data characteristics [HG09], a challenge that is amplified in long multivariate sequences.

Attention-based models such as TST can theoretically model long-range timestep interactions, although transformer architectures are generally more data-dependent than convolutional approaches.

Prototype-based models such as TapNet are explicitly designed for tasks with limited labelled examples, a property that is also advantageous under class imbalance, learning representative class prototypes rather than decision boundaries, a theoretical advantage when minority class events are rare and heterogeneous [Z⁺20].

Foundation models such as Chronos-2 encode general temporal representations through large-scale pre-training but are optimised for forecasting rather than classification, and their transferability to industrial cause attribution tasks remains an open empirical question [AST⁺24].

LLM-based approaches have shown promise for zero-shot time series forecasting when sequences are encoded as text [G⁺23]; [JWM⁺24], but the discretisation of continuous industrial signals into discrete tokens introduces information loss that may limit applicability to tasks where fine-grained continuous dynamics carry the discriminative signal.

2.4.3 Research Gap

This thesis addresses two gaps. The primary contribution targets a domain gap, to the best of our knowledge, no prior published work has framed web break analysis as a supervised post-event cause attribution problem and, no prior work has used vision-derived frame-level defect scores as a temporal input representation for web break analysis of any kind. The secondary methodological contribution addresses the limited empirical understanding of how TSC architectural families behave on real industrial cause classification tasks, characterised by long multivariate sequences, class imbalance, and subtle global temporal patterns, as opposed to benchmark datasets or failure prediction tasks. By systematically evaluating six deep learning architectures spanning four architectural families alongside foundation models, LLM-based classification, and domain-informed feature engineering on 14,073 labelled real industrial events, this thesis produces empirical evidence about which model families are most suited to this class of problem under real industrial conditions, complementing the theoretical understanding established on benchmark datasets. Together, these contributions provide a practical approach to the web break attribution problem and a methodological contribution to the empirical understanding of TSC model behaviour under realistic industrial deployment conditions.

Chapter 3

Dataset

3.1 Data Source and Collection

The dataset used in this thesis originates from a commercial web offset printing facility operated by Bertelsmann in Germany. The facility runs multiple press lines of different machine types, producing commercial print products at high operational speeds. Web offset printing is a continuous process and any disruption to this continuity results in a web break that halts production entirely, as described in Chapter 2.

The facility is equipped with a vision-based monitoring system that automatically detects and records web break events. When a break occurs, sensors capture and archive the associated camera recordings, frame-level defect scores, and operational metadata into a structured event file. This automatic capture ensures consistent recording of break events across all press lines, shifts, and machine configurations throughout the data collection period.

Following capture, domain experts at the facility review each web break event and assign a cause label, machine-caused or paper-caused based on their operational knowledge of press conditions, paper properties, and the visual characteristics of the break. This expert labelling reflects the diagnostic process currently used at the facility for post-event cause attribution. The central motivation for this thesis is to automate this manual process, augmenting time-consuming expert annotation with a reliable data-driven classification system.

The dataset covers events recorded from July 2021 to December 2024, spanning three and a half years of continuous production across multiple press configurations. Data is archived in monthly ZIP files, 42 in total, each containing all web break events recorded across all press lines during that month. The raw dataset comprises 15,843 labelled event records. Following an audit described in Section 3.6, 14,083 events were retained as valid events with JSON and label.

3.2 Camera System and Frame Data

The facility operates multiple press lines of different configurations, and the camera setup varies across machine types. The monitoring system installed across all press

lines uses industrial cameras typically mounted at fixed positions, each capturing a continuous video stream of the paper web during operation.



Figure 3.1: Multi-angle camera views of the paper web passing through the press during operation.

One press configuration, visible in Figure 3.1, includes four camera positions, Seite A (Side A), Seite B (Side B), Vor Trockner (Before Dryer), and Trennmesser (Cutting Blade). Seite A and Seite B are mounted to directly face the paper web surface from both sides, capturing the visual dynamics of the web as it moves through the press. Vor Trockner is positioned to observe the dryer area and Trennmesser observes the cutting blade mechanism. Other press configurations in the facility use different camera arrangements, including Rollenwechsler cameras monitoring roll change mechanisms and Schneidmesser cameras observing cutting positions, as evidenced by the 22 distinct camera names found across the dataset.

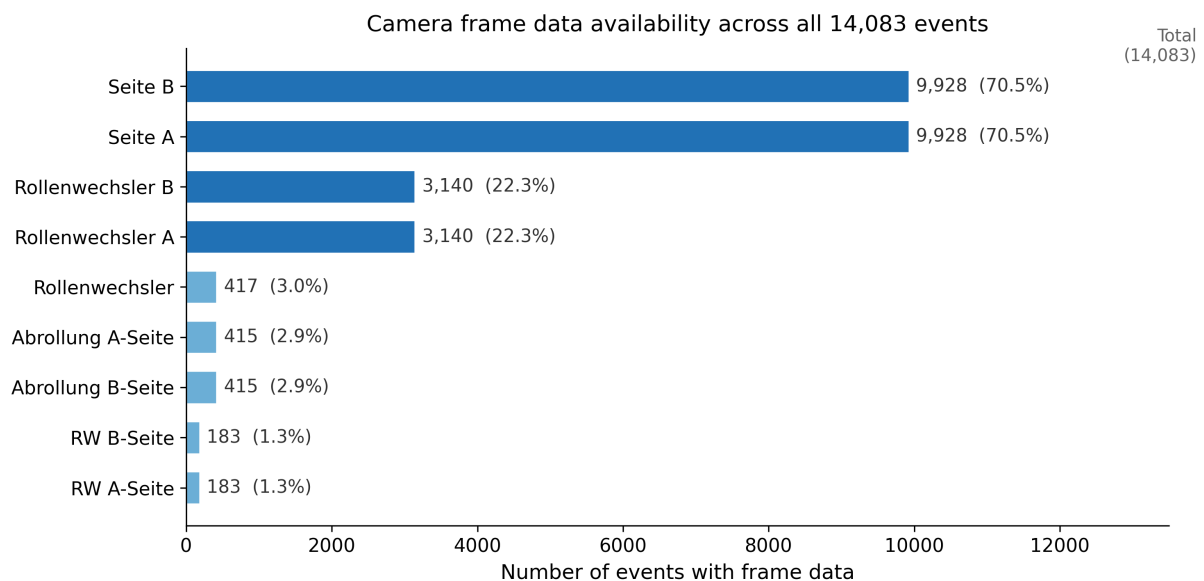


Figure 3.2: Camera frame data availability across all 14,083 events.

In total, 22 distinct camera names appear across the 14,083 events, reflecting the different monitoring configurations of each press line over the three-and-a-half-year collection period. A complete camera frame audit confirmed that 9 camera types contain actual frame-level defect score data, as shown in Figure 3.2: Seite A (9,928 events), Seite B (9,928 events), Rollenwechsler A (3,140 events), Rollenwechsler B (3,140 events), Rollenwechsler (417 events), Abrollung A-Seite (415 events), Abrollung B-Seite (415 events), RW A-Seite (183 events), and RW B-Seite (183 events). The remaining 13 camera types are recorded structurally in the event file but contain empty frames arrays across

every event in which they appear.

A further audit was conducted to determine how many cameras provide usable frame-level defect score sequences within each event. A camera was considered valid if it contained a minimum number of frames with non-zero-defect score vectors. The results show that 96.97% of events contain exactly two valid cameras, 2.96% contain a single valid camera, and only 0.07% contain no valid camera data.

This confirms that, despite the presence of multiple camera types across the facility, the monitoring system effectively operates as a two-camera setup for the vast majority of events. In practice, web break events are almost always captured by a pair of cameras providing complementary views of the paper web. The small fraction of single-camera events arises from partial data availability, while events with no valid camera data are negligible. This distribution justifies the use of a fixed two-camera representation in the modelling pipeline described in Chapter 4.

Each video stream is processed frame by frame using Microsoft Azure Custom Vision, a deep learning-based image classification model trained specifically on frames captured from the same press cameras at the Bertelsmann facility. This ensures that the defect scores reflect genuine visual patterns relevant to web break cause classification rather than generic image features.

For each frame, the Custom Vision model produces a hard label indicating the dominant defect category, and a five-dimensional probability score vector reflecting the model's confidence across five states. The label field contains the category with the highest probability score for that frame, while the scores field contains the full probability distribution across all categories. The structure of a single frame's output is shown below:

```
{
  "name": "0002.jpg",
  "label": "no_defect",
  "scores": {
    "no_defect": 0.390,
    "Kantenfehler": 0.249,
    "defect": 0.234,
    "tear": 0.075,
    "rollenwechsel": 0.053
  }
}
```

The five defect categories are defined as follows:

- **no_defect** : The frame shows normal paper condition with no visible irregularities
- **defect**: A general surface abnormality is present that may precede a break
- **Kantenfehler (Edge Defect)**: Damage or irregularity observed along the edge of the paper web
- **rollenwechsel (Roll Change)**: The frame was captured during a roll replacement or transition, representing an operational state rather than a defect

- tear: A visible rupture or structural separation in the paper web is detected

For this thesis, the continuous probability scores, not just the hard label are used as the input signal for all classification models.

3.3 Data Structure

Each web break event is stored as a single JSON file containing all information associated with that event, operational metadata, camera recordings, and frame-level defect scores, in a unified hierarchical structure. The JSON file is the primary unit of data in this thesis, and its structure is consistent across all 14,083 events, with variation only in the specific field values and camera configurations present for each press line.

The top level of each JSON file contains event-level metadata fields describing the operational context of the break. These include a unique event identifier, the press machine identifier, the date and time of the break, the duration of the event recording, the machine speed at the time of the break, the detector that triggered the recording, operator-assigned reason fields, and paper-related fields including grade, supplier, grammage weight, web width, paper length, roll number, and order information. A summary of the top-level metadata fields is shown in Table 3.1.

Field	Example Value	Description
id	123456	Unique event identifier
printer	FO123	Press machine identifier
date_time_str	20260105.012345	Event timestamp
date / time	01.05.2026 / 01:23:45	Date and time of break
duration	00:00:30	Duration of event recording
speed	14.8 m/s	Machine speed at time of break
detector	16	Detector that triggered the event
reason / sub_reason	-	Operator-assigned cause fields
grade	ABCEUTSE	Paper grade
paper_supplier	123456	Supplier identifier
grammage_weight	51 g/m2	Paper weight
web_width	224400.0	Web width in internal units
pap_len	20 m	Paper length on roll
roll_number	12345678	Roll identifier
order_number	123450000101312A0000	Production order identifier
ml_sentence	text summary	Auto-generated event description

Table 3.1: Top-level metadata field in each event JSON file

Nested within the top-level structure is a videos array, where each entry represents one camera stream associated with the event. Each video entry contains the camera identifier, camera name, frame size, and a frames array holding the frame-level data. Each frame in turn contains three fields: a frame name corresponding to the image filename, a hard label indicating the dominant defect category assigned by the Custom Vision model, and a scores dictionary containing the five-dimensional probability vector across all defect categories. The complete content map of an event JSON is illustrated in Figure 3.3.

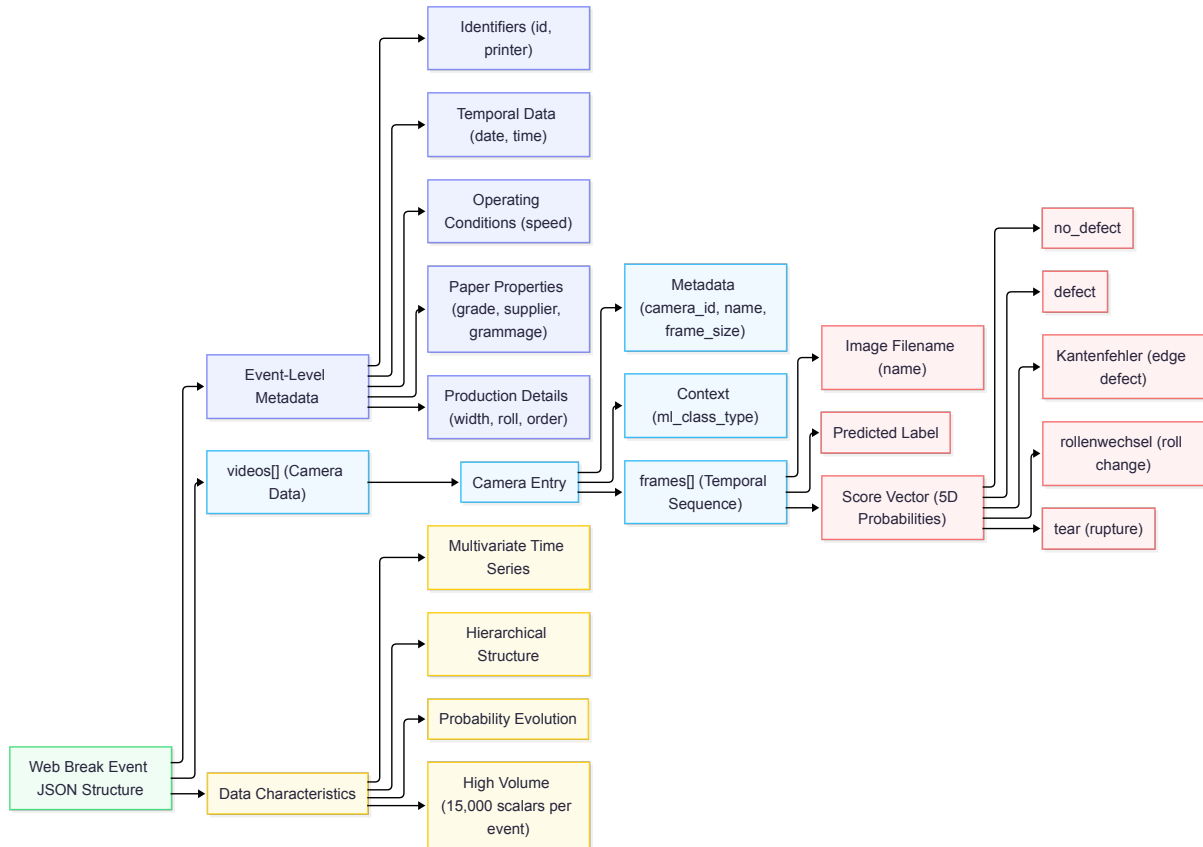


Figure 3.3: Map of an event JSON file

This structure is compact and self-contained, a single JSON file provides everything needed to reconstruct the full temporal sequence of defect scores for an event, along with the operational context in which the break occurred. The hierarchical layout naturally separates event-level context from frame-level signal, which maps directly onto the two-pipeline classification approach used in this thesis: frame-level scores form the time series input for deep learning models, while event-level metadata fields are used as features in the domain-informed XGBoost pipeline as well as in TapNet as described in Chapter 4.

3.4 Time Series Representation

The frame-level probability score vectors described in Section 3.2 are stacked sequentially to form a multivariate time series for each event. Each event is thus represented as a matrix of shape $(N, 5)$ per camera, N timesteps each described by five defect probability scores, where N is the total number of frames in the recording.

Rather than using the full event sequence, a fixed-length window of 300 frames is extracted from each event, aligned to the tear peak. The tear peak is defined as the frame with the high tear defect score within the event. This alignment ensures all events are registered to the same reference point, the moment of rupture, regardless of when the break occurred within the recording. As shown in Figure 3.4, the tear peak occurs at a median position of frame 1,245 within a typical 1,500-frame event, meaning the break develops well into the recording rather than at any fixed position from the start.

The extracted window therefore spans $t = -299$ to $t = 0$, where $t = 0$ is the tear peak. The tear peak was selected as the alignment reference rather than the onset of tear dominance because it provides a stable, unambiguous anchor across all events. The onset of rupture is not directly observable from defect scores, and brief fluctuations in the tear channel prior to the true break make dominance-based anchoring unreliable without additional smoothing.

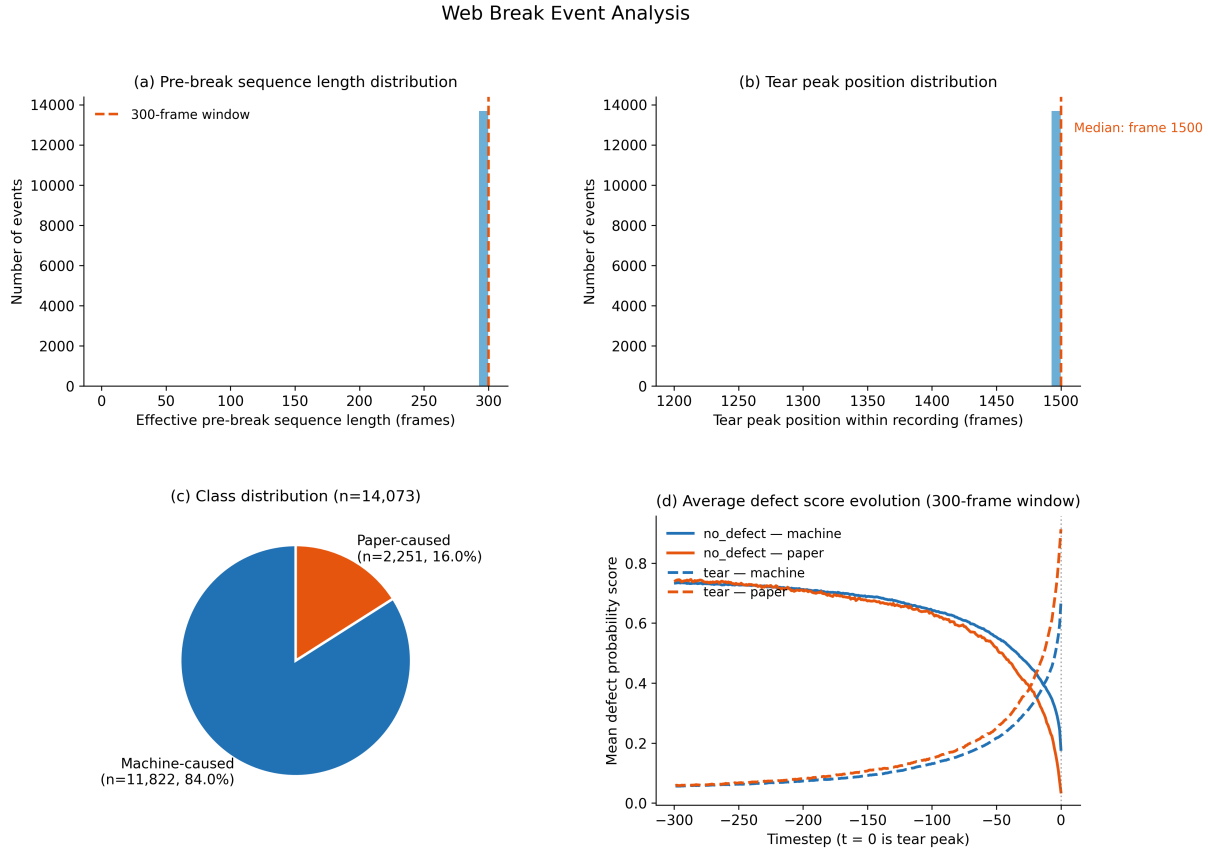


Figure 3.4: Web break event analysis

The average defect score evolution within this window is shown in Figure 3.4 (bottom right). The no_defect score begins high and declines progressively as the tear peak approaches, while the tear score rises from near zero and accelerates sharply in the final 50 frames. This confirms the 300-frame window captures the complete pre-break defect build-up trajectory.

3.4.1 Justification of the 300-Frame Window

To determine the appropriate window length, average tear score trajectories were compared across 150-frame, 300-frame, and 500-frame windows for both classes, as shown in Figure 3.5.

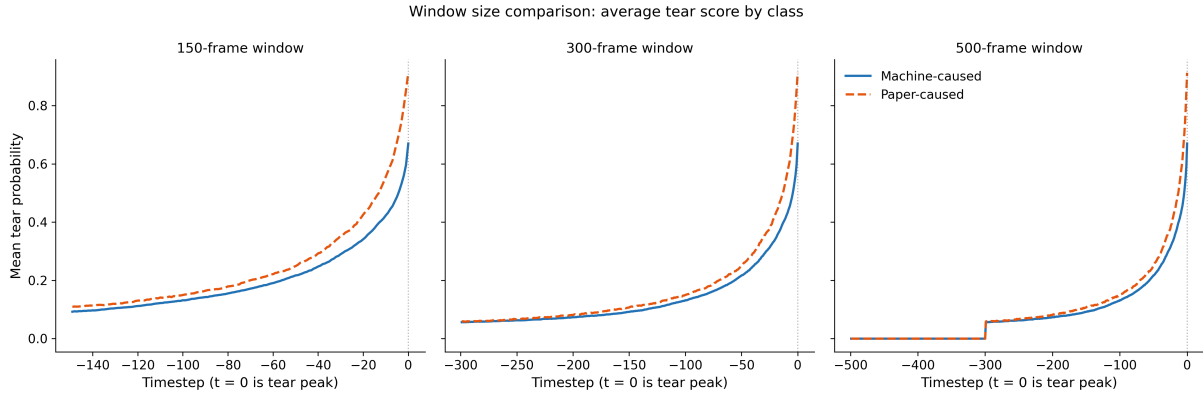


Figure 3.5: Window size comparison across 150, 300 and 500 frames

In the 500-frame panel, the region from $t = -500$ to $t = -300$ shows near-identical tear scores for both classes, no meaningful separation exists beyond $t = -300$. Reducing to 150 frames loses the early onset of class divergence visible from $t = -300$ to $t = -150$. Figure 3.6 confirms this quantitatively, extending from 300 to 500 frames reduces the per-frame signal-to-noise ratio from 0.085 to 0.059, as the additional frames introduce near-zero baseline signal. The 300-frame window therefore captures the full discriminative trajectory while covering nearly all events without data loss.



Figure 3.6: Per-timestep absolute class separation across window sizes

3.4.2 Class-Specific Temporal Signatures

Figure 3.7 shows the average defect score evolution separately for each class, revealing distinct temporal patterns that confirm the classification task is feasible.

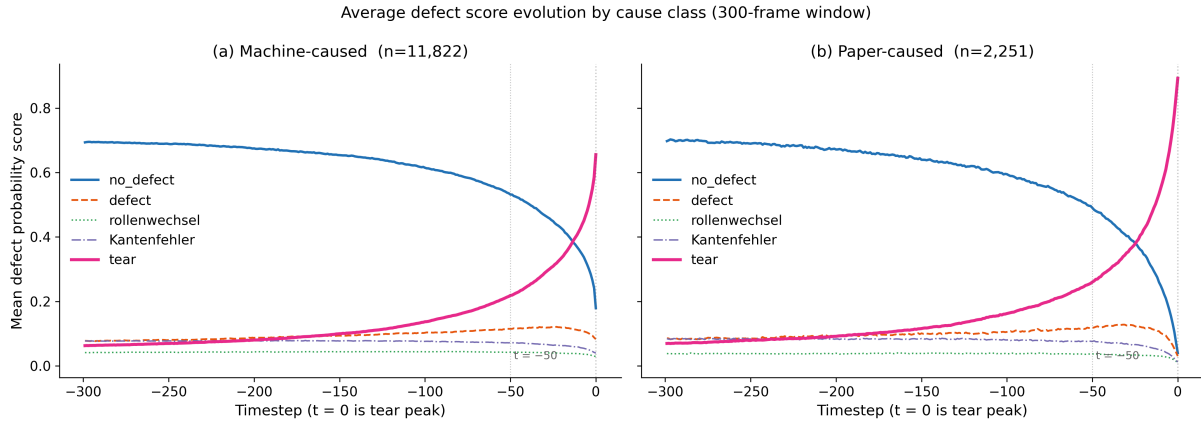


Figure 3.7: Average defect score evolution by cause class

Machine-caused breaks show a gradual progressive decline in `no_defect` across the full 300 frames, with a slow exponential tear build-up beginning well before rupture, consistent with incremental mechanical deterioration. Paper-caused breaks maintain stable `no_defect` scores until approximately 50 frames before the peak, then change sharply, consistent with sudden failure at a localised paper weakness. These distinct signatures are the core signal that classification models in this thesis are trained to detect, and their character has direct implications for model selection discussed in Chapter 5.

3.5 Operational Metadata

In addition to frame-level defect scores, each event JSON file contains structured operational metadata recorded at the time of the break. These fields describe the press configuration, paper material properties, and operational state when the break occurred. Table 3.2 summarises the metadata fields available across the dataset, along with their data types and example values as established in Section 3.3.

The metadata fields vary considerably in how consistently they are populated across events. Figure 3.8 shows the coverage of each field across all 14,083 events. Fields such as `printer`, `web width`, and `detector` are present in every even and fields such as `speed` and `grammage weight` are present in over 98% of events.

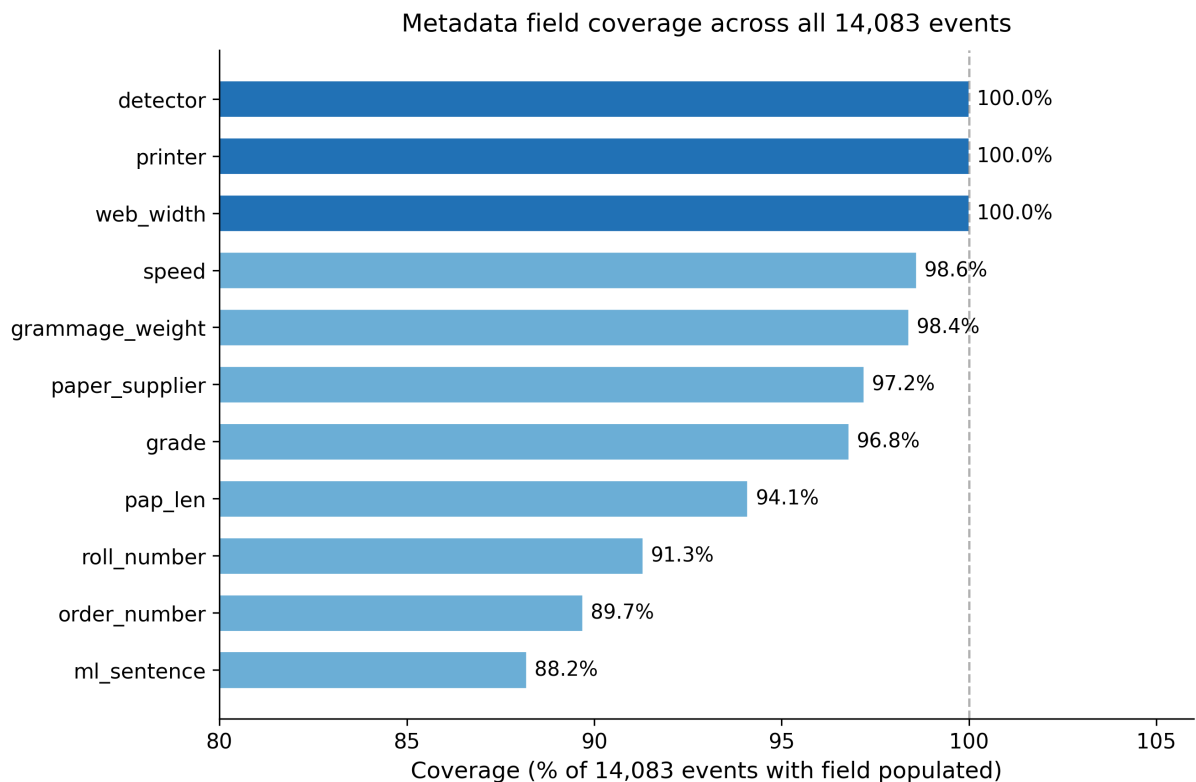


Figure 3.8: Metadata field coverage across all 14,083 events. Bars show the percentage of events in which each field is populated.

Beyond coverage, the key question for this thesis is which metadata fields carry discriminative signal, that is, which fields differ systematically between machine-caused and paper-caused breaks. Figure 3.9 shows the class distribution across the ten most frequent press machines in the dataset. The fraction of paper-caused breaks varies visibly across press configurations, some presses show a noticeably higher paper-caused rate than others, suggesting that press identity carries weak but meaningful class signal. This is consistent with the expectation that different machine types, operating conditions, and paper stock combinations produce different break cause distributions.

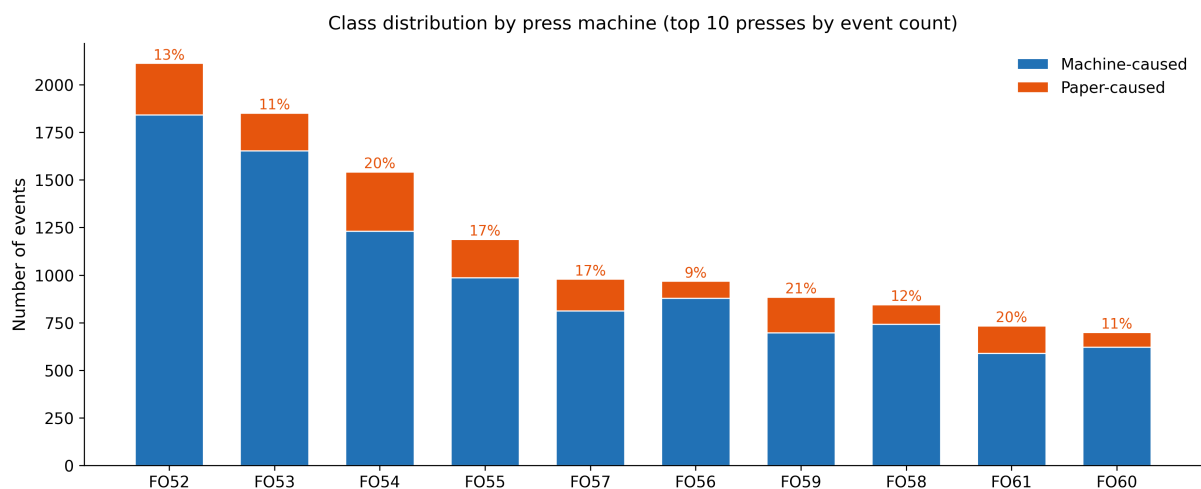


Figure 3.9: Class distribution by press machine for the ten most frequent presses

The numeric metadata fields, speed, grammage weight, web width, and seconds since

last wash, show considerable overlap between the two classes when examined individually, as shown in Figure 3.10. No single numeric field provides clean class separation. This is consistent with the nature of the classification problem: web break cause is determined by a combination of press condition, paper properties, and operational context, not by any single variable in isolation. The discriminative value of metadata therefore lies in combining multiple fields together rather than using any one field individually.

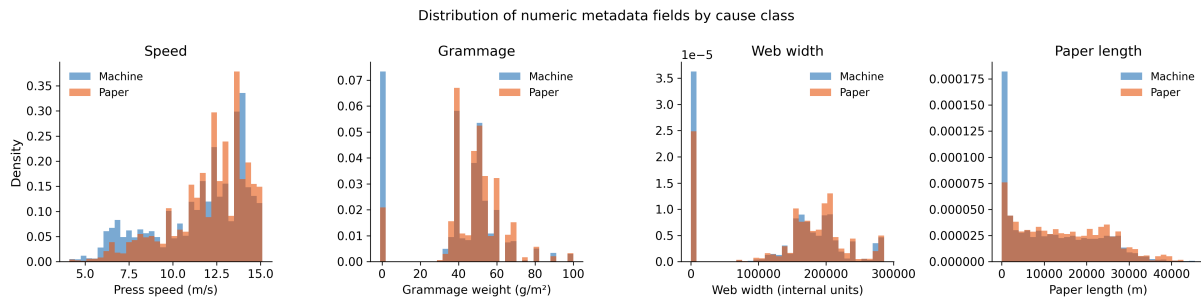


Figure 3.10: Distribution of key numeric metadata fields by cause class. Visual overlap between classes suggests no single field provides clean separation.

3.6 Data Quality and Class Distribution

The raw dataset extracted from the 42 monthly ZIP archives comprised 15,843 labelled event records. A data audit confirmed that 1,759 of these label rows had no corresponding JSON event file; these events were labelled by press operators but their associated camera recordings were not available in the archive. All remaining 14,083 events have both a valid cause label and a complete JSON recording with frame-level defect scores, confirmed by the audit showing zero JSON files without a matching label. Of the 14,083 events, 11,832 are labelled as machine-caused (Label 0) and 2,251 as paper-caused (Label 1), producing a class split of 84.1% to 15.9% and a class imbalance ratio of approximately 5.25:1. This imbalance is not an artefact of the data collection process but reflects the genuine operational distribution of break causes at the facility; machine-related factors and unknown causes dominate the recorded break rate, while paper-caused breaks represent a smaller but operationally significant minority [Web13]. A further 10 events are subsequently excluded during preprocessing due to the absence of any valid camera data (Section 4.2.1), yielding a final experimental dataset of 14,073 events containing 11,822 machine-caused and 2,251 paper-caused records. A subset of 10,492 events contains a clearly identifiable tear signal in the camera recordings, of which 8,407 are machine-caused (80.1%) and 2,085 are paper-caused (19.9%). The class imbalance ratio in this tear subset is approximately 4:1, somewhat less severe than in the full dataset. All 14,073 experimentally valid events are used for model training and evaluation, as events without a prominent tear signal still carry temporal defect score information relevant to cause classification. Figure 3.11 summarises the dataset composition and class distribution.

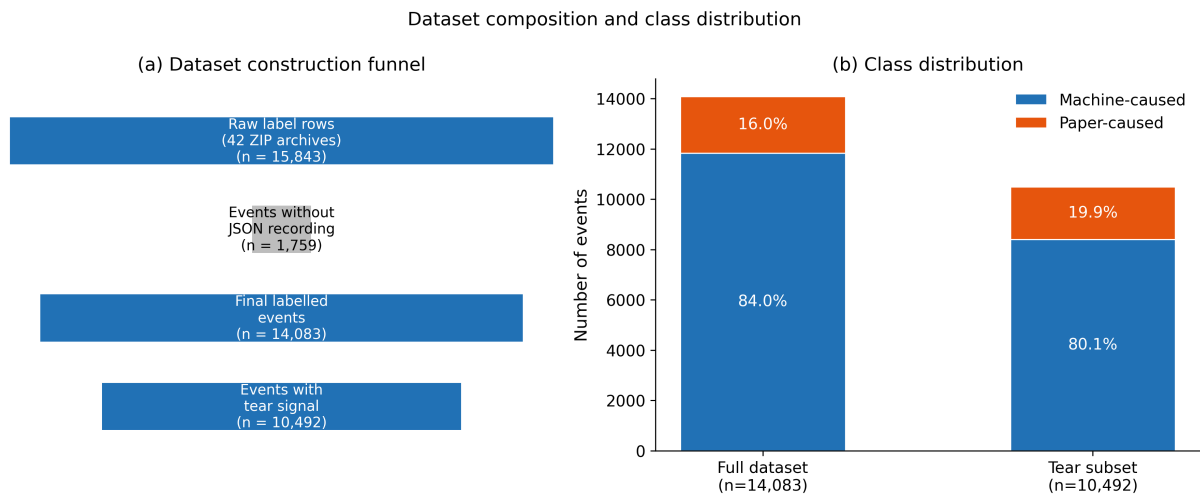


Figure 3.11: Dataset composition: (left) funnel showing how the final 14,083 events were derived from 15,843 raw records; (right) class distribution showing the 84/16 split for the full dataset and the 80/20 split for the tear subset

Class imbalance of this magnitude has direct implications for model training and evaluation. A naive classifier that predicts machine-caused for every event would achieve high accuracy, making raw accuracy a misleading metric. For this reason, ROC-AUC is used as the primary evaluation metric throughout this thesis, as it measures discriminative ability independently of the classification threshold and is robust to class imbalance. False positive rate, the fraction of machine-caused breaks incorrectly labelled as paper-caused, is used as a secondary metric reflecting the operational cost of misclassification specifically, the rate at which the classifier generates spurious supplier complaints for breaks that originated from a machine fault, as described in Chapter 2 and Chapter 4.

3.7 Dataset Summary

Table 3.3 provides a complete summary of the dataset used in this thesis.

Property	Value
Collection period	July 2021 - December 2024
Monthly ZIP archives	42
Raw label rows	15,843
Events without JSON recording	1,759
Final labelled events	14,083
Machine-caused (Label 0)	11,832 (84.1%)
Paper-caused (Label 1)	2,251 (15.9%)
Class imbalance ratio	~5.25:1
Tear signal subset	10,492 events
Machine-caused (subset)	8,407 (80.1%)
Paper-caused (subset)	2,085 (19.9%)
Press machines	14 distinct configurations
Cameras with frame data	9 camera types
Score categories per frame	5
Temporal window	300 frames before tear peak
Median event length	1,500 frames
Events with ≥ 300 frames	99.9%
Labelling method	Domain experts at Bertelsmann

Table 3.2: Dataset summary

Chapter 4

Methodology

4.1 Problem Formulation

The task addressed in this thesis is formulated as a binary supervised classification problem. As introduced in Chapter 1, for each recorded web break event, the objective is to identify whether the break was caused by a machine-related operational factor (Label 0) or a paper-related material factor (Label 1).

Each event is characterized as a multivariate time series, constructed from frame-level defect probability scores taken from camera recordings preceding the break. A two-camera representation is used throughout all experiments, based on the dataset characteristics established in Chapter 3. For each event, two camera streams are processed and combined to form an input matrix of shape $(300, 10)$, where 300 is the number of time-steps and 10 represents the concatenation of five defect score channels from each camera. The second channel is set to zeros with a recorded binary presence flag, for the specific 2.96% of events containing only one valid camera.

The temporal dimension is aligned to the tear peak, which is defined as the frame with the maximum tear defect score within each camera stream. The resulting sequence covers the interval $t = -299$ to $t = 0$, where $t = 0$ denotes the moment of rupture.

The output is a binary label $y \in \{0, 1\}$, where 0 denotes machine-caused breaks and 1 denotes paper-caused breaks. The dataset is heavily imbalanced, with 84.0% of events labeled as machine-caused and 16.0% labeled as paper-caused in the final preprocessed dataset, as established in Chapter 3. Model performance is primarily examined using ROC-AUC, as discussed in Chapter 2, due to its resilience to class imbalance and independence from a fixed decision threshold. False positive rate is used as a secondary metric to show the operational impact of misclassification, specifically the rate at which paper-caused breaks are incorrectly assigned as the machine-caused label.

4.2 Preprocessing Pipeline

This section outlines the preprocessing pipeline used to convert raw event-level JSON records into model-ready representations. All models presented in this thesis derive their inputs from a single shared preprocessing pipeline, thereby ensuring consistency

and comparability of the underlying data across all approaches.

The deep learning models utilise the preprocessed $(300, 10)$ multivariate time series directly as input, transposed to $(10, 300)$ for compatibility with the sktime classifier interface. An exception is TapNet, which is additionally evaluated in an early metadata fusion configuration in which eight operational metadata fields are repeated across all 300 timesteps and concatenated to the time series input, expanding the input shape to $(300, 18)$. This experiment is conducted specifically for TapNet as the best-performing deep learning architecture, to enable a direct comparison with the XGBoost full model which uses the same eight metadata fields. The XGBoost pipeline adds an extra feature extraction step on top of the preprocessed time series, calculating statistical, temporal, uncertainty, cross-camera, and metadata features as described in Section 4.3; the raw time series is not passed to XGBoost directly but all features are derived from it. The LLM pipeline applies argmax discretisation and run-length encoding to the preprocessed sequences to produce symbolic textual representations, as described in Section 4.5.3. Chronos-2 uses single-camera $(300, 5)$ input from the first valid camera stream, evaluated under a separate temporal split rather than the stratified cross-validation applied to all other models, as described in Section 4.5.4. The core preprocessing steps are frame validation, tear peak alignment, temporal window extraction, and multi-camera concatenation, all these form the shared foundation for all pipelines.

4.2.1 Frame-Level Validation and Camera Selection

Each web break event, as described in Chapter 3, is stored as a JSON file containing multiple camera streams and associated metadata. Each camera stream consists of a series of frames, where each frame has a probability distribution over five defect categories.

Not all frames contain meaningful information. Each frame is therefore validated by checking whether its defect score dictionary includes at least one non-zero probability value. Frames that do not meet this criterion are excluded from further processing.

All valid frames are collected for each camera streams. A camera is considered usable only if it contains at least 10 valid frames, a threshold selected to exclude very short or noisy sequences while retaining all practically informative camera streams. Camera streams that do not meet this criterion are discarded.

As established by the camera frame audit in Section 3.2, applying this validation across all 14,083 events results in 96.97% of events (13,656) yielding exactly two usable camera streams, 2.96% (417 events) yielding a single usable camera, and 0.07% (10 events) yielding no usable camera data. Events with no usable camera data cannot produce any time series input and are therefore excluded from all experiments, resulting in a working dataset of 14,073 events.

4.2.2 Tear Peak Alignment and Temporal Window Extraction

For each usable camera stream, the temporal sequence is aligned to the tear peak, defined as the frame with the maximum tear score within the sequence. This alignment ensures that all events are referenced to a common physical point corresponding to the

moment of rupture, irrespective of when the break occurred within the recording.

Once the tear peak is identified, all frames from the start of the sequence up to and including the tear peak are retained. Each frame is converted into a five-dimensional numerical vector representing the defect score distribution across the five predefined categories in a fixed order: no_defect, defect, rollenwechsel, Kantenfehler, tear.

A fixed-length window of 300 timesteps is then constructed. If more than 300 frames precede the tear peak, only the last 300 are retained, as shown in Section 3.4.1, as frames beyond this window do not provide additional discriminative signal. Conversely, if fewer than 300 frames are available before the tear peak, the sequence is left-padded with zero vectors to maintain a consistent input shape. The effective pre-tear sequence length and padding fraction are recorded for each camera and used as auxiliary features in the XGBoost pipeline.

This procedure produces a fixed-length matrix of shape $(300, 5)$ for each camera, capturing the full pre-break defect evolution aligned to the tear peak.

4.2.3 Multi-Camera Sequence Construction

For each event, the sequences from up to two usable cameras are integrated to form a unified multivariate representation. When two usable cameras are available, both sequences are retained. If only one usable camera is available, a second sequence of zeros is introduced to maintain a consistent input shape, and a binary indicator flag (`camera2_present = 0`) is recorded.

The two camera sequences are concatenated along the feature dimension, resulting in a final representation of shape $(300, 10)$. This allows all models to access information from both camera views simultaneously at each timestep within a unified input structure.

Although the two camera streams are not strictly temporally synchronised, each is independently aligned to its respective tear peak, thereby referencing the same physical event of rupture. The inter-camera tear peak offset is expected to be small relative to the 300-frame discriminative window, as both cameras observe the same physical rupture event simultaneously. This ensures that the critical pre-break dynamics are represented within the same temporal window for both cameras, enabling models to learn joint patterns across both views.

Concatenation along the feature dimension was selected over alternative fusion strategies for three primary reasons. First, stacking the sequences along the time dimension would separate the two camera views by 300 frames, preventing models from learning joint cross-camera patterns at the same relative pre-break position. Second, late fusion, where each stream is processed independently before combining event-level embeddings, this would require architectural modification incompatible with the standardised sktime evaluation framework used across all six deep learning architectures evaluated in this thesis. Third, the temporal synchronisation assumption inherent in feature-dimension concatenation is of limited practical significance: both streams are aligned to the same physical event, and any inter-camera tear peak offset is small relative to the 300-frame window. Furthermore, cross-camera features derived from the concatenated representation, including differences in mean and maximum tear scores between cameras, are included in the XGBoost feature set, contributing to the joint

multivariate representation used across all pipelines.

4.2.4 Dataset Construction

The preprocessing pipeline described in Chapter 3 is applied to all events extracted from the 42 monthly ZIP archives. Events lacking a valid label or without any usable camera streams are excluded from analysis. As noted in Section 4.2.1, 10 events contain no usable camera data and are therefore removed, producing a final pre-processed dataset of 14,073 events. Of these, 11,822 are machine-caused (Label 0) and 2,251 are paper-caused (Label 1), maintaining an imbalance ratio of approximately 5.25:1.

For each valid event, the processed $(300, 10)$ time series representation, the corresponding binary label, and the associated auxiliary metadata are stored. The auxiliary information, effective sequence length per camera, padding fraction per camera, and camera presence indicator, is used as additional features in the feature-engineered XGBoost pipeline described in Section 4.3.

4.2.5 Feature Scaling

Within each cross-validation fold, feature scaling is applied to the time series data to standardize feature distributions prior to model input. A `StandardScaler` is fitted exclusively on the training split of each fold. To compute per-channel statistics across all timesteps, the time series data is reshaped from $(N, 300, 10)$ to $(N \times 300, 10)$. The fitted scaler is then applied to the validation and test splits without refitting, further preventing data leakage. The scaled data are then reshaped back to $(N, 300, 10)$ before being provided to the models. This procedure ensures that all models receive consistently standardised inputs derived exclusively from training data statistics.

4.3 Feature Engineering for XGBoost

In addition to the raw time series representation utilised by deep learning models, a feature-engineered tabular representation is constructed for the XGBoost pipeline. All features are derived from the same preprocessed $(300, 10)$ time series produced by the pipeline described in Section 4.2, ensuring that the XGBoost model and deep learning models operate on consistent underlying data. The feature design is motivated by the class-specific temporal signatures like some established in Section 3.4.2: machine-caused breaks exhibit a gradual progressive decline in the `no_defect` score across the full 300-frame window, while paper-caused breaks maintain stable `no_defect` scores until a sharp late-stage tear spike. Features are therefore designed to explicitly encode the global trajectory shape, the timing and magnitude of the tear build-up, and the operational context of the break properties that end-to-end deep learning models must discover from scratch but that can be directly computed from domain knowledge of the break process.

For each event, the $(300, 10)$ matrix is separated into two camera-specific sequences of shape $(300, 5)$, corresponding to the five defect score channels per camera. Features are

computed per camera and combined into a fixed-length feature vector.

Statistical Features (per camera)

Statistical summaries capture the overall distribution of each defect score channel across the full 300-frame window. Mean values reflect the average defect state during the pre-break period: a high mean no_defect score indicates stable operation while a high mean tear score indicates sustained defect activity. Standard deviation captures the variability of each channel, distinguishing gradual monotonic trajectories from fluctuating ones. Maximum values record the peak intensity of each defect category, which is particularly informative for the tear channel where the peak magnitude differs between machine-caused and paper-caused breaks.

For each of the five defect score channels (no_defect, defect, rollwechsel, Kantenfehler, tear) the following statistical features are computed:

- Mean value across all 300 timesteps
- Standard deviation across all 300 timesteps
- Maximum value across all 300 timesteps

This produces 15 features per camera, 30 in total across both cameras.

Temporal Features (per camera)

Temporal features directly encode the trajectory shape that distinguishes the two break causes. The linear slope of the no_defect channel captures the gradual deterioration pattern characteristic of machine-caused breaks: a strongly negative slope across 300 frames indicates progressive mechanical deterioration, while a near-zero slope followed by a sudden drop is more consistent with paper-caused failure. The linear slope of the tear channel captures the rate of tear score escalation leading up to the break. Window-based tear features compare the early and late portions of the sequence: machine-caused breaks show tear activity building from early in the window, while paper-caused breaks concentrate tear activity in the final frames. The difference between the mean tear score in the last 50 frames and the first 50 frames directly quantifies this temporal asymmetry and is expected to be one of the most discriminative features in the set.

From the tear and no_defect channels the following temporal features are derived:

- Linear slope of the tear score across all 300 timesteps
- Linear slope of the no_defect score across all 300 timesteps
- Mean tear score across the last 50 timesteps
- Mean tear score across the first 50 timesteps
- Difference between mean tear scores of the final 50 and initial 50 timesteps

This produces 5 features per camera, 10 in total.

Uncertainty and Variability Features (per camera)

Uncertainty features capture the confidence and stability of the Custom Vision model's defect score predictions across the sequence. Shannon entropy of the five-category score distribution at each timestep measures how concentrated or diffuse the model's confidence is: low entropy indicates the model is highly confident about the dominant defect category, while high entropy indicates ambiguity across categories. Sustained high entropy may indicate transitional states in the defect evolution. The variance of individual channels captures channel-level instability, and the mean absolute difference between consecutive tear scores measures the frame-to-frame smoothness of the tear signal, machine-caused breaks tend to show a smoother, more gradual tear escalation, while paper-caused breaks may show sharper frame-to-frame changes near rupture. Peak sharpness quantifies how abruptly the tear score peaks relative to its recent baseline, directly encoding the suddenness of failure onset.

- Mean Shannon entropy of the five-category defect score distribution, averaged across all 300 timesteps
- Standard deviation of the per-timestep Shannon entropy
- Variance of each of the five defect score channels over 300 timesteps (5 features)
- Mean absolute difference between consecutive tear scores, measuring instability of the tear signal
- Peak sharpness: the difference between the maximum tear score and the mean tear score over the last 50 timesteps

This produces 9 features per camera, 18 in total.

Cross-Camera Features

Cross-camera features capture disagreement between the two camera views of the same break event. If both cameras observe a consistent tear build-up, the difference in their mean and maximum tear scores will be small. A large inter-camera discrepancy may indicate that the defect originates on one side of the web like an edge defect captured by one camera but not the other, which is more consistent with paper-caused failure than with a press-side mechanical fault that would affect the full web width uniformly. These features are only meaningful for events with two valid camera streams, but are included for all events as the zero-padded second camera produces zero cross-camera differences that are handled naturally by XGBoost.

- Difference in mean tear score between camera 1 and camera 2
- Difference in maximum tear score between camera 1 and camera 2

This produces 2 cross-camera features.

Event-Level Auxiliary Features

Auxiliary features derived from the preprocessing pipeline encode structural properties of the event recording itself. The padding fraction per camera captures how much of the 300-frame window is actual signal versus zero-padding, reflecting the length of the pre-break recording. The camera presence indicator flags single-camera events, allowing the model to account for the reduced information available in those cases and to avoid interpreting zero-padded channels as genuine signal.

- Padding fraction for camera 1
- Padding fraction for camera 2
- Binary indicator for presence of a second camera

This produces 3 auxiliary features.

Metadata Features

Operational metadata fields provide contextual information about the press conditions and paper properties at the time of the break that is invisible to the camera signal alone. Press speed at the time of the break affects the mechanical stress on the paper web, higher speeds increase tension and reduce the tolerance for paper weaknesses. Grammage weight and paper grade encode the mechanical properties of the paper stock, with lighter or lower-grade papers being more susceptible to material-caused failure. Web width determines the surface area under tension. Paper length reflects how much of the roll has been consumed, which influences the probability of encountering a pre-existing splice or edge defect. The detector identifier encodes which sensor triggered the break recording, providing indirect information about where in the press the break was initiated. Printer identity, paper grade, and paper supplier together encode facility-level and supplier-level priors about break cause distributions: certain press configurations, paper types, and suppliers are systematically associated with higher rates of paper-caused breaks, information that the camera signal alone cannot capture.

- Speed (numeric, parsed from string with unit)
- Grammage weight (numeric, parsed from string with unit)
- Web width (numeric)
- Paper length (numeric)
- Detector identifier (numeric)
- Printer (categorical, one-hot encoded)
- Paper grade (categorical, one-hot encoded)
- Paper supplier (categorical, one-hot encoded)

In total, the engineered feature set consists of 63 numerical features derived from the time series and auxiliary preprocessing information, plus 5 numerical metadata features, giving 68 numerical features in total. Three categorical variables that is printer, paper grade, and paper supplier are one-hot encoded within each cross-validation fold, with the encoder fitted on the training split and applied to the validation and test splits without refitting. After encoding, the final feature dimensionality is approximately 80-90 depending on the number of unique category values present in each training fold.

4.4 Evaluation Design

This section describes the experimental framework and performance metrics used to compare model performance.

4.4.1 Cross-Validation Strategy

All the models are trained using event-level stratified 3-fold cross validation. While the data is time series, the classification is at event level, the goal is to determine the cause of an individual break event based on defect evolution within that event. Each event is considered as an isolated case and time information is wholly captured within the sequence of events. The ordering of events does not contain information useful to the classification task.

We initially considered a train-test split in time, but dismissed this for two reasons. First, the data was collected over three and a half years, and different press configurations. There are inevitable distributional drifts due to changing machine configurations, paper suppliers and process conditions over this time. As such, a temporal split would conflate results by evaluating adaptation to temporal drift instead of generalisation of cause signature. Second, the class imbalance in the data would result in varying distributions of minority class across temporal splits, with unreliable estimates of performance.

We create three event-level folds with the proportion of classes (84% machine-caused and 16% paper-caused) maintained in each fold. Each fold is used as the test set in turn, and the other two folds as the training pool. A randomly selected 20% of the training pool is set aside, stratified to maintain the class distribution. This results in three mutually exclusive sets: training (used to fit the model), validation (used to select operating points and thresholds, and for early stopping) and test (used to report final performance).

4.4.2 Evaluation Metrics

Model performance is primarily assessed using the area under the receiver operating characteristic curve (ROC-AUC). It measures the probability that a randomly chosen paper-caused event receives a higher predicted score than a randomly chosen machine-caused event across all possible classification thresholds, making it robust to class imbalance and independent of any specific operating point. A random classifier achieves AUC 0.5 regardless of class distribution, providing a meaningful lower bound for

comparison. Two secondary metrics are reported at the threshold that maximizes the F1-score. False positive rate which is the fraction of machine-caused breaks that are incorrectly labelled as paper-caused, reflects the operational cost of unnecessary supplier complaints as a machine-caused break wrongly attributed to paper quality triggers an unwarranted claim against the supplier. False negative rate which is the fraction of paper-caused breaks incorrectly labelled machine-caused, reflects the cost of missed detections i.e a paper-caused break that goes unidentified prevents supplier feedback and may result in the same defective material being used in future print runs. Both error types carry distinct operational consequences, motivating their separate reporting alongside the threshold-independent AUC.

4.4.3 Threshold Selection

All models produce continuous probability outputs for the paper-caused class. Model performance is compared using ROC-AUC, computed on the full probability output across all thresholds, making the primary comparison threshold-independent and robust to class imbalance. For operational evaluation, a classification threshold is selected on the validation set and applied to the test set for every model. Two selection criteria are considered: (i) the threshold maximising F1-score on the validation set, and (ii) the threshold maximising paper-caused recall subject to a machine-caused accuracy constraint of at least 90 percent. The 90 percent machine-caused accuracy shows the operational requirement at the facility: machine-caused breaks represent the majority of events, and a high rate of correct machine-caused labelling is needed for an automated system to be trusted in production. The threshold is selected on the validation set within each fold and applied to the test set without refitting, which prevents the threshold inflation. This produces per-model confusion matrices, classification reports, accuracy, and false positive rate at the operationally relevant operating point, enabling direct comparison of each model's practical classification behaviour alongside its threshold-independent AUC.

4.5 Experimental Pipelines

Three experimental pipelines are evaluated, each representing a distinct hypothesis about how web break cause classification is best approached. All pipelines use the same preprocessed data, cross-validation splits, scaling procedure, and evaluation metrics described in Sections 4.2 and 4.4.

4.5.1 End-to-End Deep Learning Pipeline

In this pipeline, the preprocessed (300, 10) multivariate time series is directly transferred to deep learning time series classification models without the need for manual feature engineering. Each model receives the same standardised input and produces a probability score for the paper-caused class. The hypothesis that is tested is, the end-to-end learning from raw defect score sequences is sufficient to capture the discriminative temporal patterns identified in Chapter 3.

All deep learning models in this pipeline use the following shared training configuration: 30 epochs, batch size 64, cross-entropy loss unless stated otherwise, and the Adam optimiser that is implemented internally by sktime and tsai. All models are trained independently within each cross-validation fold using the scaled input described in Section 4.2.5. Performance is evaluated using ROC-AUC on the held-out test fold.

4.5.1.1 CNN-FCN Classifier

The FCN classifier follows the architecture described in Section 2.2, implemented via the sktime FCNClassifier. The (300, 10) input is transposed to (10, 300) for compatibility with the sktime implementation, which applies convolution along the temporal axis. The model uses three convolutional blocks with kernel sizes of 8, 5 and 3 and filter counts of 128, 256 and 128 respectively, each followed by batch normalisation and ReLU activation, and a global average pooling layer producing the event-level representation.

4.5.1.2 InceptionTime

The InceptionTime model follows the architecture described in Section 2.2, implemented via the sktime InceptionTimeClassifier. The (300, 10) input is transposed to (10, 300). The model applies parallel convolutional filters at multiple temporal scales within each Inception module, capturing both short-term fluctuations and longer-range patterns in the defect score sequences.

4.5.1.3 ConvTimeNet

The ConvTimeNet model follows the architecture described in Section 2.2, implemented via the sktime ConvTimeNetClassifier. The model was configured with a patch size of 16, patch stride of 8, hidden dimension `d_model=64`, feedforward dimension `d_ff=128`, and depthwise convolution kernel size 3. The (300, 10) input is transposed to (10, 300) for compatibility with sktime's convention.

4.5.1.4 LSTM-FCN

The LSTM-FCN model follows the architecture described in Section 2.2, implemented via the sktime LSTMFCNClassifier with default parameters. The sktime implementation uses a simplified variant of the original architecture: the squeeze-and-excitation block and dimension shuffle operation are omitted, and the input is passed directly to both branches in parallel. The FCN branch uses three convolutional blocks with kernel sizes 8, 5 and 3 and filter counts of 128, 256 and 128 respectively, each followed by batch normalisation and ReLU activation, and Global Average Pooling. The LSTM branch uses a single LSTM layer with output size 8 and dropout rate 0.8. The outputs of both branches are concatenated before the softmax classification layer.

4.5.1.5 TST - Time Series Transformer

The TST model follows the architecture described in Section 2.2, implemented via the `tsai` library. The model was configured with `d_model=64`, `n_heads=4`, `n_layers=3`, and `dropout=0.2`, giving a key/value dimension of $d_k = d_v = 16$ per head and a feedforward dimension of 256. The (300, 10) input is transposed to (10, 300) for compatibility with `tsai`'s convention. Unlike the original [Z⁺21] et al. architecture, the `tsai` implementation does not apply variable masking during training. A weighted cross-entropy loss is applied with positive class weight equal to the training fold imbalance ratio (≈ 5.25), assigning proportionally higher loss to paper-caused events. Training uses the one-cycle learning rate policy via `fastai` with learning rate 10^{-3} and weight decay 10^{-2} .

- Learning rate: 1×10^{-3}
- Weight decay: 1×10^{-2}
- Loss function: weighted cross-entropy
- Optimisation: one-cycle learning rate policy via `fastai`

Class weights are computed per fold as the ratio of machine-caused to paper-caused events in the training split ($\text{pos_weight} = N_{\text{machine}}/N_{\text{paper}} \approx 5.25$), assigning proportionally higher weight to paper-caused examples.

4.5.1.6 TapNet

The TapNet model follows the architecture described in Section 2.2, implemented via the `sktime` `TapNetClassifier` with default parameters. The random projection groups input dimensions into 3 groups of size $\lfloor 10 \times 2/3 \rfloor = 6$. The LSTM branch uses 128 units with dropout 0.8 and multiplicative self-attention (128 units) before global average pooling. The CNN branch applies three Conv1D layers with filter sizes 256, 256 and 128, all using kernel size 8 and dilation rate 1, followed by multiplicative self-attention and global average pooling. Leaky ReLU activation and batch normalisation are applied after each convolutional layer. The concatenated branch outputs pass through two fully connected layers of size 500 and 300 to produce the final event embedding. A custom weighted cross-entropy loss assigns weight $\approx 5.25 \times$ to paper-caused events to address class imbalance.

4.5.1.7 Note on class weighting

Weighted cross-entropy loss was applied to TapNet but not to the other deep learning models which are CNN, InceptionTime, ConvTimeNet, LSTM-FCN because the `sktime` implementations of those architectures wrap their internal Keras training loops without exposing a configurable loss function parameter. TapNet's `sktime` implementation exposes the loss argument directly, making custom loss injection possible without modifying the underlying library. For the remaining models, class imbalance is addressed through threshold tuning on the validation set, which adjusts the operating point post-hoc without requiring access to the training loss.

4.5.2 XGBoost Pipeline

The XGBoost pipeline tests the domain knowledge hypothesis that manually engineered features encoding expert understanding of the web break process outperform end-to-end learning from raw sequences. The pipeline uses the feature representation described in Section 4.3, derived from the same preprocessed (300, 10) time series used by the deep learning models.

The XGBoost classifier is trained using the following configuration:

- Number of trees: 500
- Maximum tree depth: 5
- Learning rate: 0.05
- Subsample ratio: 0.8
- Column subsample ratio: 0.8
- Early stopping: 30 rounds based on validation AUC

Class imbalance is addressed through the `scale_pos_weight` parameter, set to the ratio of machine-caused to paper-caused events in each training fold:

$$\text{scale_pos_weight} = \frac{\text{\#machine}}{\text{\#paper}} \quad (4.1)$$

This scales the gradient contribution of paper-caused examples during tree construction, improving the model’s ability to learn from underrepresented events.

To quantify the contribution of operational metadata to model performance, an ablation study is conducted comparing the full feature configuration against a time-series-only variant that retains the 63 time-series-derived numerical features and excludes all metadata fields: speed, grammage weight, web width, paper length, detector, printer, paper grade, and paper supplier. All model hyperparameters, cross-validation splits, and evaluation settings are identical between the two variants. This ablation isolates whether the performance advantage of XGBoost over deep learning is attributable to the time-series feature engineering, the metadata, or both.

4.5.3 LLM Classification Pipeline

The LLM pipeline is an experiment which is designed to test two competing hypotheses about why LLM-based classification may underperform on this task, as introduced in Section 2.2. The first hypothesis is that the discretisation process, specifically argmax encoding followed by run-length compression, loses the fine-grained continuous probability dynamics that carry discriminative information between classes. The second hypothesis is that the bottleneck lies not in information loss but in the model’s architectural inability to execute quantitative numerical reasoning on encoded representations. Two configurations of GPT-4o are evaluated to distinguish between these hypotheses.

Input Encoding

Each event's (300, 10) sequence is converted to text by first selecting the dominant defect category at each timestep using argmax over the five score channels from each camera, producing a sequence of 300 categorical labels. Run-length encoding is then applied and consecutive frames with the same dominant label are compressed into a single token with an associated count, producing a compact text string that captures the temporal ordering and duration of defect states within the event.

As a concrete example, a 300-frame sequence where the camera observes 180 consecutive `no_defect` frames, followed by 45 defect frames, followed by 12 `Kantenfehler` frames, followed by 63 tear frames would be encoded as:

`no_defect×180, defect×45, Kantenfehler×12, tear×63`

This compact representation preserves the temporal ordering and duration of defect states but discards all continuous probability information. A text `no_defect` frame with score 0.51 and one with score 0.99 receive the same symbolic label, making the two indistinguishable to the model. Operational metadata including press speed, paper grammage, web width, paper length, detector, printer, grade, and paper supplier is appended to each encoded event as structured text.

Evaluation Sample

Both configurations are evaluated on 100 randomly sampled held-out events drawn from a stratified 80/20 split of the full dataset, containing approximately 89 machine-caused and 11 paper-caused events. The small paper-caused sample means each individual prediction changes paper recall by approximately 9 percentage points; results are therefore indicative rather than statistically definitive. Few-shot examples are drawn exclusively from the training split to prevent data leakage. ROC-AUC is not reported as GPT-4o produces discrete class labels rather than calibrated probability scores.

Configuration v3: Baseline Few-Shot

Configuration v3 establishes the baseline LLM performance under standard few-shot prompting. GPT-4o is prompted with 5 labelled examples per class (10 total) drawn from the training split. The system prompt instructs the model to classify each event as `machine_problem` or `paper_problem` based on the encoded defect sequence and appended metadata. Events are processed in batches of 5 per API call with temperature set to 0 for deterministic output. Performance is evaluated using precision, recall, F1-score, and overall accuracy on the 100-event sample.

Configuration v5: Domain-Enriched Prompt

Configuration v5 is specifically designed to test the second hypothesis: whether the LLM's limitation stems from encoding information loss or from its inability to execute quantitative numerical reasoning. The prompt is expanded to 12 examples per class (24

total) and the system prompt is enriched with XGBoost-derived decision boundaries expressed in natural language, including explicit thresholds on press speed, paper grammage, and defect sequence structure derived from the trained XGBoost model, alongside domain reasoning rules about the physical differences between machine-caused and paper-caused break signatures. By providing the model with the exact numerical decision logic that XGBoost uses to achieve its mean test AUC of 0.86, configuration v5 eliminates information loss as a potential explanation. If performance improves substantially, the primary limitation was encoding. If it does not, the limitation is the model’s inability to act on quantitative rules regardless of the information provided. Events are processed in batches of 10 per API call with temperature 0.

Hypothesis Resolution

Together the two configurations form a controlled experiment. Configuration v3 establishes the baseline LLM performance under standard few-shot encoding. Configuration v5 isolates whether improved domain knowledge in the prompt, specifically the exact numerical thresholds that drive the best-performing model, can compensate for the encoding limitation. If v5 outperforms v3, information loss during encoding is the primary bottleneck. If v5 performs similarly to or worse than v3 despite receiving complete decision-relevant information, the bottleneck is the model’s architectural inability to execute quantitative threshold logic rather than insufficient information access. The results of both configurations and their implications are discussed in Chapter 5.

4.5.4 Chronos Foundation Model

Chronos is a pre-trained time series foundation model based on a T5 encoder-decoder transformer backbone, pre-trained on large-scale time series forecasting corpora as described in Section 2.2. Unlike the task-specific deep learning models evaluated in this thesis, Chronos learns general-purpose temporal representations from diverse real-world and synthetic time series without being designed for any specific domain or task. This experiment evaluates whether representations learned from large-scale forecasting pre-training transfer usefully to the web break cause classification problem, a setting where the pre-training objective differs fundamentally from the target task.

Both configurations use Chronos’s group attention mechanism to process all five defect score channels jointly across each timestep via shared group identifiers passed to the encoder. This preserves the inter-channel relationships in the defect score distribution, specifically the joint evolution of no_defect, defect, and tear scores, which carries class-discriminative information. Both configurations receive single-camera (300, 5) input from camera Seite A.

In the first configuration, the backbone weights are kept frozen and only a lightweight MLP classification head is trained on the extracted encoder embeddings. This tests whether the pre-trained representations are directly useful for cause classification without any adaptation to the target domain.

In the second configuration, the full backbone is fine-tuned end-to-end alongside the classification head. This tests whether the frozen representations can be improved through domain-specific adaptation on the target dataset.

Both configurations are evaluated on a temporal train, validation, and test split of 30, 6, and 6 months respectively, rather than the stratified 3-fold cross-validation applied to all other models. This departure is necessary because Chronos’s pre-training corpus was assembled in chronological order, and using randomly shuffled cross-validation folds would allow the model to be evaluated on events that pre-date its training split, creating a form of temporal leakage inconsistent with real deployment conditions. This means Chronos results are not directly comparable to the 3-fold cross-validation results reported for all other models.

4.6 Model Selection Rationale

The models evaluated in this thesis are selected to provide systematic coverage of the major architectural families for time series classification, as reviewed in Chapter 2. Each model represents a distinct inductive bias about how temporal patterns should be extracted from the defect score sequences.

CNN and InceptionTime represent the convolutional family, testing whether local temporal patterns in the defect score sequence are sufficient for classification. InceptionTime extends the plain CNN with multi-scale parallel filters, allowing it to capture patterns at different temporal resolutions simultaneously. ConvTimeNet represents a more recent hierarchical convolutional approach with adaptive patch-based representations. LSTM-FCN introduces explicit sequential memory through the LSTM branch, enabling modelling of temporal dependencies across the full 300-frame sequence. TST tests whether global self-attention, which can relate any two timesteps directly, improves performance for sequences where early defect patterns connect to late-stage rupture events. TapNet tests whether prototype-based classification at the event embedding level outperforms boundary-based classification on defect score trajectories with distinct class-level temporal signatures. These six deep learning architectures are implemented via the `sktime` and `tsai` libraries, ensuring consistent preprocessing, training, and evaluation conditions across all deep learning models.

Chronos is included as a pre-trained foundation model, testing whether large-scale pre-training on time series forecasting data provides transferable representations for this classification task, implemented via the `chronos-forecasting` library. The LLM pipeline is included as an exploratory approach, testing whether symbolic reasoning over text-encoded defect sequences can contribute to cause classification, with the specific hypothesis being whether the limitation of LLM-based approaches lies in information loss during encoding or in the model’s inability to execute quantitative numerical reasoning.

XGBoost is included to directly test the domain knowledge hypothesis stated in Chapter 1: that features encoding physical understanding of the break process, including slope, onset timing, and temporal build-up shape, are more data-efficient than end-to-end learning on a dataset of 14,073 events. XGBoost is implemented via the native `xgboost` library on a manually engineered tabular feature representation, as no `sktime`-native XGBoost classifier exists for time series classification.

4.7 Handling Class Imbalance

The 5.25:1 class imbalance ratio in the preprocessed dataset is addressed through three complementary strategies applied consistently across all experimental pipelines.

Loss weighting: For TapNet and TST, a weighted cross-entropy loss is applied during training. For TapNet, the class weight for paper-caused events is computed per fold as the ratio of machine-caused to paper-caused events in the training split, giving approximately $5.25\times$ across all folds. TST uses inverse frequency weights derived from the training fold class distribution in the same manner. CNN, InceptionTime, ConvTimeNet, and LSTM-FCN use standard unweighted cross-entropy, as the sktime implementations of these architectures do not expose a configurable loss function parameter; class imbalance for these models is addressed solely through threshold calibration on the validation set.

Gradient reweighting: For XGBoost, the `scale_pos_weight` parameter is set to the training fold imbalance ratio (approximately 5.25), scaling the gradient contribution of minority class examples during tree construction.

Threshold calibration: For all models, the default decision threshold of 0.5 is not used for operational evaluation. Instead, the threshold is selected on the validation set of each fold by maximising the F1-score over the paper-caused class across a grid of 91 thresholds from 0.05 to 0.95 in steps of 0.01, and applied to the test set without refitting. This decouples the classification threshold from the training objective and allows the model's probability output to be calibrated to the actual class distribution and operational requirements.

ROC-AUC as the primary comparison metric is itself robust to class imbalance, evaluating discriminative ability across all thresholds rather than at any single operating point, making it appropriate as the basis for model comparison regardless of the imbalance ratio.

Chapter 5

Results

5.1 Experimental Setup

Following the protocol established in Chapter 4, all models are evaluated. Results are reported as test AUC computed on the held-out test fold of each stratified 3-fold cross-validation split, with classification metrics reported at the F1-optimised threshold selected on the inner validation set. Each fold contains 4,691 test events: 3,941 machine-caused and 750 paper-caused in folds 1 and 2, and 3,940 machine-caused and 751 paper-caused in fold 3. Chronos uses a temporal train/validation/test split of 30/6/6 months described in Section 5.7. The LLM pipeline is evaluated on a 100-event held-out sample due to API cost constraints, as described in Section 5.8.

The metrics used throughout this chapter are defined in Section 4.4.2. ROC-AUC is the primary threshold independent comparison metric; paper precision and recall describe minority class performance at the operational threshold; FPR is the fraction of machine-caused breaks incorrectly labelled paper-caused, corresponding to spurious supplier complaints; and overall accuracy is reported for completeness only, as it is misleading under the 84/16 class imbalance.

5.2 Overview of Results

Table 5.1 presents the complete test AUC results for all 3-fold cross-validation models, ordered by mean AUC. Chronos and the LLM pipeline use different evaluation protocols and are reported separately in Sections 5.7 and 5.8.

Model	Pipeline	Fold 1	Fold 2	Fold 3	Mean AUC	Std
CNN-FCN	Deep Learning	0.68	0.67	0.68	0.68	± 0.0052
TST	Deep Learning	0.67	0.69	0.67	0.67	± 0.0089
ConvTimeNet	Deep Learning	0.68	0.70	0.71	0.70	± 0.0131
LSTM-FCN	Deep Learning	0.71	0.70	0.70	0.70	± 0.0033
InceptionTime	Deep Learning	0.73	0.74	0.71	0.73	± 0.0138
TapNet	Deep Learning	0.76	0.77	0.77	0.77	± 0.0019
XGBoost	Feature Eng.	0.79	0.79	0.79	0.79	± 0.0026
TapNet + meta	Deep Learning	0.82	0.80	0.81	0.81	± 0.0072
XGBoost + meta	Feature Eng.	0.86	0.86	0.86	0.86	± 0.0009

Table 5.1: Test ROC-AUC results for all 3-fold CV models ordered by mean AUC. XGBoost TS-only and TapNet + meta are ablation/extension experiments. Chronos (temporal split) and LLM (100-event sample) use different protocols and are reported in Sections 5.7 and 5.8.

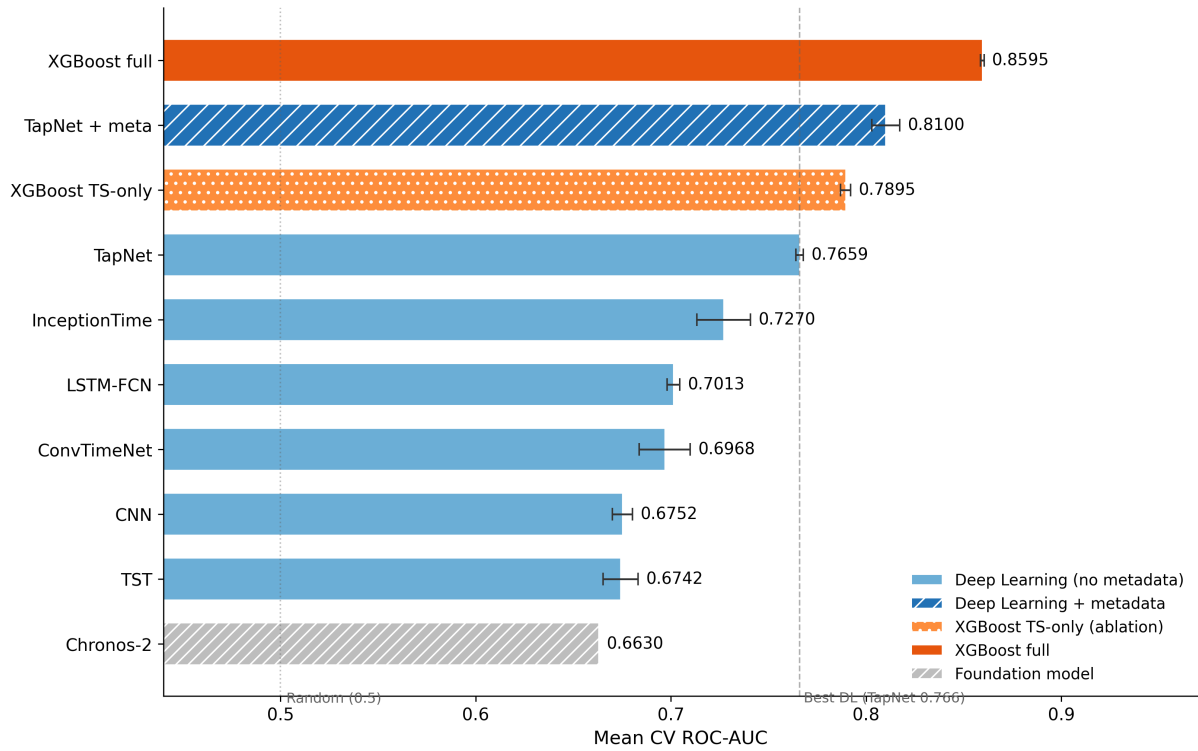


Figure 5.1: Mean test ROC-AUC across all models with standard deviation error bars.

Table 5.1 presents the complete test AUC results for all 3-fold cross-validation models, ordered by mean AUC. Chronos and the LLM pipeline use different evaluation protocols and are reported separately in Sections 5.7 and 5.8

Across all models that produce calibrated probability outputs, mean test AUC ranges from 0.67 (TST) to 0.86 (XGBoost full), with every configuration exceeding the random classifier baseline of 0.5 [Faw06]. This confirms that all evaluated architectures extract some discriminative information from the defect score sequences, and that the task is learnable across fundamentally different modelling paradigms. The variation between 0.67 and 0.86 therefore reflects differences in how effectively each architecture exploits the available signal, rather than whether a signal exists at all.

5.3 Convolutional Models

The following subsections evaluate three convolutional architectures of increasing sophistication: the plain CNN, InceptionTime, and ConvTimeNet. Each represents a different strategy for local temporal pattern detection, and together they establish whether the convolutional family can access the global 300-frame trajectory structure required by this task.

5.3.1 CNN

The CNN classifier achieved a mean test AUC of 0.68 ± 0.0052 across the three folds (0.68, 0.67, 0.68). The narrow standard deviation of ± 0.0052 reflects stable and consistent convergence. Table 5.2 presents the complete per-fold results.

	Fold 1	Fold 2	Fold 3
Threshold	0.27	0.28	0.30
Machine precision	0.90	0.90	0.89
Machine recall	0.66	0.66	0.69
Paper precision	0.26	0.25	0.26
Paper recall	0.62	0.59	0.57
Paper F1	0.36	0.35	0.36
Overall accuracy	0.65	0.65	0.67
FPR	0.34	0.34	0.31
Test AUC	0.68	0.67	0.68

Table 5.2: CNN per-fold classification results at F1-optimised threshold.

In fold 1, of 3,941 machine-caused test events the model correctly labelled 2,588 (65.70%) but incorrectly flagged 1,353 (34.30%) as paper-caused. Of 750 paper-caused events, 464 (61.90%) were correctly detected and 286 (38.10%) were missed.

The mean FPR of 0.33 is the highest of all deep learning models, one in three machine-caused breaks is incorrectly flagged as paper-caused. Paper precision of 0.26 means that of every four events flagged as paper-caused, three are actually machine-caused, making the CNN operationally unusable as a supplier complaint trigger. Overall accuracy of 65.60% confirming that the model is operationally unsuitable at this threshold, the F1-optimised operating point imposes an unacceptable false positive burden relative to its minority class detection rate. The CNN’s discriminative limitation reflects its architectural constraint: the three convolutional blocks use kernel sizes of 8, 5 and 3 timesteps, covering at most 8 consecutive frames out of 300, preventing the model from capturing the global trajectory signals that distinguish the two classes.

5.3.2 InceptionTime

InceptionTime achieved a mean test AUC of 0.73 ± 0.0138 across the three folds (0.73, 0.74, 0.71). The standard deviation of ± 0.0138 is the highest among all deep learning models.

	Fold 1	Fold 2	Fold 3
Threshold	0.12	0.38	0.08
Machine precision	0.89	0.89	0.88
Machine recall	0.89	0.87	0.93
Paper precision	0.42	0.39	0.48
Paper recall	0.43	0.44	0.33
Paper F1	0.42	0.41	0.39
Overall accuracy	0.81	0.80	0.84
FPR	0.11	0.13	0.07
Test AUC	0.73	0.74	0.71

Table 5.3: InceptionTime per-fold classification results at F1-optimised threshold.

In fold 1, of 3,941 machine-caused events 3,498 (88.80%) are correctly labelled and 443 (11.20%) are incorrectly flagged as paper-caused. Of 750 paper-caused events, 320 (42.70%) are correctly detected and 430 (57.30%) are missed. Fold 3 shows a substantially different profile: threshold 0.08 produces the lowest FPR of any convolutional model (0.07) but paper recall drops to 33.00%, meaning two in three paper-caused breaks are missed entirely.

InceptionTime achieves substantially better machine recall (0.90 mean) than CNN (0.67 mean) at the cost of lower paper recall (0.40 mean vs CNN’s 0.60 mean). The threshold range of 0.08 to 0.38 across folds indicates fold-dependent probability calibration. The mean FPR of 0.10 is the lowest among convolutional models. Multi-scale parallel filters allow InceptionTime to capture patterns at different temporal resolutions simultaneously, producing a more stable precision-recall trade-off than the plain CNN.

5.3.3 ConvTimeNet

ConvTimeNet achieved a mean test AUC of 0.70 ± 0.0131 across the three folds (0.68, 0.70, 0.71), with a monotonically increasing trend across folds suggesting sensitivity to fold composition. The standard deviation of ± 0.0131 is the second highest among all deep learning models behind InceptionTime (± 0.0138), consistent with the sensitivity to fold composition noted in the monotonically increasing AUC trend across folds. In practical terms, this means ConvTimeNet’s performance depends significantly on which events appear in the test fold, suggesting that its adaptive patch-based mechanism has not converged to a stable representation of the break cause signal across different subsets of the 14,073 events. The mean log loss of 0.75 ± 0.02 is the highest of all deep learning models, indicating poorly calibrated probability outputs. The sktime ConvTimeNet implementation does not expose the internal probability outputs required to compute per-threshold classification metrics; accordingly only AUC and log loss are reported for this model. Despite being a more recent and architecturally sophisticated model than CNN, ConvTimeNet’s adaptive patch-based mechanism requires more data to discover relevant temporal scales than CNN’s fixed filters, a disadvantage on the 1,201 minority class training examples available per fold.

5.4 Recurrent Models

Recurrent models address the key limitation of convolutional architectures by introducing explicit sequential memory. Rather than detecting local patterns at fixed positions in the sequence, the LSTM branch of LSTM-FCN maintains a hidden state that accumulates information across all 300 timesteps, allowing it to encode the cumulative history of the defect score evolution up to the moment of rupture. This sequential processing is theoretically better suited to capturing the gradual monotonic decline in the no_defect channel that characterises machine-caused breaks, a pattern that spans the full window and cannot be detected by any fixed local filter. The following subsection examines whether this sequential memory translates into a meaningful improvement in classification performance and where its limitations lie on this task.

5.4.1 LSTM-FCN

LSTM-FCN achieved a mean test AUC of 0.70 ± 0.0033 across the three folds (0.71, 0.70, 0.70). The standard deviation of ± 0.0033 is the second lowest among all deep learning models behind TapNet (± 0.0019), reflecting highly consistent convergence across folds.

	Fold 1	Fold 2	Fold 3
Threshold	0.25	0.16	0.11
Machine precision	0.89	0.90	0.91
Machine recall	0.79	0.73	0.66
Paper precision	0.30	0.28	0.27
Paper recall	0.48	0.55	0.64
Paper F1	0.37	0.37	0.37
Overall accuracy	0.74	0.70	0.66
FPR	0.21	0.27	0.34
Test AUC	0.71	0.70	0.70

Table 5.4: LSTM-FCN per-fold classification results at F1-optimised threshold.

In fold 1, of 3,941 machine-caused events 3,111 (78.90%) are correctly labelled and 830 (21.10%) are incorrectly flagged as paper-caused. Of 750 paper-caused events, 363 (48.40%) are correctly detected and 387 (51.60%) are missed. As the threshold decreases from fold 1 (0.25) to fold 3 (0.11), paper recall improves from 48.40% to 63.80% but FPR worsens from 0.21 to 0.34 and machine recall drops from 78.90% to 66.30%. Paper precision remains low across all folds (0.27 to 0.30), meaning only about one in four events flagged as paper-caused is genuine. The mean FPR of 0.27 is the second highest among deep learning models. The progressive threshold drift across folds reflects training instability in the LSTM branch under class imbalance which means the hidden state bottleneck causes progressive attenuation of early-timestep signals via the forget gate, preventing stable probability separation between classes.

5.5 Attention-Based Models

Attention-based models represent the most expressive architectural family evaluated in this study. By applying self-attention directly across all 300 timesteps simultaneously, the Time Series Transformer can in principle relate any two points in the defect score sequence to each other, including the early no defect decline at $t = -299$ and the terminal tear spike at $t = 0$, which is precisely the long-range dependency that distinguishes the two break causes. Whether this theoretical expressiveness translates into practical discriminative ability on a dataset with 1,201 minority class training examples per fold is the question examined in the following subsection.

5.5.1 TST

TST achieved a mean test AUC of 0.67 ± 0.0089 across the three folds (0.67, 0.69, 0.67). The standard deviation of ± 0.0089 reflects moderate fold-to-fold variation, driven primarily by the qualitatively different probability distribution produced in fold 2 compared to folds 1 and 3. In the context of this task, this instability is operationally significant as a model whose threshold must shift from 0.40 in two folds to 0.17 in a third to achieve reasonable minority class recall cannot be reliably calibrated for deployment, as the operating point selected on a validation set may not transfer to new production data.

	Fold 1	Fold 2	Fold 3
Threshold	0.40	0.17	0.40
Machine precision	0.88	0.90	0.89
Machine recall	0.82	0.67	0.81
Paper precision	0.31	0.26	0.31
Paper recall	0.43	0.62	0.46
Paper F1	0.36	0.37	0.37
Overall accuracy	0.75	0.66	0.75
FPR	0.18	0.33	0.19
Test AUC	0.67	0.69	0.67

Table 5.5: TST per-fold classification results at F1-optimised threshold.

In fold 2, of 3,941 machine-caused events only 2,626 (66.60%) are correctly labelled and 1,315 (33.40%) are incorrectly flagged as paper-caused FPR 0.33. Of 750 paper-caused events, 465 (62.00%) are correctly detected and 285 (38.00%) are missed. This contrasts sharply with folds 1 and 3, which both use threshold 0.40 and produce FPR approximately 0.19 and paper recall approximately 0.44. The fold 2 divergence threshold 0.17 versus 0.40 in the other folds reflects training instability: the one-cycle learning rate schedule interacts differently with the weighted cross-entropy loss in fold 2, producing a qualitatively different probability distribution despite a similar AUC value (0.69 vs 0.67 and 0.67). Paper precision is consistently the lowest of any model at comparable recall levels (0.26 to 0.31), and the mean AUC of 0.67 is effectively identical to CNN (0.68) despite TST’s global self-attention mechanism, confirming that the attention mechanism is underspecified relative to the available minority class data at this training set size.

5.6 Prototype-Based Models

The prototype-based model family takes a fundamentally different approach to classification from the convolutional, recurrent, and attention-based architectures evaluated in the preceding sections. Rather than learning a decision boundary that separates the two classes in feature space, TapNet learns a compact embedding of each event’s full 300-frame defect trajectory and classifies by proximity to learned class prototypes. This approach is theoretically better suited to the web break classification task, where the two classes are distinguished by the global shape of their trajectories rather than by local frame-level patterns, and where the minority class is small enough that boundary-based methods struggle to estimate a reliable decision surface. The results confirm this theoretical expectation: TapNet achieves the best deep learning performance in this study, both without and with metadata access.

5.6.1 TapNet

TapNet achieved the best deep learning result among models without metadata, with a mean test AUC of 0.77 ± 0.0019 across the three folds (0.76, 0.77, 0.77). The standard deviation of ± 0.0019 is the lowest among all deep learning models, indicating that prototype-based classification produces highly consistent performance across fold compositions, a property that distinguishes TapNet from the convolutional and recurrent models. For the web break classification task, this stability is meaningful, it suggests that the class prototypes learned from the 300-frame defect trajectories capture a genuine and reproducible signal rather than fold-specific patterns, making TapNet a more reliable candidate for deployment than the other deep learning architectures.

	Fold 1	Fold 2	Fold 3
Threshold	0.58	0.59	0.65
Machine precision	0.91	0.91	0.90
Machine recall	0.81	0.85	0.91
Paper precision	0.36	0.41	0.47
Paper recall	0.57	0.53	0.45
Paper F1	0.44	0.46	0.46
Overall accuracy	0.77	0.80	0.83
FPR	0.19	0.15	0.10
Test AUC	0.76	0.77	0.77

Table 5.6: TapNet per-fold classification results at F1-optimised threshold.

In fold 1, of 3,941 machine-caused events 3,181 (80.70%) are correctly labelled and 760 (19.30%) are incorrectly flagged as paper-caused FPR 0.19. Of 750 paper-caused events, 429 (57.20%) are correctly detected and 321 (42.80%) are missed. Fold 3 shows a notably different profile: threshold 0.65 produces FPR 0.10, the lowest of any baseline deep learning fold, but paper recall drops to 45.00%.

Compared to all other deep learning models without metadata, TapNet produces substantially higher thresholds (0.58 to 0.65 vs 0.08 to 0.40 for convolutional and recurrent models), indicating that prototype-based classification at the event embedding level generates better-separated probability distributions. Paper precision improves sub-

stantially across folds (0.36 to 0.47) as the threshold rises, reflecting that fold 3’s more conservative operating point yields cleaner positive predictions. The mean paper F1 of approximately 0.46 is the highest among all deep learning models without metadata. Prototype-based classification separating events by their overall 300-frame trajectory shape rather than by local frame patterns provides the most effective deep learning representation for this task without metadata.

5.6.2 TapNet with Early Metadata Fusion

To evaluate whether the same metadata that benefits XGBoost can also improve deep learning performance, an early fusion experiment was conducted with TapNet. The eight metadata fields used in the XGBoost full model five numeric (speed, grammage weight, web width, paper length, detector) and three label-encoded categorical (printer, grade, paper supplier) were repeated across all 300 timesteps and concatenated to the time series input, expanding the input shape from (300, 10) to (300, 18). TapNet was trained with the same weighted cross-entropy loss, 30 epochs, batch size 64, and evaluation protocol as the baseline experiment.

	Fold 1	Fold 2	Fold 3
Threshold	0.55	0.57	0.67
Machine precision	0.92	0.92	0.90
Machine recall	0.85	0.86	0.90
Paper precision	0.43	0.44	0.49
Paper recall	0.60	0.58	0.50
Paper F1	0.50	0.50	0.49
Overall accuracy	0.80	0.81	0.84
FPR	0.15	0.14	0.10
Test AUC	0.82	0.80	0.81

Table 5.7: TapNet + metadata (early fusion, input shape (300, 18)) per-fold classification results at F1-optimised threshold.

In fold 1, of 3,941 machine-caused events 3,334 (84.60%) are correctly labelled and 607 (15.40%) are incorrectly flagged as paper-caused, FPR 0.15, an improvement over baseline TapNet’s fold 1 FPR of 0.19. Of 750 paper-caused events, 453 (60.40%) are correctly detected and 297 (39.60%) are missed. Fold 3 produces the highest threshold of any TapNet fold (0.67), the lowest FPR (0.10), and paper recall of 49.80%.

Early metadata fusion improves mean AUC from 0.77 to 0.81, a gain of +0.04 AUC points. The standard deviation of ± 0.0072 is higher than baseline TapNet (± 0.0019), reflecting modest sensitivity to fold composition introduced by the additional metadata channels. This increase in variance is expected: categorical fields such as printer identity and paper grade vary in their distribution across folds depending on which press configurations and paper types appear in each test split, introducing fold-dependence that was not present in the camera-only baseline. Mean FPR improves from 0.15 to 0.13. Paper precision improves substantially across all folds (from 0.36 to 0.47 to 0.43 to 0.49), and paper F1 improves from the 0.44 to 0.46 range to 0.49 to 0.50. The comparison with XGBoost is presented in Section 5.9.2.

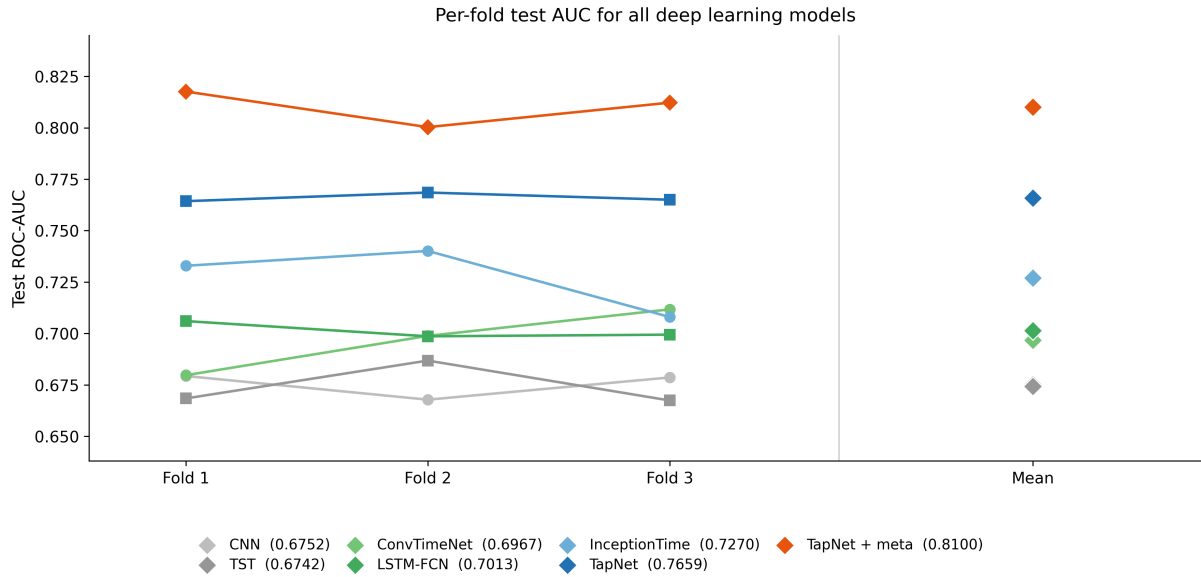


Figure 5.2: Per-fold test AUC results for all deep learning models. Diamond markers show mean AUC across three folds.

5.7 Foundation Model - Chronos

Chronos was evaluated using a temporal train/validation/test split of 30/6/6 months, yielding 8,563 training events, 2,183 validation events, and 2,948 test events, using single-camera (300, 5) input from camera Seite A. Two configurations were evaluated, both using Chronos’s group attention mechanism to process all five defect score channels jointly at each timestep, preserving inter-channel relationships. In the frozen configuration the backbone weights are fixed and only a lightweight MLP classification head is trained. In the fine-tuned configuration the full backbone is updated end-to-end alongside the head.

Configuration	Val AUC	Test AUC	Threshold	Test FPR	Pap. recall	Pap. F1
Frozen group attn	0.68	0.66	0.16	0.19	0.38	0.33
Fine-tuned group attn	0.71	0.66	0.06	0.20	0.38	0.32

Table 5.8: Chronos results under temporal split evaluation

For the frozen configuration, of 2,948 test events 1,983 machine-caused events (81.40%) are correctly labelled and 453 (18.60%) are incorrectly flagged as paper-caused. Of 512 paper-caused test events, 193 (37.70%) are correctly detected and 319 (62.30%) are missed. The fine-tuned configuration produces an essentially identical test AUC of 0.66, a gain of only 0.00 despite the substantially greater computational cost. The higher validation AUC of the fine-tuned model (0.71 vs 0.68) without a corresponding test improvement indicates overfitting to the validation period rather than genuine generalisation improvement. The fine-tuned model requires a threshold of 0.06 vs 0.16 for the frozen model, indicating its probability scores are compressed toward zero, consistent with a model that has adapted its outputs to the validation distribution but not the test distribution.

Both configurations underperform all task-specific deep learning models trained from scratch, including the plain CNN (0.68 under 3-fold CV). A test AUC of 0.66 repre-

sents discriminative ability only 16 percentage points above chance, insufficient for reliable cause attribution. Chronos’s forecasting pre-training objective, specifically autoregressive next-token prediction instilling priors for local temporal continuity, is fundamentally misaligned with the global trajectory classification task required here, and fine-tuning cannot overcome this misalignment at this dataset size.

5.8 LLM Classifier

Two configurations of GPT-4o were evaluated on 100 randomly sampled, held-out events using run-length encoded defect sequences combined with operational metadata. The 100-event sample contains 89 machine-caused and 11 paper-caused events. ROC-AUC is not reported as both configurations produce discrete class labels rather than calibrated probability scores.

	v3 - 5-shot GPT-4o	v5 - 12-shot + domain rules
Machine precision	0.91	0.89
Machine recall	0.66	0.78
Paper precision	0.14	0.09
Paper recall	0.46	0.18
Paper F1	0.22	0.12
Overall accuracy	0.64	0.71

Table 5.9: LLM classifier results on 100 held-out events. Confusion matrices: v3 - machine [59 correct, 30 wrong], paper [5 correct, 6 missed]; v5 - machine [69 correct, 20 wrong], paper [2 correct, 9 missed].

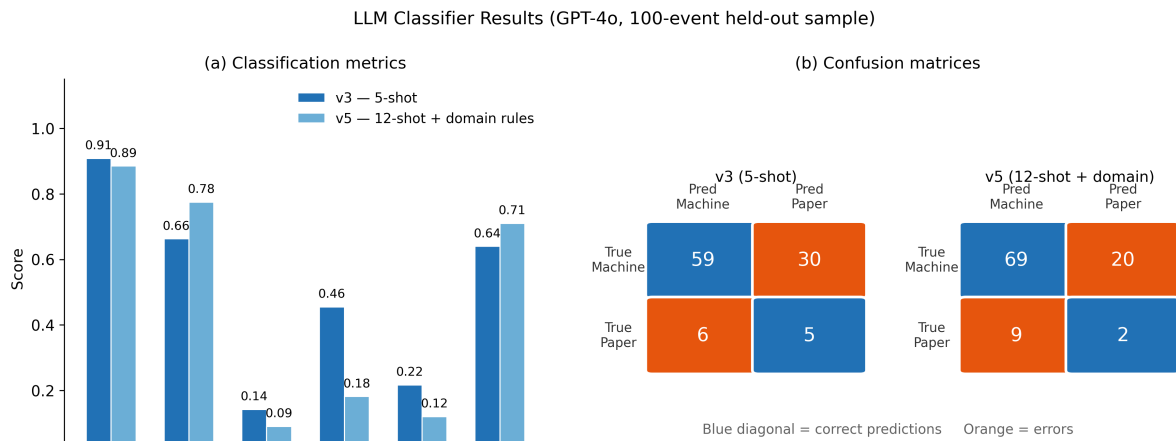


Figure 5.3: LLM classification results: metric comparison and confusion matrices for both configurations.

In v3, of 89 machine-caused events 59 (66.30%) are correctly classified and 30 (33.70%) are incorrectly flagged as paper-caused. Of 11 paper-caused events 5 (45.50%) are correctly detected and 6 (54.50%) are missed. In v5, the domain-enriched prompt with XGBoost decision boundaries shifts the model strongly toward the majority class: 69 of 89 machine-caused events are correctly classified, but only 2 of 11 paper-caused events are detected. Overall accuracy improves from 64.00% to 71.00% but paper recall

collapses from 45.50% to 18.20% and paper F1 drops from 0.22 to 0.12. Adding domain rules derived from XGBoost thresholds causes the model to over-apply majority-class priors, reducing sensitivity to the minority class. Paper precision of 0.14 in v3 and 0.09 in v5 are operationally unusable. Both configurations fall substantially below the CNN baseline.

5.9 XGBoost - Feature Engineering Pipeline

The XGBoost pipeline represents a fundamentally different approach from the end-to-end deep learning models evaluated in the preceding sections. Rather than learning representations directly from the raw time series, XGBoost operates on a manually engineered tabular feature vector that encodes domain knowledge of the web break process explicitly, as described in Section 4.3. Two configurations are evaluated: the full feature set combining time-series-derived features with operational metadata, and a time-series-only ablation variant that excludes all metadata fields. Together these two configurations isolate the contribution of metadata to the overall performance advantage and directly address RQ3 and RQ4.

5.9.1 Full Feature Set

XGBoost with the full feature set achieved the best overall performance with a mean test AUC of 0.86 ± 0.0009 across the three folds (0.86, 0.86, 0.86). The feature set comprises 68 numerical features derived from the preprocessed (300, 10) time series plus three categorical variables encoded via one-hot encoding, giving approximately 80 to 90 total features after encoding.

	Fold 1	Fold 2	Fold 3
Threshold	0.61	0.50	0.54
Machine precision	0.92	0.93	0.93
Machine recall	0.90	0.85	0.89
Paper precision	0.54	0.47	0.53
Paper recall	0.60	0.68	0.63
Paper F1	0.57	0.55	0.57
Overall accuracy	0.85	0.82	0.85
FPR	0.10	0.15	0.11
Test AUC	0.86	0.86	0.86

Table 5.10: XGBoost full (TS + metadata) per-fold classification results at F1-optimised threshold.

In fold 1, of 3,941 machine-caused test events 3,557 (90.30%) are correctly labelled and 384 (9.70%) are incorrectly flagged as paper-caused. Of 750 paper-caused events 449 (59.90%) are correctly identified and 301 (40.10%) are missed. This is the first model where the classification threshold exceeds 0.50 in all folds (0.50 to 0.61), indicating that XGBoost assigns genuinely high probability scores to paper-caused events rather than requiring an artificially lowered threshold to detect them. The mean paper recall of 0.63 is the highest of all models. The mean FPR of 0.12 is the lowest of all models. The mean

paper F1 of approximately 0.56 is the highest of all models. The standard deviation of ± 0.0009 is by far the smallest of any model evaluated, demonstrating exceptional cross-fold stability.

5.9.2 Ablation Study - Metadata Contribution

To quantify the contribution of operational metadata, the full feature set is compared against a time-series-only variant retaining only the 63 time-series-derived numerical features, with all metadata fields excluded (speed, grammage weight, web width, paper length, detector, printer, paper grade, and paper supplier).

	Fold 1	Fold 2	Fold 3
Threshold	0.59	0.50	0.57
Machine precision	0.91	0.92	0.91
Machine recall	0.83	0.76	0.85
Paper precision	0.40	0.34	0.41
Paper recall	0.58	0.66	0.54
Paper F1	0.47	0.45	0.47
Overall accuracy	0.79	0.74	0.80
FPR	0.17	0.25	0.15
Test AUC	0.79	0.79	0.79

Table 5.11: XGBoost TS-only (no metadata) per-fold classification results at F1-optimised threshold.

In fold 1 without metadata, of 3,941 machine-caused events 3,280 (83.20%) are correctly labelled and 661 (16.80%) are incorrectly flagged as paper-caused, FPR 0.17, substantially higher than the full model's 0.10. Of 750 paper-caused events, 434 (57.90%) are correctly detected and 316 (42.10%) are missed. The standard deviation of ± 0.0026 reflects highly consistent performance across folds, comparable to the full XGBoost model (± 0.0009), confirming that the cross-fold stability of the gradient boosted pipeline does not depend on the inclusion of metadata. This is important for the ablation interpretation: the 0.07 AUC gap between XGBoost TS-only and XGBoost full is not a sampling artefact driven by fold variation, but a stable and reproducible difference that metadata consistently contributes across all three test splits.

Table 5.12 compares all four configurations to isolate the metadata contribution across both model families.

Configuration	Fold 1	Fold 2	Fold 3	Mean AUC
TapNet (no metadata)	0.76	0.77	0.77	0.77 ± 0.0019
XGBoost TS-only (no metadata)	0.79	0.79	0.79	0.79 ± 0.0026
TapNet + metadata	0.82	0.80	0.81	0.81 ± 0.0072
XGBoost full (TS + metadata)	0.86	0.86	0.86	0.86 ± 0.0009
Metadata Δ AUC - TapNet			+0.04	
Metadata Δ AUC - XGBoost			+0.07	

Table 5.12: Metadata contribution comparison across both model families. The same eight metadata fields are used in both TapNet early fusion and XGBoost full.

Removing metadata reduces XGBoost AUC from 0.86 to 0.79, a drop of 0.07 AUC points. Without metadata, XGBoost TS-only (0.79) outperforms TapNet baseline (0.77) by 0.02 AUC points, indicating that engineered time-series features alone provide a modest but measurable advantage over prototype embeddings on this task. Adding metadata improves TapNet by +0.04 AUC points (from 0.77 to 0.81) and XGBoost by +0.07 AUC points (from 0.79 to 0.86). The metadata contribution to XGBoost (0.07) is 1.6 times larger than its contribution to TapNet (0.04), demonstrating that explicit feature engineering is a substantially more effective mechanism for exploiting operational metadata than early fusion into a deep learning backbone. The remaining gap between TapNet + metadata (0.81) and XGBoost full (0.86) is 0.05 AUC points, suggesting that the XGBoost advantage is driven jointly by both superior metadata exploitation and the explicit time-series feature engineering. The interpretation of these findings is discussed in Chapter 6.

5.9.3 Feature Importance Analysis

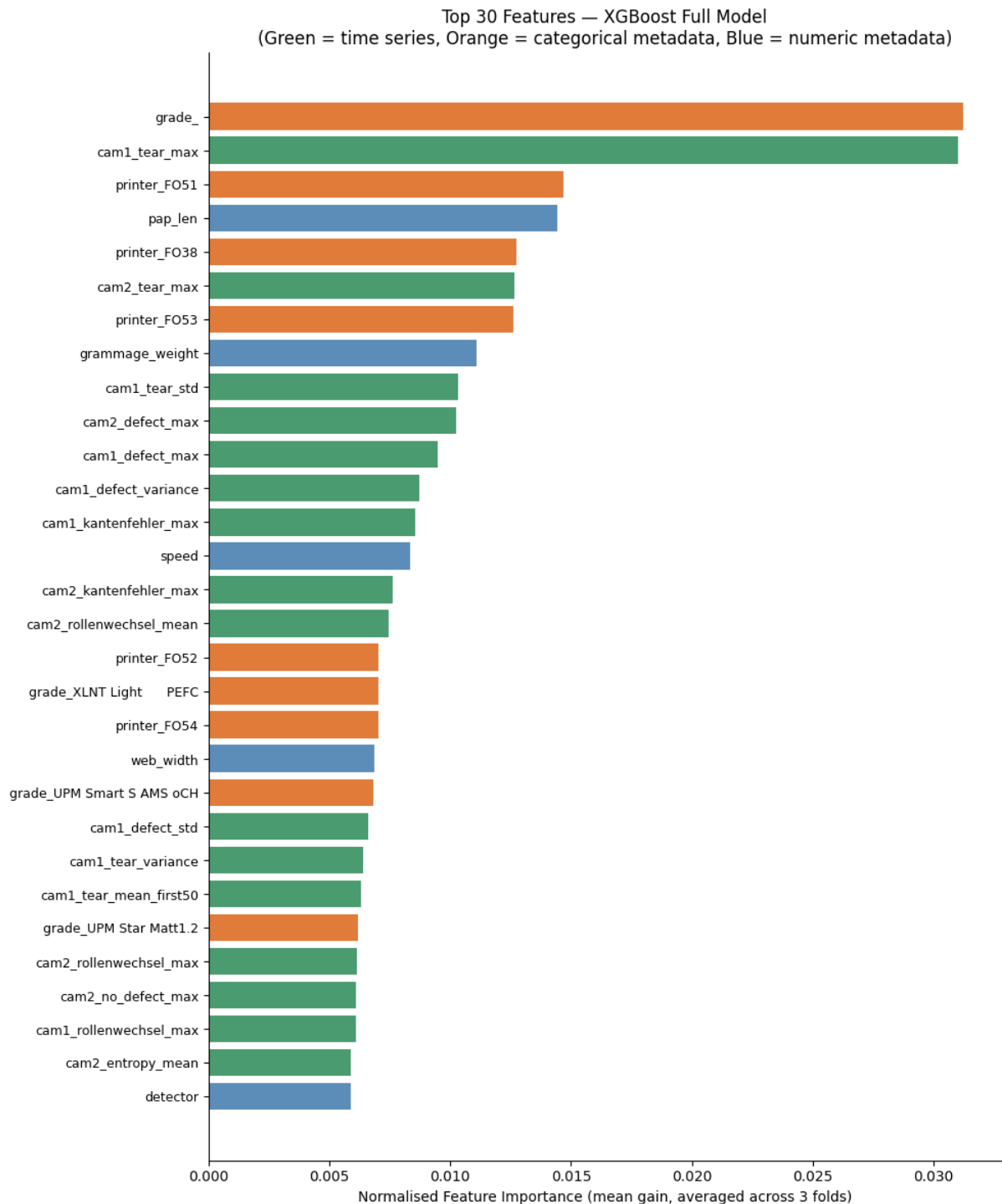


Figure 5.4: Top 30 features by normalised mean gain importance from the XGBoost full model, averaged across three cross-validation folds. Green bars indicate time-series-derived features, orange bars indicate categorical metadata features, and blue bars indicate numeric metadata features.

Figure 5.4 presents the top 30 features by normalised mean gain importance from the XGBoost full model, averaged across the three cross-validation folds. Green bars indicate time-series-derived features, orange bars indicate categorical metadata features

(printer identity and paper grade), and blue bars indicate numeric metadata features (speed, grammage weight, web width, paper length, and detector).

The importance distribution confirms the joint contribution of both feature groups established by the ablation study in Section 5.9.2. The two highest-ranked features are a paper grade category (`grade_unknown`, importance 0.031) and the maximum tear score from camera 1 (`cam1_tear_max`, importance 0.031), ranked essentially equal. This near-tie between the top metadata feature and the top time-series feature directly illustrates why neither information source alone is sufficient: the ablation confirms that removing metadata drops AUC by 0.07 points, while the TS-only model still outperforms the best deep learning model, showing that both contribute independently.

Among the top 30 features, categorical metadata fields dominate the high-importance positions: paper grade categories and printer identity features (`printer_F051`, `printer_F038`, `printer_F053`) occupy three of the top seven ranks, consistent with the domain expectation that certain press configurations and paper grades are systematically associated with higher rates of paper-caused failure. Numeric metadata fields including `pap_len` (rank 4), `grammage_weight` (rank 8), `speed` (rank 14), `web_width` (rank 20), and `detector` (rank 30) are distributed throughout the ranking, confirming that structured operational context contributes at multiple importance levels rather than being concentrated in a single field.

Time-series features are interleaved throughout the top 30, with `cam1_tear_max` and `cam2_tear_max` ranking second and sixth respectively, reflecting the discriminative value of the peak tear intensity captured in the 300-frame pre-break window. Variance and standard deviation features of the defect and tear channels also appear consistently, capturing the trajectory stability differences between the gradual machine-caused deterioration pattern and the sudden paper-caused failure identified in Section 3.4.2. The presence of `cam1_kantenfehler_max` and `cam2_rollewechsel_mean` in the top 30 confirms that non-tear defect channels also carry discriminative information that prototype-based deep learning models must discover implicitly but that XGBoost can exploit directly through explicit statistical summaries.

The balanced interleaving of metadata and time-series features throughout the top 30 is consistent with the ablation finding that the XGBoost advantage over deep learning is driven jointly by both components rather than by either alone.

Figure 5.5 visualises the first decision tree (tree 0) from the XGBoost ensemble trained on fold 1, providing a mechanistic view of how the model arrives at individual predictions. The root split on `cam1_max_tear`, the maximum tear score recorded by camera 1 across the 300-frame window, reflects the single most discriminative condition in this tree, consistent with its top ranking in the feature importance analysis. Events where `cam1_max_tear` falls below 0.579 standardised units take the left branch, where subsequent splits examine camera-derived signals including `cam2_mean_rolle`, `cam2_std_kanten`, and `cam1_std_tear`, reflecting the model's attempt to characterise the defect trajectory shape in the absence of a strong tear peak. Events where `cam1_max_tear` exceeds this threshold take the right branch, where the model immediately interrogates operational metadata, `pap_len` at node 2 and `grammage_weight` at node 14, to refine the classification once a prominent tear signal has already been detected. Paper grade categories appear as binary splits at nodes 6, 8, and 9, confirming that categorical metadata is actively used in the decision logic rather than being treated as noise. All split thresh-

olds are expressed in standardised units following StandardScaler preprocessing. This single tree contributes a small additive score at each leaf; the final paper-caused probability is the sigmoid-transformed sum of scores across all approximately 250 trees in the ensemble.

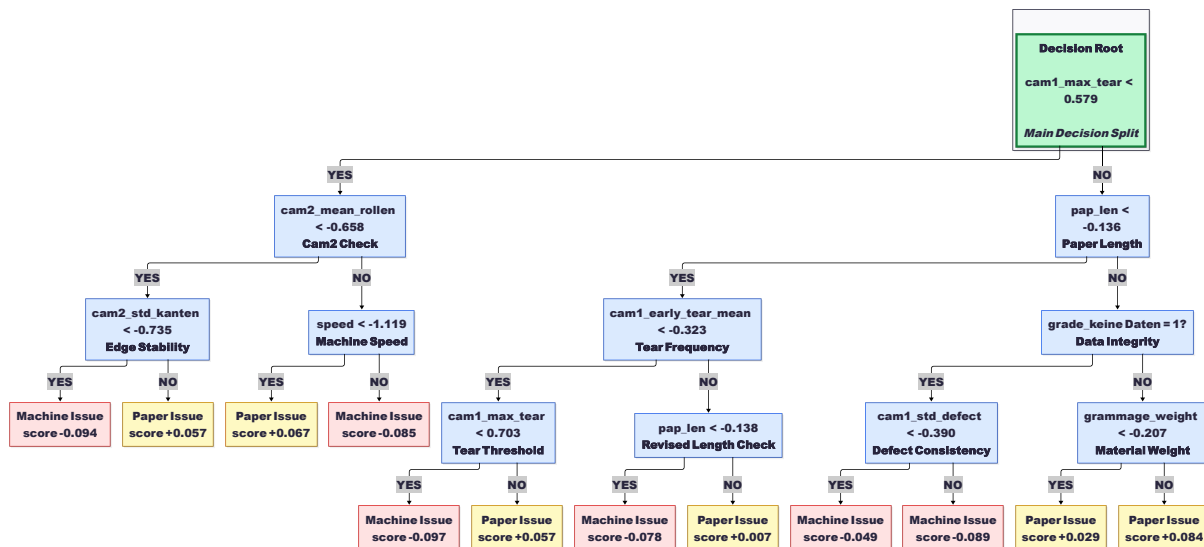


Figure 5.5: The first decision tree (tree 0) from the XGBoost ensemble trained on fold 1. Split thresholds are in standardised units following StandardScaler preprocessing. The root split on `cam1_max_tear` is consistent with the feature importance analysis. Camera time series features dominate the upper levels of the tree while metadata fields (`pap_len`, `speed`, `grammage_weight`, paper grade) appear at deeper levels. Green leaves indicate a positive score contributing towards paper-caused classification and red leaves a negative score towards machine-caused classification. The final prediction is the sigmoid-transformed sum of scores across all trees in the ensemble.

5.10 Summary and Comparison

Table 5.13 summarises all models and configurations with their key operational metrics.

Model	Pipeline	Mean AUC $\pm$ Std	Mean FPR	Pap. recall	Pap. F1
CNN	DL	0.68 ± 0.0052	0.33	0.60	0.36
TST	DL	0.67 ± 0.0089	0.24	0.50	0.37
ConvTimeNet	DL	0.70 ± 0.0131	-	-	-
LSTM-FCN	DL	0.70 ± 0.0033	0.27	0.56	0.37
InceptionTime	DL	0.73 ± 0.0138	0.10	0.40	0.41
TapNet	DL	0.77 ± 0.0019	0.15	0.52	0.46
XGBoost TS-only	Feat.Eng.	0.79 ± 0.0026	0.19	0.59	0.46
TapNet + meta	DL + meta	0.81 ± 0.0072	0.13	0.56	0.50
XGBoost full	Feat.Eng.	0.86 ± 0.0009	0.12	0.63	0.56
Chronos (frozen)*	Foundation	0.66	0.19	0.38	0.33
LLM v3 [†]	LLM	-	-	0.46	0.22
LLM v5 [†]	LLM	-	-	0.18	0.12

Table 5.13: Summary of all models and configurations. *Chronos uses temporal split evaluation (not comparable to 3-fold CV). [†]LLM evaluated on 100-event sample without probability outputs. ConvTimeNet threshold-based metrics unavailable due to implementation constraints.

The results organise into four performance tiers.

All models producing probability outputs exceed the AUC = 0.5 random baseline, confirming task learnability across every modelling paradigm evaluated. The results organise into four performance tiers.

Tier 1 - Feature-engineered pipeline with metadata: XGBoost full at 0.86 ± 0.0009 . Best AUC, lowest FPR, highest paper recall and F1, and the smallest cross-fold standard deviation of any model.

Tier 2 - Metadata-augmented deep learning: TapNet + metadata at 0.81 ± 0.0072 . A meaningful improvement of +0.04 AUC points over baseline TapNet, but still 0.05 AUC points below XGBoost full.

Tier 3 - Mid-range: XGBoost TS-only (0.79), TapNet baseline (0.77), and InceptionTime (0.73). XGBoost without metadata outperforms TapNet by 0.02 AUC points, confirming that engineered time-series features provide a modest advantage over prototype embeddings even without metadata. TapNet is the best deep learning model without metadata.

Tier 4 - Weak models: LSTM-FCN, CNN, ConvTimeNet, TST, Chronos, and the LLM configurations. All fall below 0.71 AUC and provide limited operational utility.

The central empirical finding is that XGBoost with domain-informed features and operational metadata exceeded the best unaided deep learning result (TapNet, 0.77) by 0.09 AUC points and exceeded the best metadata-aided deep learning result (TapNet + metadata, 0.81) by 0.05 AUC points. The metadata contribution to XGBoost (+0.07) is 1.6 times larger than its contribution to TapNet via early fusion (+0.04), demonstrating that explicit feature engineering provides a substantially more effective mechanism for exploiting operational metadata than early fusion into a deep learning backbone.

Chapter 6

Discussion

6.1 Interpretation of Results: The Performance Hierarchy

A coherent and interpretable story about the nature of the web break classification problem is presented in the results of Chapter 5. The performance hierarchy is not arbitrary, each tier corresponds to a fundamentally different hypothesis about how to access the discriminative information in a 300-frame defect score sequence, and the hierarchy reflects precisely how well each assumption matches the actual structure of the data.

As established in Chapter 3, the two classes are distinguished by global temporal signatures. Machine-caused breaks exhibit gradual progressive deterioration of the no_defect score across the full 300-frame window, a trend that begins early and builds continuously towards the tear peak. Paper-caused breaks exhibit sudden late-stage failure and the defect signal remains largely absent until the final frames before rupture, at which tear scores spike sharply. These two signatures are structurally different at the event level, not the frame level. A model that operates on local temporal windows or individual frames cannot access this distinction reliably, but the one that operates on global trajectory-level representations can, and this observation explains every result in the hierarchy and forms the central interpretive framework for this chapter.

6.2 Performance of Local Pattern Detection Models

This section examines four architectures that fail to reliably access the global 300-frame trajectory structure required by this task: CNN, InceptionTime, ConvTimeNet, and TST. The first three rely on local or fixed-scale convolutional pattern detection, while TST's global self-attention is theoretically capable of long-range modelling but likely fails in practice due to data limitations. Despite spanning different architectural families, all four share the consequence that they cannot consistently distinguish the gradual machine-caused deterioration trajectory from the sudden paper-caused failure pattern. Their performance, ranging from AUC 0.67 to 0.73, establishes the lower bound of what is achievable without explicit global trajectory modelling or domain-informed feature engineering.

6.2.1 CNN

CNN-FCN model achieved a mean test AUC of 0.67, the weakest result among all models with per-fold classification metrics. The architectural reason is, the three convolutional blocks use kernel sizes of 8, 5 and 3 time steps respectively, meaning each filter detects patterns within a window of at most 8 consecutive frames out of 300, covering only 2.7% of the input sequence. Global average pooling then averages these local activations across all timestep positions to produce the event-level representation. The consequence is structural, as CNN can only ask whether there are short-term defect fluctuations at any position in the sequence. It cannot ask whether the no_defect score shows a consistent downward trend across the entire 300-frame window. The first question is insufficient to distinguish the two classes as both exhibit local defect fluctuations. The second question is precisely what separates them. Global average pooling collapses the temporal structure entirely before classification, thus likely destroying the sequential information that differentiates a gradual slope from a sudden spike. The confusion matrices quantify this directly. Across all three folds the CNN requires thresholds of 0.27-0.30, which is well below the default 0.5, likely because the model uniformly assigns low probability to the paper-caused class regardless of the true event structure, thereby inflating the FPR to 0.332 which is the highest of all deep learning models. At these aggressive thresholds, machine recall drops to a mean of 66.8% which means one in three machine-caused breaks is incorrectly labeled paper-caused while paper-caused tear precision is only 25.4% that means out of every four events flagged as paper-caused, three are actually machine-caused. Overall accuracy of 65.6% falls below the naive majority classifier (84%).

6.2.2 ConvTimeNet

ConvTimeNet with an AUC of around 0.69 underperforms InceptionTime’s AUC of 0.73 despite being a more recent and architecturally sophisticated model. ConvTimeNet’s deformable patch-based representations theoretically offer greater expressiveness, the model learns where in the sequence to look and at what temporal scale, rather than applying fixed-size filters uniformly. This adaptive design should in principle provide better coverage of the global 300-frame trajectory.

The failure is likely a data efficiency problem. Learning effective patch positions and scales requires the model to first discover that the relevant information spans the full 300 frames and that the slope of no_defect is the key discriminative feature and a meta-learning task that demands substantially more labelled examples than applying fixed filters. With approximately 7,505 training events per fold and only 1,201 paper-caused training examples, the adaptive patch mechanism is likely severely underdetermined. InceptionTime’s fixed multi-scale filters provide guaranteed multi-resolution coverage without requiring the model to discover which scales matter which is a crucial advantage when minority class examples are scarce. The monotonically improving AUC trend across folds ($0.68 \rightarrow 0.70 \rightarrow 0.71$) is likely attributable to fold-to-fold variation in minority class composition rather than any systematic learning progression. This result illustrates a general principle that architectural sophistication does not guarantee data efficiency, and on small or imbalanced industrial datasets, simpler models with appropriate fixed inductive biases often outperform adaptive models, that must learn

their own structure from limited data.

6.2.3 InceptionTime

InceptionTime achieved a mean AUC of 0.73, the best result among the convolutional architectures and a meaningful improvement of 0.05 AUC points over the plain CNN (0.68). Its multi-scale parallel filter design, applying convolutional kernels of 10, 20 and 40 timesteps simultaneously within each Inception module, allows it to capture temporal patterns at short, medium and long scales without requiring the model to discover which scales matter. This fixed multi-resolution coverage is the key advantage over both the plain CNN, which is limited to 8-timestep kernels, and ConvTimeNet, whose adaptive patch mechanism must learn the relevant scales from data.

The improvement over CNN is real but bounded. With kernel sizes of up to 40 timesteps, InceptionTime covers 13% of the 300-frame window at its widest scale, compared to 2.70% for the CNN. This is a meaningful expansion but it still likely falls far short of the full 300-frame global trajectory. The model can detect whether tear scores escalate over a 40-frame window near the end of the sequence, but it cannot likely compare the no_defect slope across the first 100 frames with the tear activity in the last 50 frames, which is the comparison that most cleanly separates the two classes.

The fold-to-fold standard deviation of ± 0.0138 is the highest among all deep learning models, reflecting sensitivity to the minority class composition of each fold. This variance is consistent with a model that has partially learned the discriminative signal, capturing some of the global trajectory shape through its wider kernels, but incompletely enough that performance is sensitive to exactly which paper-caused events appear in the training set. InceptionTime therefore represents the ceiling of what local multi-scale convolutional pattern detection can achieve on this task without explicit global trajectory features or metadata.

6.2.4 TST

TST (AUC 0.67) is the most counterintuitive result in the study. Multi-head self-attention over all 300 timesteps simultaneously can theoretically relate the no_defect value at $t = -299$ directly to the tear score at $t = 0$ which is precisely the long-range dependency that separates the two classes. Yet TST performs essentially identically to the CNN, whose effective receptive field is 12 timesteps. Two factors explain this performance, both supported by direct evidence from the per-fold results.

The first is parameter count relative to available minority class data. Standard multi-head self-attention over a sequence of length $T = 300$ requires learning $\mathcal{O}(T^2)$ attention weights per head. With 8 attention heads, the attention mechanism alone contributes approximately 720,000 weights that must be estimated from the training data. The paper-caused class provides only approximately 1,201 training examples per fold - a severe underspecification. The attention mechanism likely cannot reliably learn which timestep relationships are discriminative for the minority class from so few examples, even with weight decay regularisation. The CNN, by contrast, has a small number of convolutional filter weights and is a far more tractable learning problem relative to the

available training signal.

The second factor is training instability, evidenced directly by the fold 2 anomaly. Folds 1 and 3 converge to threshold 0.400, producing similar profiles and approximately 75% accuracy and 42-46% paper recall. Fold 2 converges to an entirely different solution at threshold 0.170, achieving 62.0% paper recall but FPR 0.334 and only 65.9% accuracy. This divergence is not attributable to fold composition variation that stratified splitting ensures similar class distributions across folds. This likely reflects the one-cycle learning rate schedule interacting differently with the weighted cross-entropy loss in fold 2, producing a probability distribution systematically skewed toward the paper-caused class. TST is sensitive to training trajectory in a way that convolutional models are not, because its attention weights start from a flat uninformed initialisation with no structural prior about which timestep relationships matter, and small differences in the optimisation path can produce qualitatively different solutions.

Together, an overparameterised attention mechanism relative to minority class data, and sensitivity to training trajectory. These two factors likely explain why global self-attention fails to deliver its theoretical advantage on this task and dataset size.

6.3 Why Sequential Memory Helps but is Insufficient: LSTM-FCN

LSTM-FCN (AUC 0.70) improves meaningfully over CNN (0.67) by 0.026 AUC points, confirming that sequential memory adds discriminative value beyond local pattern matching. The LSTM branch processes the 300-frame sequence step by step, maintaining a hidden state that accumulates information across all timesteps. In principle the hidden state at $t = 0$ encodes the cumulative history of the defect score evolution including the gradual deterioration trend of machine-caused breaks that the CNN's local filters structurally cannot access.

However the improvement is modest relative to InceptionTime (+ 0.052 AUC over CNN) and TapNet (+ 0.091 AUC over CNN), and the per-fold results reveal a fundamental limitation. The optimal threshold drops from fold 1 (0.25) to fold 3 (0.11) which is the most pronounced collapse across folds of any model with paper recall rising from 48.4% to 63.8% and FPR rising from 0.21 to 0.33 simultaneously. Mean paper precision across folds is only 0.283 meaning, one in four paper-caused predictions is correct indicating that the LSTM's sequence encoding does not produce sufficient class separation.

The explanation lies in the LSTM's information bottleneck. The hidden state at $t = 0$ is a fixed-dimensional vector that must compress the entire 300-step history into a compact representation. For sequences with slowly evolving signals precisely the machine-caused gradual deterioration pattern, the LSTM's forget gate learns to suppress information from early timesteps as later frames are processed. Over 300 timesteps, the subtle early-stage signal from $t = -299$ is progressively attenuated before reaching the classifier at $t = 0$. The convolutional branch compensates by capturing local patterns, but the concatenated output primarily reflects recent frame dynamics rather than the full global trajectory shape. High paper recall is achievable at aggressive thresholds, but the resulting mean FPR of 0.271 makes LSTM-FCN operationally unreliable.

6.4 Why TapNet performs better amongst all the Deep Learning Models

TapNet (AUC 0.77) achieves the best deep learning result without metadata, outperforming InceptionTime by 0.039 AUC points and CNN by 0.091 AUC points. Two factors jointly explain this advantage: the prototype-based classification mechanism and the weighted cross-entropy loss.

6.4.1 Prototype-Based Classification

TapNet's core architectural insight is to perform classification at the level of complete event trajectory embeddings rather than at the level of local feature activations fed to a linear boundary. The model learns a shared embedding space that is a compact representation of the full 300-frame defect evolution and classifies each event by computing its distance to learned class prototypes.

This is precisely the right abstraction for the web break classification problem. Machine-caused and paper-caused breaks are distinguished by the overall shape of their 300-frame trajectories. Prototype learning groups events by their global trajectory structure, pushing machine-caused embeddings toward the machine prototype and paper-caused embeddings toward the paper prototype. In practice, TapNet learns that a gradual downward no defect slope with no early tear escalation constitutes a machine-caused trajectory, while a late-stage sharp tear peak constitutes a paper-caused trajectory that is exactly the signatures identified in Chapter 3.

The classification profile reflects this advantage. TapNet achieves thresholds of 0.58 - 0.65 across folds, substantially higher than other deep learning models (0.08-0.40), indicating better-separated probability distributions. Mean paper F1 of 0.45 is the highest among all deep learning models without metadata.

6.4.2 Weighted Cross-Entropy Loss

Without class weighting the minority class contributes only 16% of the total gradient signal during training. The paper prototype is therefore estimated from relatively few examples, producing a noisy centroid. The weighted cross-entropy loss assigns a weight of $5.25\times$ to paper-caused events, increasing the effective gradient contribution of the minority class during prototype learning and producing a cleaner classification boundary. The interaction between weighting and architecture is crucial as TST also uses weighted loss but still underperforms substantially because the weighting cannot compensate for TST's overparameterised attention mechanism relative to available minority class data. In TapNet, weighting improves prototype estimation because the architecture is already well-suited to the task.

6.4.3 Why TapNet Still Cannot Close the Gap to XGBoost

Despite these advantages, TapNet’s mean AUC of 0.77 remains 0.093 AUC points below XGBoost full (0.86). The residual gap is due to prototype estimation noise and within-class heterogeneity as paper-caused events vary in onset timing, defect dynamics, and speed of escalation, making a single prototype an imperfect centroid representation. XGBoost’s engineered features capture these variations more directly through explicit numerical summaries, and the addition of metadata provides complementary discriminative information that TapNet cannot access without architectural modification.

6.5 Why Domain Knowledge Extraction performed better: XGBoost vs Deep Learning

The 0.094 AUC gap between XGBoost full (0.86) and TapNet (0.77) is the central empirical finding of this thesis. Both models are trained on the same 14,073 events from the same unified preprocessing pipeline. The gap is attributable to how discriminative information is accessed and how efficiently each model learns to exploit it.

6.5.1 Data Efficiency: The Feature Engineering Advantage

Consider the learning problem each model faces quantitatively. XGBoost receives 80-90 pre-computed features per event, each is a physically meaningful quantity whose relationship to the class label is direct: a slope, a mean, a variance, a cross-camera difference. With 7,505 training events per fold, each feature has roughly 83-94 training examples to establish its decision boundary which becomes a tractable supervised learning problem.

Deep learning models face a qualitatively different challenge. Even TapNet must simultaneously learn what features to extract from the raw 300-frame sequence and how to classify from those features. This compound learning task requires estimating far more parameters from the same 7,505 training events. The linear slope of no defect over 300 frames is a direct mathematical encoding of the discriminative trajectory; XGBoost receives it as a single pre-computed feature, while TapNet must discover this relationship through gradient descent from scratch. Feature engineering is therefore not merely a preprocessing step but it is a domain knowledge injection that dramatically reduces the learning task from discovering relevant representations to estimating decision boundaries in a space of already-meaningful quantities.

The decision tree visualisation in figure 5.5 further confirms this interpretation: the model’s root split on `cam1_max_tear` encodes the physical understanding that a prominent tear peak is the primary indicator distinguishing paper-caused from machine-caused breaks, while metadata fields appear at deeper levels as secondary refinements once the primary signal has been assessed.

6.5.2 The Ablation: Isolating the Metadata Contribution

The XGBoost ablation study and TapNet early fusion experiment jointly reveal the contribution of metadata with precision. Table 6.1 summarises all four configurations.

Configuration	Mean AUC	Std	Mean FPR	Δ AUC
TapNet (no metadata)	0.77	± 0.0019	0.14	-
XGBoost TS-only (no metadata)	0.79	± 0.0026	0.19	-
TapNet + metadata (early fusion)	0.81	± 0.0072	0.13	+0.044
XGBoost full (TS + metadata)	0.86	± 0.0009	0.12	+0.070

Table 6.1: Metadata contribution across both model families. The same eight metadata fields are used in TapNet early fusion and XGBoost full. Δ AUC is relative to the corresponding no-metadata baseline.

Without metadata, XGBoost TS-only (0.79) outperforms TapNet (0.77) by 0.024 AUC points, confirming that explicit time-series feature engineering provides a modest but measurable advantage over prototype embeddings on the camera signal alone. Adding metadata improves both models, but the benefit is substantially larger for XGBoost (+0.07) than for TapNet (+0.04), despite both using the same eight fields. This asymmetry reveals a fundamental difference in how the two architectures exploit structured operational context: gradient boosted trees apply direct categorical splits on metadata values, requiring minimal learning to exploit a feature such as paper supplier identity; TapNet’s early fusion concatenates repeated metadata channels to the time series input and must learn their relevance through gradient descent over 300 timesteps, a substantially less efficient mechanism. The remaining gap between TapNet + metadata (0.8100) and XGBoost full (0.86) is 0.0495 AUC points, driven jointly by superior metadata exploitation and the explicit time-series feature engineering.

6.5.3 Why Metadata is So Informative

The metadata contribution of +0.070 AUC points to XGBoost is larger than the gain from any single architectural improvement among deep learning models. Printer identity, paper grade, and paper supplier encode structural priors about break probability that the camera signal cannot capture: certain paper grades are inherently more susceptible to sudden failure; certain suppliers produce paper with higher variance in tensile strength; certain printer configurations place different mechanical stresses on the web. These properties are invisible to a camera-only model but directly encoded in the metadata. XGBoost can therefore effectively combine two complementary information sources, the camera trajectory signal and the operational context which in a way that deep learning architectures without metadata cannot.

6.6 Why Chronos underperformed: Forecasting Pre-Training on a Classification Task

Chronos achieves a test AUC of 0.66 under the temporal evaluation protocol, underperforming even the plain CNN (0.67 under 3-fold CV). The result is consistent across

both configurations, frozen (0.66) and fine-tuned (0.66) confirming that the failure is not due to insufficient parameter updating but to a fundamental mismatch between the pre-training objective and the task requirements.

Configuration	Val AUC	Test AUC	Thr	FPR	Pap. rec	Pap. F1
Frozen group attn	0.68	0.66	0.16	0.19	0.38	0.33
Fine-tuned group attn	0.71	0.66	0.06	0.20	0.37	0.32

Table 6.2: chronos results under temporal split evaluation (30/6/6 months, single-camera (300,5) input).

Chronos is pre-trained on large-scale time series forecasting corpora using an autoregressive objective: given the first t values of a sequence, predict the next value. This instils strong inductive biases toward local temporal continuity that the model learns to represent sequences in terms of their short-range predictability, smoothness, and periodicity, because these are the properties that determine forecasting accuracy. The web break classification task requires the opposite: the model must detect a global trajectory pattern spanning 300 frames where the discriminative signal is the slope across the entire sequence and the relative timing of the terminal spike. These properties are orthogonal to forecasting quality. A sequence that is locally smooth and highly predictable (gradual machine-caused deterioration) and a sequence that is locally smooth but terminates in a sharp discontinuity (paper-caused sudden failure) have similar local predictability profiles but opposite class labels.

The fine-tuned configuration achieves a validation AUC of 0.71 but test AUC of 0.66, a 0.049-point gap between validation and test performance. This overfitting to the temporal validation period is consistent with the backbone learning surface-level statistics of the validation months rather than generalisable classification features. The extremely low threshold required by the fine-tuned model (0.06 vs 0.16 for the frozen model) indicates that fine-tuning compresses probability scores toward zero for the majority of events, reflecting calibration to the validation period’s class distribution rather than generalisation. The frozen configuration, which makes no claim to adapt the backbone is therefore the more honest evaluation of what chronos’s pre-trained representations contribute to this task, and the answer is modest: 0.66 AUC, equivalent to 16.3 percentage points above chance.

These results suggest a broader principle for applying foundation models to industrial classification tasks: the value of pre-trained representations is contingent on alignment between the pre-training objective and the target task structure. For time series forecasting, the relevant signal is local continuity. For trajectory-level classification, the relevant signal is global shape. These are different enough that far more fine-tuning data would be needed to overcome the pre-training prior than the 8,563 training events available here.

6.7 Why the LLM Underperformed: An Architectural Mismatch

The GPT-4o results provide the clearest diagnosis of all models in this study, precisely because this mode is so well-controlled by the v5 experiment.

	M. prec	M. rec	P. prec	P. rec	P. F1	Acc
v3 (5-shot)	0.91	0.66	0.14	0.45	0.22	64.0%
v5 (12-shot + domain rules)	0.88	0.77	0.09	0.18	0.12	71.0%

Table 6.3: LLM classifier results on 100 held-out events. v5 provided the model with exact XGBoost-derived feature thresholds and domain reasoning rules in addition to more labelled examples.

Configuration v5 gave GPT-4o the exact XGBoost-derived decision boundaries - the precise feature thresholds, slope values, and domain reasoning rules that produce XGBoost's 0.8595 AUC expressed in natural language in the system prompt. The LLM was given, in explicit form, the complete decision logic required to classify correctly. Paper recall did not improve. It dropped from 45.5% to 18.2% and paper F1 dropped from 0.22 to 0.12.

This result isolates the performance with precision: it is not that the LLM lacked information. It lacked the architectural mechanism to act on it. GPT-4o is a next-token predictor trained on natural language. When presented with conditional rules of the form if slope of no_defect is less than threshold and tear sharpness exceeds threshold then paper_caused, it processes these as linguistic patterns rather than executing them as computational conditionals. The model's internal representation of numbers embedded in text does not support faithful threshold comparison or logical branching. Instead, the domain rules in v5 reinforced the statistical prior that 84% of events are machine-caused, causing the model to default to that majority label more aggressively than in v3. The more decision rules provided, the stronger the pull toward the majority class.

This is a fundamental mismatch between the task and the architecture. The web break classification task requires executing quantitative threshold logic on numerical features derived from a time series. XGBoost does this natively through tree splits. Deep learning models do it implicitly through learned weights. A language model has no equivalent mechanism, it approximates numerical reasoning through linguistic pattern matching, which affects performance precisely when the correct answer requires overriding the majority-class prior on the basis of specific numerical conditions. The LLM performance is therefore not a result of the prompt, the encoding, or the information provided, but of the computational substrate to perform the required operation.

6.8 The Role of Class Imbalance

The 5.25:1 class imbalance ratio affects all models, but its impact varies substantially by architecture and training strategy.

Threshold tuning on the validation set is necessary for all models because default thresholds of 0.5 are inappropriate under imbalance and they assume equal class costs and equal class probabilities, neither of which holds here. However, threshold tuning can adjust the operating point of a model but cannot create discriminative ability that the model's representation does not support. The CNN requires a threshold of 0.270 - 0.300 to achieve any paper recall, but the resulting FPR of 0.332 makes it operationally harmful. Threshold calibration exposes the model's probability distribution; it cannot improve the quality of that distribution.

Weighted cross-entropy loss is effective when paired with architectures suited to the task. TapNet with $5.25\times$ weighting achieves substantially better minority class performance than the plain CNN with unweighted loss. But the same weighting applied to TST does not produce commensurate improvement, because TST's attention mechanism cannot effectively learn from the available minority class data regardless of their gradient contribution. Weighting amplifies the learning signal from the minority class; it cannot compensate for architectural unsuitability.

XGBoost's gradient reweighting via `scale_pos_weight` is the most effective imbalance strategy in this study, producing the smallest gap between paper recall and machine recall among all models. This is consistent with the broader finding that explicit feature engineering reduces the compound learning task to boundary estimation in a space of pre-computed meaningful features, making the imbalance problem substantially more tractable: 1,201 minority class examples are sufficient to establish reliable decision boundaries in an 80-90 dimensional pre-computed feature space, but insufficient to learn discriminative trajectory representations from scratch in a 300×10 raw sequence space.

6.9 Industrial Relevance and Deployment Pathway

The results establish that automated web break cause classification is feasible using the data already captured by the facility's existing monitoring system. This section translates the experimental findings into operational terms, quantifies the practical significance of the XGBoost result for the Bertelsmann facility, and describes a concrete deployment pathway that integrates naturally into the existing event recording workflow.

6.9.1 Operational Significance of the XGBoost Result

The XGBoost model provides a consistent, quantitative first-pass classification with mean AUC 0.86, mean paper recall 63.4%, and mean FPR 11.7%. In operational terms, across 4,691 test events per fold the model correctly flags approximately two in three paper-caused breaks as requiring supplier investigation, while generating spurious alerts on only one in nine machine-caused breaks. This represents a substantial improvement over the current process, in which cause attribution relies entirely on operator judgment and is not systematically recorded.

The mean paper F1 of 0.56 and the cross-fold stability (± 0.0009 AUC) indicate that the model generalises reliably across different time periods in the dataset, a prerequisite for production deployment.

6.9.2 Proposed Deployment Architecture

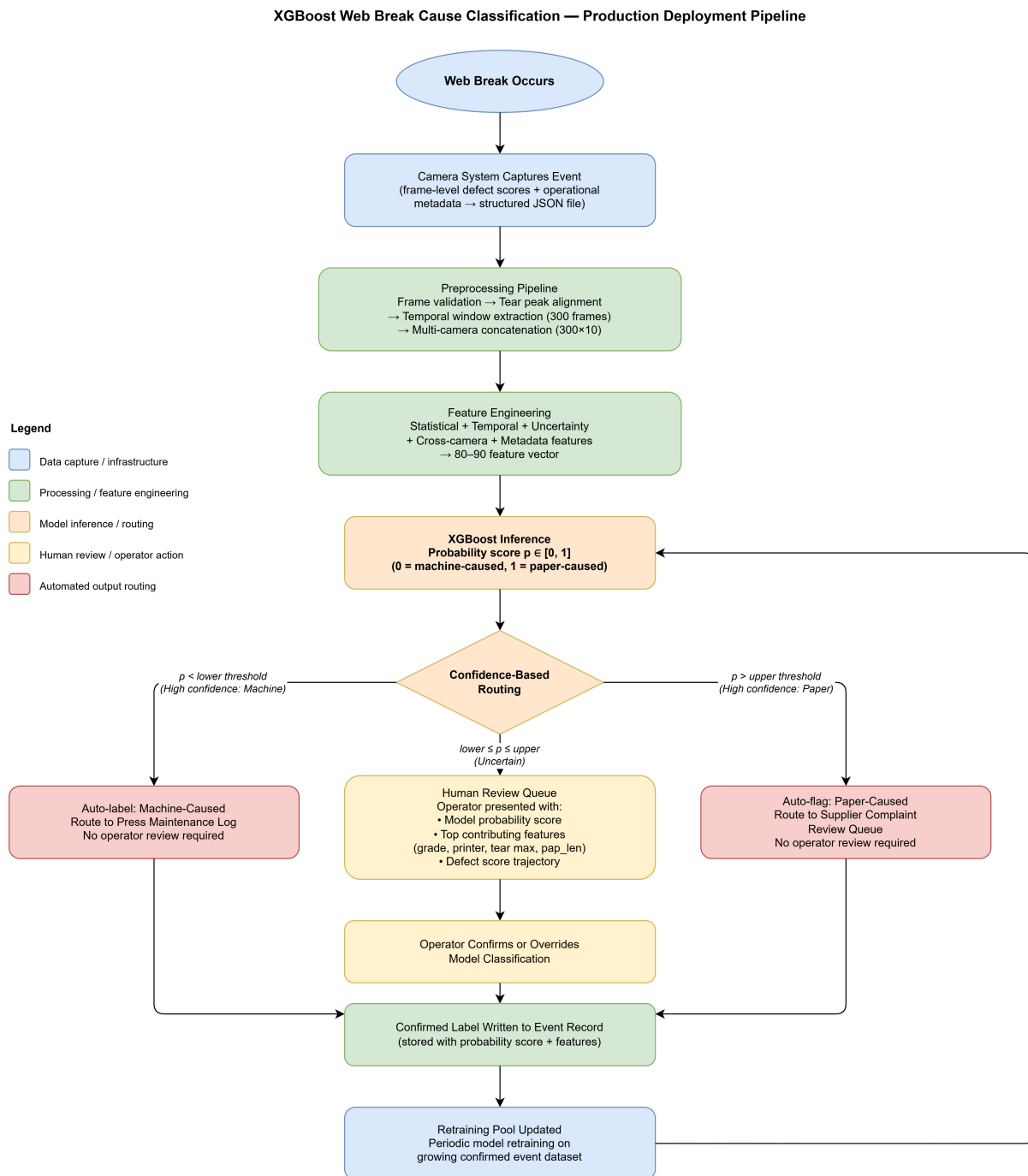


Figure 6.1: Top 30 features by normalised mean gain importance from the XGBoost full model, averaged across three cross-validation folds. Green bars indicate time-series-derived features, orange bars indicate categorical metadata features, and blue bars indicate numeric metadata features.

Figure 6.1 presents the proposed end-to-end deployment pipeline, which integrates directly into the facility's existing event recording workflow without requiring changes to the camera monitoring infrastructure. The pipeline consists of six stages.

When a web break occurs, the camera system captures the event as a structured JSON

file containing frame-level defect scores and operational metadata, exactly as described in Section 3.1. The preprocessing pipeline then applies frame validation, tear peak alignment, temporal window extraction, and multi-camera concatenation to produce the (300, 10) time series representation. The feature engineering step computes the approximately 80 to 90 features described in Section 4.3, covering statistical, temporal, uncertainty, cross-camera, and metadata features. XGBoost inference then produces a continuous probability score between 0 and 1 for each event, where scores close to 0 indicate high confidence in a machine-caused origin and scores close to 1 indicate high confidence in a paper-caused origin.

The probability score then enters the confidence-based routing stage, which divides events into three operational zones using two configurable thresholds. Events below the lower threshold are automatically routed to the press maintenance log as machine-caused, requiring no operator review. Events above the upper threshold are automatically flagged for supplier complaint review as paper-caused. Events falling between the two thresholds are routed to a human review queue, where the operator is presented with the model probability score, the defect score trajectory, and the top contributing features identified in Section 5.9.3, to support an informed classification decision. The confirmed label is then written back to the event record and added to the retraining pool, closing the feedback loop for periodic model updates.

The TapNet with early metadata fusion result (AUC 0.81) suggests that as the confirmed event pool grows through this workflow, a deep learning model may eventually approach XGBoost performance without requiring manual feature re-engineering. XGBoost is therefore proposed as the initial production system, with a transition to a deep learning successor as a longer-term development pathway once sufficient labelled data has accumulated.

6.9.3 Prerequisites Before Deployment

Four conditions must be satisfied before the system is activated in a live production environment.

Prospective validation on events collected after the December 2024 training cutoff is required to confirm that the cross-fold stability observed in historical evaluation holds under genuine temporal extrapolation, and to identify any distribution shift introduced by changes in paper suppliers, press configurations, or camera calibration since the training period.

The two routing thresholds must be calibrated on the prospective validation period based on the facility's operational priorities. The F1-optimised thresholds reported in Chapter 5 provide a starting point, but the final values should reflect the facility's tolerance for missed paper-caused breaks versus operator workload.

The system should be introduced in shadow mode for at least one month before becoming operational, running alongside the existing manual classification process without influencing decisions. This allows operators to build familiarity with the model outputs, identify systematic errors, and establish trust before the system is given decision authority.

A scheduled retraining pipeline must be in place before deployment, retrieving con-

firmed event labels on a defined schedule, retraining XGBoost on the updated dataset, and deploying the updated model only if performance on a held-out validation set meets a minimum AUC threshold. This prevents silent model degradation as the facility's operational conditions evolve over time.

6.10 Limitations and Threats to Validity

The results reported in this thesis are subject to several limitations and threats to validity that must be acknowledged before drawing conclusions about generalisability. These are organised into limitations, which are inherent constraints of the study design and data, and threats to validity, which are factors that could affect the correctness or transferability of the reported findings.

6.10.1 Limitations

Single facility generalisation

All data originates from a single Bertelsmann printing facility. The trained models - particularly XGBoost, which incorporates facility-specific metadata including printer identity, paper grade, and supplier may not generalise to other facilities with different equipment, paper stock, or operational profiles without retraining.

Expert label quality

The binary labels were assigned by facility operators based on their operational judgment. Label noise cases where the true cause is ambiguous or the operator's assessment was incorrect imposes a ceiling on achievable classification performance that cannot be quantified from the available data.

Metadata incompleteness

Some events have missing metadata fields, handled through imputation during preprocessing. Systematic patterns in missingness, for example, if metadata is more likely to be absent for events from a particular printer or time period could introduce bias into the XGBoost model's feature distributions.

6.10.2 Threats to Validity

External validity

Results are specific to the dataset characteristics: 14,073 events, 84/16 class split, a 300-frame temporal window, and the defect score distributions of this facility's camera system. Models trained on this data may not generalise to facilities with different camera systems, different defect category definitions, or substantially different class distributions.

Construct validity

ROC-AUC measures ranking quality of probability outputs and is robust to class imbalance, making it the appropriate primary metric. However it does not fully capture

the operational cost asymmetry between false positives (spurious supplier complaints) and false negatives (missed paper-caused breaks). A deployment decision should be based on the threshold-specific classification report at the intended operating point, not AUC alone.

Chapter 7

Conclusion

7.1 Summary of the Thesis

This thesis addressed the problem of classifying the root cause of paper web breaks in an active web offset printing facility operated by Bertelsmann in Germany. When a web break occurs, the facility must determine whether the break was caused by a fault in the printing machine or a defect in the paper material. This distinction is operationally significant: a paper-caused classification enables formal supplier complaints and cost recovery, while a machine-caused classification signals the need for targeted press maintenance. Currently the attribution is performed manually by press operators post-event, a process that is time-consuming, expertise-dependent, and inconsistent across shifts. This thesis set out to automate that attribution using the data already captured by the facility's existing vision-based monitoring system.

The input to every model is a multivariate time series of frame-level defect probability scores extracted from camera footage recorded in the 300 frames preceding each break, combined with operational metadata including press speed, paper grammage, printer identity, and paper supplier. Three experimental approaches were evaluated on the same dataset of 14,073 labelled break events: end-to-end deep learning across six architectures spanning convolutional, recurrent, attention-based, and prototype-based families; large language model classification via text-encoded symbolic representations of the defect sequences; and a domain-informed feature engineering pipeline using XGBoost. All models were evaluated using event-level stratified 3-fold cross-validation with test AUC as the primary comparison metric.

The central finding is consistent and robust across all three evaluation folds: domain-informed feature engineering combined with XGBoost outperforms all end-to-end deep learning approaches by a margin of 0.09 AUC points (0.86 vs 0.77), with the lowest cross-fold variance of any model (± 0.0009), achieving a mean test AUC of 0.86 ± 0.0009 compared to the best deep learning model TapNet at 0.77 ± 0.0019 . The gap of 0.09 AUC points is consistent across all three folds and cannot be attributed to evaluation variability. A subsequent ablation study revealed that this gap is driven by two components: without metadata, XGBoost TS-only (0.79) outperforms TapNet (0.77) by 0.02 AUC points, confirming that explicit time-series feature engineering provides a modest but measurable advantage; the dominant component is operational metadata, which contributes +0.07 AUC points to XGBoost and +0.04 AUC points

to TapNet via early fusion. The full XGBoost advantage requires both components: domain-engineered time-series features and structured operational metadata exploited through gradient boosted tree splits.

The LLM and foundation model approaches both underperformed the CNN baseline, confirming that generic pre-trained representations do not transfer effectively to this task. Chronos’s failure reflects the mismatch between its forecasting pre-training objective and the global trajectory classification task required. GPT-4o’s failure is architectural: even when provided with the exact XGBoost decision boundaries and feature thresholds in the prompt, the model cannot execute quantitative threshold logic on numerical inputs. It pattern-matches to the majority-class prior rather than applying the conditional rules, causing performance to worsen when more rules are added. Together these results confirm that web break cause classification requires models that can exploit both the global temporal trajectory structure of the camera signal and the structured operational context of the metadata, properties that are most efficiently captured through explicit domain-informed feature engineering rather than generic pre-trained architectures.

7.2 What Did Not Work Well

Not all approaches performed as expected, and these failures carry as much interpretive value as the successes. TST produced the most counterintuitive result: despite global self-attention being theoretically the most capable mechanism for capturing the long-range trajectory dependencies that separate the two break causes, it achieved the same AUC as the plain CNN (0.67), the model with an effective receptive field of only 12 timesteps. The attention mechanism is likely overparameterised relative to the 1,201 minority class training examples available per fold, and its sensitivity to training trajectory produced qualitatively different probability distributions across folds, making it operationally unreliable. ConvTimeNet similarly underperformed its simpler counterpart InceptionTime (0.70 vs 0.73), illustrating that architectural sophistication does not guarantee data efficiency on small imbalanced industrial datasets: its adaptive patch mechanism requires substantially more labelled examples to discover relevant temporal scales than InceptionTime’s fixed multi-scale filters, a disadvantage that outweighs its theoretical expressiveness at this training set size. Fine-tuning Chronos produced no measurable improvement over the frozen configuration (0.66 in both cases), with the higher validation AUC of the fine-tuned model (0.71) failing to transfer to the test period, indicating overfitting to the validation distribution rather than genuine generalisation. The LLM experiments demonstrated a clear ceiling: adding the exact XGBoost-derived decision boundaries and feature thresholds to the GPT-4o prompt worsened rather than improved performance, with paper F1 dropping from 0.22 to 0.12, confirming that the bottleneck is not informational but architectural. The model cannot execute quantitative threshold logic on numerical inputs regardless of how precisely those thresholds are specified.

7.3 Answers to Research Questions

RQ1: Can end-to-end deep learning models applied to preprocessed time series of frame-level defect scores reliably classify web break causes in an industrial web offset printing environment?

Partially. The best deep learning model, TapNet with prototype-based classification and weighted cross-entropy loss, achieves a mean test AUC of 0.77, correctly identifying 51.80% of paper-caused breaks with a mean false positive rate of 0.15 at the F1-optimised threshold. This represents genuine discriminative ability and demonstrates that the temporal defect score sequences contain learnable signal for cause attribution. However it falls short of reliable autonomous classification for operational deployment. The remaining five deep learning architectures, CNN, InceptionTime, ConvTimeNet, LSTM-FCN, and TST, achieve AUCs between 0.67 and 0.73, representing limited discriminative value.

RQ2: Can general-purpose pre-trained models, specifically large language models and time series foundation models effectively classify web break causes without domain-specific training and does the performance stem from information loss during input encoding or from an architectural inability to execute quantitative numerical reasoning?

No, and the failure is architectural rather than informational. Chronos achieves a best test AUC of 0.66, below the CNN baseline and below every task-specific deep learning model trained from scratch. GPT-4o in two few-shot configurations achieves paper recall of 0.46 and 0.18 respectively on a 100-event holdout, with paper F1-scores of 0.22 and 0.12 that are operationally unusable. Both failures have specific, identifiable causes: Chronos's forecasting pre-training objective is misaligned with global trajectory classification; GPT-4o cannot execute the quantitative threshold logic required by this task even when the exact decision rules are provided in the prompt, because its architecture performs linguistic pattern matching rather than numerical computation. Critically, the v5 experiment demonstrated that providing GPT-4o with the exact XGBoost-derived feature thresholds and decision boundaries worsened rather than improved performance, isolating the bottleneck as architectural inability to act on numerical rules rather than information loss during text encoding. Pre-trained general-purpose models offer no advantage over task-specific training on this problem.

RQ3: Does domain-informed feature engineering that is encoding expert knowledge of the defect build-up pattern preceding a break outperform generic end-to-end deep learning approaches on this classification task?

Yes, decisively, and the advantage holds under both fair comparisons. Without metadata, XGBoost achieves a mean test AUC of 0.79, outperforming the best deep learning model without metadata (TapNet, 0.77) by 0.02 AUC points, confirming that explicit time-series feature engineering alone provides a measurable advantage over prototype embeddings. With equivalent metadata access, XGBoost full (0.86) outperforms TapNet with early fusion (0.81) by 0.05 AUC points, demonstrating that the feature engineering advantage persists even when both models receive the same operational context. The engineered features directly encode the physical understanding of the break process established in Chapter 3, the slope of the no_defect score, the late-window tear mean, and the peak sharpness, making the learning task for XGBoost a matter of finding decision boundaries on physically meaningful quantities rather than discovering those

quantities from raw sequences. On this dataset, domain knowledge is a more data-efficient source of discriminative information than gradient descent on raw time series, and this conclusion holds regardless of metadata availability.

RQ4: Does the inclusion of operational metadata improve classification performance, and does its utility vary across different modelling approaches?

Yes, substantially, and its utility depends critically on how it is consumed. The ablation study reveals a metadata contribution of $+0.07$ AUC points to XGBoost, accounting for the majority of its advantage over deep learning. Without metadata, XGBoost TS-only (0.79 ± 0.0026) and TapNet (0.77 ± 0.0019) have an AUC gap of only 0.02 points. Metadata contributes $+0.04$ AUC points to TapNet via early fusion, meaningful, but 1.6 times smaller than the benefit to XGBoost from the same fields. XGBoost exploits metadata through direct categorical splits requiring minimal learning; TapNet must learn metadata relevance through gradient descent over 300 timesteps, a substantially less efficient mechanism. Metadata is highly informative, but its value is maximised when consumed through explicit feature engineering.

7.4 Future Work

Of the directions outlined below, two are of immediate priority. The first is live deployment of the XGBoost pipeline, which represents the natural completion of this thesis and will generate the labelled data on which all subsequent improvements depend. The second is the transition to prototype-based deep learning once sufficient confirmed minority class events have accumulated, which is the most promising research direction identified by the experimental findings. The remaining directions are longer-term and contingent on the outcomes of these two.

Full deployment and prospective evaluation.

The proposed deployment architecture described in Section 6.9.2 defines a concrete, facility-ready pipeline with confidence-based routing, operator review integration, and a scheduled retraining loop. Deploying this pipeline on live event data is the most operationally urgent next step: it would provide real-world validation of the cross-fold stability observed in historical evaluation, generate the continuously growing pool of confirmed labelled events required for model maintenance, and complete the transition from research to production. The four prerequisites identified in Section 6.9.3 i.e prospective validation, threshold calibration, shadow-mode operation, and a retraining pipeline must be satisfied before the system is given decision authority.

Transition to prototype-based deep learning as labelled data grows.

The deployment pipeline generates confirmed paper-caused labels at every break event, steadily expanding the minority class pool beyond the 2,251 training examples available in this study. The ablation study established that the XGBoost advantage over TapNet is driven jointly by superior metadata exploitation and the compound learning task that deep learning models must solve from limited minority class data. As the confirmed paper-caused pool grows, the prototype estimation noise that currently limits TapNet will reduce. A natural transition point exists once the paper-caused class reaches a threshold that supports stable prototype learning, estimated to require at minimum

several thousand confirmed events based on the data efficiency analysis in chapter 3. This transition would eliminate the manual feature re-engineering burden required to maintain XGBoost as operational conditions evolve.

Extension to multiple facilities and press types.

All models were trained and evaluated on data from a single Bertelsmann facility. Evaluating cross-facility generalisation, either through direct transfer or domain adaptation, would establish whether the discriminative patterns identified here are universal to web offset printing or facility-specific.

Continuous learning and model adaptation over time.

The dataset spans three and a half years across changing paper suppliers, printer configurations, and operational conditions. Periodic retraining or online learning approaches would allow the model to adapt to distribution shift without requiring full retraining from scratch.

Bibliography

- [AST⁺24] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, et al. Chronos: Learning the language of time series. *Transactions on Machine Learning Research*, 2024. arXiv:2403.07815.
- [AVH⁺09] A. Alzghoul, A. Verikas, M. Hållander, M. Bacauskiene, and A. Gelzinis. Data mining techniques for web break prediction in offset printing. In *Proceedings of the Industrial Conference on Data Mining (ICDM)*, 2009.
- [B⁺02] P. Bonissone et al. Predictive models for paper breaks. *Industrial Applications*, 2002.
- [B⁺16] P. Branco et al. A survey of predictive modelling under imbalanced distributions. *ACM Computing Surveys*, 2016.
- [BLB⁺17] Anthony Bagnall, Jason Lines, Aaron Bostrom, James Large, and Eamonn Keogh. The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Mining and Knowledge Discovery*, 31(3):606–660, 2017.
- [C⁺02] N. V. Chawla et al. Smote: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 2002.
- [C⁺24] X. Cheng et al. Convtimenet: Adaptive convolutional networks for time series. *IEEE Transactions*, 2024.
- [CBNKL18] Maximilian Christ, Nils Braun, Julius Neuffer, and Andreas W. Kempa-Liehr. Time series FeatuRe extraction on basis of scalable hypothesis tests (tsfresh – a python package). *Neurocomputing*, 307:72–77, 2018.
- [CG16] T. Chen and C. Guestrin. XGBoost: a scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [D⁺23] L. Dias et al. Machine learning for web break prediction in tissue mills. *Applied Sciences*, 2023.
- [F⁺20] H. I. Fawaz et al. Inceptiontime: Finding alexnet for time series classification. *Data Mining and Knowledge Discovery*, 2020.
- [F⁺24] A. Farahani et al. Benchmarking time series classification in manufacturing. *IEEE Access*, 2024.

- [Faw06] T. Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 2006.
- [G⁺23] N. Gruver et al. Large language models are zero-shot time series forecasters. *NeurIPS*, 2023.
- [HG09] H. He and E. A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 2009.
- [HU02] D. T. Hristopulos and T. Uesaka. Web tension dynamics in paper machines. *Journal of Pulp and Paper Science*, 2002.
- [JWM⁺24] M. Jin, S. Wang, L. Ma, et al. TIME-LLM: time series forecasting by reprogramming large language models. *ICLR*, 2024.
- [Kip01] Helmut Kipphan, editor. *Handbook of Print Media: Technologies and Production Methods*. Springer, 2001.
- [KMDH19] Fazle Karim, Somshubra Majumdar, Houshang Darabi, and Samuel Harford. Multivariate LSTM-FCNs for time series classification. *Neural Networks*, 116:237–245, 2019.
- [LY⁺20] Yaguo Lei, Bin Yang, Xinwei Jiang, Feng Jia, Naipeng Li, and Asoke K. Nandi. Applications of machine learning to machine fault diagnosis: a review and roadmap. *Mechanical Systems and Signal Processing*, 138:106587, 2020.
- [RFL⁺21] A. P. Ruiz, M. Flynn, J. Large, M. Middlehurst, and A. Bagnall. The great multivariate time series classification bake off: a review and experimental evaluation of recent algorithmic advances. *Data Mining and Knowledge Discovery*, 2021.
- [Ues04] T. Uesaka. Web breaks in pressroom: a review. Technical report, FSCN, Mid Sweden University, 2004.
- [V⁺24] Nicola Oreste Pinciroli Vago et al. Deep learning for industrial failure prediction. *Engineering Applications*, 2024.
- [Web13] Web Offset Champions Group. Web break prevention and diagnosis: Best Practice Guide No. 2, 2013.
- [WYO17] Z. Wang, W. Yan, and T. Oates. Time series classification from scratch with deep neural networks. *International Joint Conference on Neural Networks*, 2017.
- [Z⁺20] X. Zhang et al. Tapnet: Multivariate time series classification with attentional prototypical network. *AAAI*, 2020.
- [Z⁺21] G. Zerveas et al. A transformer-based framework for multivariate time series representation learning. *KDD*, 2021.